

FR-2

Overview Dashboard

หน้าหลัก ภาพรวมรายรับ รายจ่าย ยอดคงเหลือ
เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ
คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-2 ละเอียดทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-2 Overview Dashboard เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

i ภาค ก • พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข • FR-2 Overview Dashboard

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผล การออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปปต์นำเสนอ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบล่างแบบนี้:

src/domain/runway.ts

ตัวอย่าง · บรรทัด 17-23

```

17  *   so the status is "no_spending" and no date is shown.
18  */
19  import { addDays, bkkDayKey, daysBetween } from './dates';
20  import { confirmedOnly } from './summary';
21  import type { Transaction } from './types';
22
23  export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด **Ctrl+P** (Mac: **Cmd+P**) แล้วพิมพ์ข้อความตัวหน้าที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น `src/domain/runway.ts` ก็แตะเปิดได้เหมือนกัน

i ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดยังไ

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

!ระวัง

จุดที่คนมักเข้าใจผิด หรือนึกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดทีละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างขีดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจสอบโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอต่่ายจากเว็บเวอร์ชันจริงในโหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไปเล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดอย่างไร	2
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	10
MindPay คืออะไร และทีมเราสร้างอะไร	11
ปัญหาที่ทีมเราอยากแก้	11
6 ฟังก์ชันหลัก (Functional Requirements)	11
หน้าต่างแอปโดยรวม	12
ใช้แอปได้ 3 แบบ	13
ตัวเลขของโปรเจกต์	13
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	14
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันอย่างไร	15
ระบบมี 4 ส่วนใหญ่	15
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	15
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนถึงเลขบนหน้าหลักเปลี่ยน	16
ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ	20
เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	22
กล่องเครื่องมือของทีมเรา	22
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	22
เอาโค้ด MindPay มาไว้ในเครื่อง	22
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	23
คำสั่งที่ใช้บ่อย	26
รู้จักหน้าจอส VS Code	26
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	27
ส่วนเสริม (Extensions) ที่แนะนำ	28
ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	28
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	28
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย	29
โปรแกรมคืออะไร	29
1. ค่า (value) และชนิดของค่า	29
2. ตัวแปร: กล่องที่มีชื่อ	29
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	30
4. เชื้อไข if: ถ้า... ให้ทำ...	30
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	31
6. Array: รายการของหลายค่า	31
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	33
8. วงซ้ำ (loop)	34
9. ข้อความ (string) ที่ใช้บ่อย	34
10. คณิตศาสตร์ที่ใช้	34
แบบฝึกหัด A4	34

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay	36
ทำไมต้อง TypeScript	36
การบอชนิดข้อมูล (type annotation)	36
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	36
ชนิดข้อมูลแบบพิเศษที่เจอบ่อย	38
import และ export: แคร่โค้ดระหว่างไฟล์	39
งานที่ต้องรอ: async, await และ Promise	40
จัดการความผิดพลาด: try, catch, throw	40
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	41
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	41
Regular expression (regex) อ่านยังงี้	43
แปลงสตางค์กลับเป็นข้อความบาท	44
React: วาดหน้าจอจากข้อมูล	46
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	46
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	46
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	48
Hook ที่ใช้ในโปรเจกต์	49
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	51
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังงี้	52
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	54
React Native คืออะไร	54
Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น	55
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	55
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	57
แถบเมนูล่าง: (tabs)/_layout.tsx	61
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	62
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	64
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	66
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	66
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	66
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	67
แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน	68
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	70
ฐานข้อมูลคืออะไร	70
Supabase ให้อะไรทีมเราบ้าง	70
Migration: ไฟล์สร้างฐานข้อมูล	71
Row Level Security (RLS): หัวใจความปลอดภัย	74
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	76
แอปต่อ Supabase ยังงี้: src/data/supabase.ts	76
การสมัครและเข้าสู่ระบบ	78
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	79
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	84
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังงี้	87
AI ในแอปคืออะไร	87
ทำไมแอปไม่เรียก AI เอง	87
เส้นทางของคำถาม 1 ข้อถึงน้องกล่า	87

ฝั่งแอป: src/services/coach.ts	88
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบัสกัถ	89
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	91
เลือก AI ตัวไหน: Claude หรือ Gemini	94
ส่งคำขอไปที่ Claude: callClaude	96
ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)	99
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	99
UX/UI: การออกแบบหน้าจอและ Design System	100
หลักการออกแบบของ MindPay	100
Design System คืออะไร	100
สี: Green + Gold	100
ความอ่านง่ายของสี (WCAG) และเทสที่บังคับ	102
ตัวอักษร	104
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	104
Accessibility: ทุกคนใช้ได้	107
แอนิเมชัน: สวยแต่ไม่กินแบต	107
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	107
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	109
ทำไมต้องเขียนเทส	109
Unit test: ตรวจสอบฟังก์ชันทีละตัว	109
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	111
Manual test: ทดสอบบนมือถือจริง	114
Traceability: ผูก Requirement → โค้ด → เทส	115
ตรวจสอบคุณภาพอื่น ๆ	115
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	117
Git: ระบบบันทึกประวัติ	117
ใช้ Git ประจำวัน	117
ประวัติของ MindPay UU GitHub	118
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push	118
Secrets UU GitHub	123
ไฟล์เอกสารใน repo	123
6 FR ทำงานร่วมกันอย่างไร และคำถามระดับระบบ	125
สายข้อมูลของทั้งระบบ	125
ใครใช้ของใคร	125
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	125
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	126
คำศัพท์รวม	127
FR-2 OVERVIEW DASHBOARD	129
FR-2 ในหน้าเดียว	130
อยู่ตรงไหนในแอป	130
3 ข้อที่ต้องจำ	130
ปัญหาที่ฉันทอยากแก้	132
เป้าหมายที่ฉันทตั้งไว้ตอนออกแบบ	132
Requirement และเงื่อนไขผ่าน	133

ตัวเลขทุกตัวบนหน้าหลักมาจากไหน	134
ทัวร์หน้าจอ: หน้าหลักส่วนบน	136
ทัวร์หน้าจอ: กลางหน้าหลัก	137
ทัวร์หน้าจอ: ภาพรวม (เลือกช่วงได้ 5 แบบ)	138
เปลี่ยนช่วง: ตัวเลขเปลี่ยนยังไง	139
ทัวร์หน้าจอ: กราฟ 7 วัน จบเดือน และรายการล่าสุด	141
ทัวร์หน้าจอ: ปฏิทินการใช้จ่าย	143
ทัวร์หน้าจอ: สรุปเดือน (เรื่องเล่า 7 หน้าจอ)	145
วิธีใช้	146
ทัวร์หน้าจอ: เมื่อข้ามเที่ยงคืน	147
หลักการคำนวณ: ยอดคงเหลือ	148
ตัวอย่างที่ 1: ข้อมูลตัวอย่างในภาพหน้าจอ	148
ตัวอย่างที่ 2: แบบร่างไม่นับ (TC-09)	148
หลักการคำนวณ: ช่วงเวลา และการสรุปตามช่วง	149
ขั้นตอนของ summarizeRange (ทำทุกครั้งทีแต่ละปุ่มช่วง)	149
ตัวอย่างที่ 3: ทำตาม TC-10 ด้วยมือ (วันนี้ = 27 ก.ย. 2569)	149
ความยาวแถบหมวด	150
หลักการคำนวณ: กราฟ 7 วัน และจบเดือน	151
dailyTotals: 7 ช่องเสมอ	151
ความสูงของแท่ง	151
จบเดือนนี้	151
หลักการคำนวณ: ปฏิทินไล่สี่	152
ตัวอย่างที่ 4: TC-70 (ก.ย. 2569, วันนี้ = 15 ก.ย.)	152
ตัวอย่างที่ 5: ปฏิทินกันยายนในภาพหน้าจอ	152
หลักการคำนวณ: สรุปเดือน	154
ทำไมต้องเทียบ "จำนวนวันเท่ากัน"	154
เมื่อโหล์การ์ดสรุปเดือนขึ้นบนหน้าหลัก (recapToOffer)	154
อ่านโค้ด: src/domain/summary.ts ทั้งไฟล์	156
ส่วนที่ 1: หัวไฟล์ confirmedOnly และ computeBalance (บรรทัด 1–17)	156
ส่วนที่ 2: รูปร่างของผลสรุป (บรรทัด 19–38)	157
ส่วนที่ 3: totalsByCategory — รวมตามหมวด (บรรทัด 40–64)	158
ส่วนที่ 4: summarizeRange — สรุปตามช่วง (บรรทัด 66–82)	159
ส่วนที่ 5: dailyTotals — ยอดรายวันสำหรับกราฟ (บรรทัด 84–108)	160
ส่วนที่ 6: groupByDay และ monthExpense (บรรทัด 110–134)	161
อ่านโค้ด: ช่วงเวลาและการแสดงวันแบบไทย (dates.ts บรรทัด 53–157)	163
ส่วนที่ 1: ตารางช่วงเวลา (บรรทัด 53–89)	163
ส่วนที่ 2: ชื่อเดือนและวันแบบไทย (บรรทัด 91–110)	164
ส่วนที่ 3: ข้อความช่วงเวลาใต้ปุ่ม (บรรทัด 112–124)	165
ส่วนที่ 4: เดือน และป้ายวันแบบสัมพัทธ์ (บรรทัด 126–157)	166
อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 1 — ส่วนประกอบและข้อมูล	168
ส่วนที่ 1: คอมเมนต์และ import (บรรทัด 1–52)	168

ส่วนที่ 2: ป้ายสถานะ และชนิดปุ่มลัด (บรรทัด 54–62)	170
ส่วนที่ 3: ปุ่มลัด ActionTile (บรรทัด 64–141)	170
ส่วนที่ 4: ไฟจอตต่อเนื่อง StreakPill (บรรทัด 143–178)	172
ส่วนที่ 5: ข้อมูลของหน้าหลัก (บรรทัด 180–212) — ส่วนสำคัญของ FR-2	173
ส่วนที่ 6: เอฟเฟกต์เลื่อน และน้องกล่า (บรรทัด 213–250)	175
อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 2 — ส่วนบนและการ์ดเขียว	177
ส่วนที่ 7: หัวหน้า น้องกล่า และการ์ด "มีอะไรใหม่" (บรรทัด 252–289)	177
ส่วนที่ 8: การ์ดเหรียญใหม่ และสกินใหม่ (บรรทัด 291–347)	178
ส่วนที่ 9: การ์ดสถานะต่าง ๆ (บรรทัด 349–368)	179
ส่วนที่ 10: การ์ดเขียว — ยอดคงเหลือ (บรรทัด 370–443)	180
ส่วนที่ 11: วันนี้ใช้ได้ และแบบร่าง (บรรทัด 445–469)	182
ส่วนที่ 12: ปุ่มลัด สรุปเดือน และกระปุก (บรรทัด 471–527)	183
อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 3 — ส่วนภาพรวมของ FR-2	185
ส่วนที่ 13: ปุ่มช่วง และเงินเข้า/ออก (บรรทัด 529–556)	185
ส่วนที่ 14: การ์ดรายจ่าย/รายรับตามหมวด (บรรทัด 557–581)	186
ส่วนที่ 15: กราฟ 7 วัน และงบเดือน (บรรทัด 583–605)	187
ส่วนที่ 16: รายการล่าสุด (บรรทัด 607–629)	187
อ่านโค้ด: กราฟ charts.tsx ทั้งไฟล์	189
ส่วนที่ 1: import และแท่งที่โตขึ้น RisingBar (บรรทัด 1–42)	189
ส่วนที่ 2: DayBars — เตรียมสเกล (บรรทัด 44–58)	190
ส่วนที่ 3: DayBars — วาด (บรรทัด 60–130)	191
ส่วนที่ 4: CategoryBars (บรรทัด 132–166)	193
อ่านโค้ด: ปฏิทินการใช้จ่าย SpendCalendar.tsx ทั้งไฟล์	195
ส่วนที่ 1: สีและ props (บรรทัด 1–43)	195
ส่วนที่ 2: หัวปฏิทินและตารางวัน (บรรทัด 44–95)	196
ส่วนที่ 3: คำอธิบายสี และแถบวันที่เลือก (บรรทัด 96–125)	197
อ่านโค้ด: src/domain/recap.ts ทั้งไฟล์	200
ส่วนที่ 1: รูปร่างของสรุปเดือน (บรรทัด 1–38)	200
ส่วนที่ 2: recapMonths และ monthRecap ครั้งแรก (บรรทัด 40–72)	201
ส่วนที่ 3: เทียบเดือนก่อน และผลลัพธ์ (บรรทัด 74–106)	202
ส่วนที่ 4: recapToOffer (บรรทัด 108–120)	203
ส่วนที่ 5: ข้อมูลปฏิทิน (บรรทัด 122–145)	204
ส่วนที่ 6: spendCalendar — สร้างตารางเดือน (บรรทัด 147–179)	205
อ่านโค้ด: หน้าสรุปเดือน app/recap.tsx	207
ส่วนที่ 1: import ค่าคงที่ และตัวช่วย (บรรทัด 1–58)	207
ส่วนที่ 2: เตรียมข้อมูล (บรรทัด 60–80)	208
ส่วนที่ 3: หน้า 1–4 (บรรทัด 81–179)	209
ส่วนที่ 4: หน้า 5–7 (บรรทัด 180–279)	211
ส่วนที่ 5: การเปลี่ยนหน้า และการวาด (บรรทัด 281–365)	214
อ่านโค้ด: ข้อมูลกลางที่ FR-2 ใช้ (AppProvider)	217
ส่วนที่ 1: useMoney — ตัวเลขเงินที่ทุกหน้าใช้ร่วมกัน (บรรทัด 442–459)	217
ส่วนที่ 2: loadAll — โหลดข้อมูลและกันข้อมูลปน (บรรทัด 127–149)	218
ส่วนที่ 3: ยอดยกมาจากฐานข้อมูล (repo.ts บรรทัด 180–216)	219
ส่วนที่ 4: เทียบคืนเวลาไทย (บรรทัด 182–212)	220
ส่วนที่ 5: ดึงลงเพื่อรีเฟรช (บรรทัด 359–366)	221

อ่านโค้ด: PeriodSummary – ภาพรวมย่อในหน้าสแกน	223
การทดสอบ FR-2	225
Unit test: วันไทยและช่วงเวลา (TC-03, TC-08)	225
Unit test: ภาพรวม (TC-09, TC-10, TC-11)	226
Unit test: สรุปเดือนและปฏิทิน (TC-69, TC-70)	227
E2E: ทดสอบเหมือนคนใช้จริง	229
เช็กलिस्टทดสอบเองบนมือถือ	231
FR-2 เชื่อมกับ FR อื่นยังไ	232
ทำไมฉันออกแบบแบบนี้	233
ข้อจำกัด และสิ่งที่จะทำต่อ	234
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	235
แบบฝึกหัดเช็กความเข้าใจ	237
เฉลย	237
สคริปต์นำเสนอ 1 นาที และคำศัพท์	238
คำศัพท์ของ FR-2	238

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คุยกันยังไง

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไง
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

- นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ
- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
 - **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
 - **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

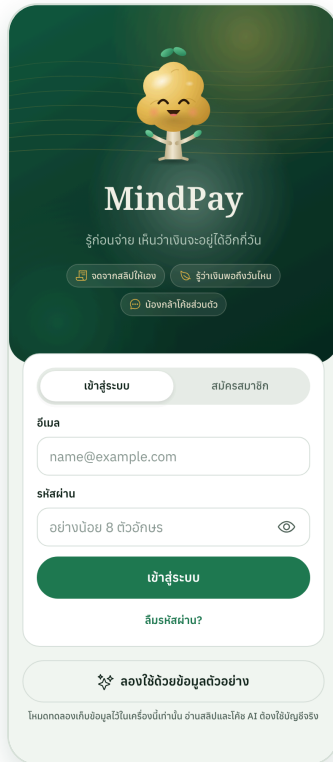
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ทองบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขียบบนหน้าหลัก

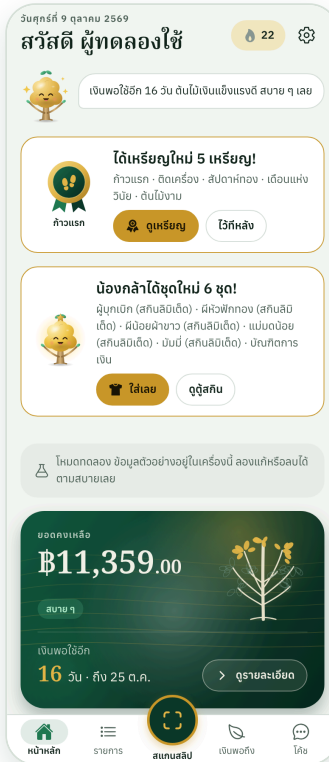
★ จำให้ได้: ทุก FR ต่อกันเป็นสายเดี่ยว

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จุดเอง พุดจกด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนพัง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดตามไปด้วย

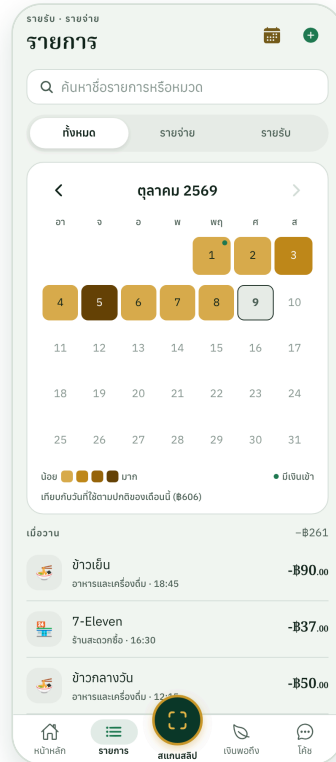
หน้าตาแอปโดยรวม



หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"



หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), บอกลำไย พุดสรุป



แท็บรายการ (FR-1): รายการ ทั้งหมด ค้นหา กรองรายรับ/รายจ่าย



สแกนสลิป (FR-4): อ่านสลิปแล้ว
สรุปตามช่วงเวลา

พุดจดย่อยรายการ (FR-3): พุดประโยคเดียว ได้
หลายรายการ

เงินพอถึง (FR-6): 16 วัน ถึง
ประมาณ 25 ต.ค.

ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน `src/app/(tabs)/_layout.tsx` ซึ่งจะอ่านทีละบรรทัดใน un A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไ้	ต่างกันยังไ้
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแคสเลอร์ ทั้งหมดเอง (บนเว็บต้องกดเลือกรูป สลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest-apk	สแกนแคสเลอร์อัตโนมัติ พุดจดย่อยด้วยไมค์ น้องกล้าพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและไอซ์ได้
- **โหมดทดลอง** แตะ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียวไม่ต้องสมัคร ใช้ตอนฟรีเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700

บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับ
เทสต์)

157

unit test ผ่านทั้งหมด

145

ข้อตรวจในเบราวเซอร์ (E2E) ผ่านทั้งหมด

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไมค์ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสต์ในเบราวเซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอ)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสต์ (ไฟล์เทสต์อีกประมาณ 2,000 บรรทัด)

≈ เปรียบแอปเป็นร้านอาหาร

หน้าจอ (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโถงกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บทต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ตรวจ ยังไม่นับในยอดเงิน
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล้า	มาสคอตของแอป เป็นต้นกล้าจากต้นไม้เงิน และเป็นโค้ช AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ต (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขได้เฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วคือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ **จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)**

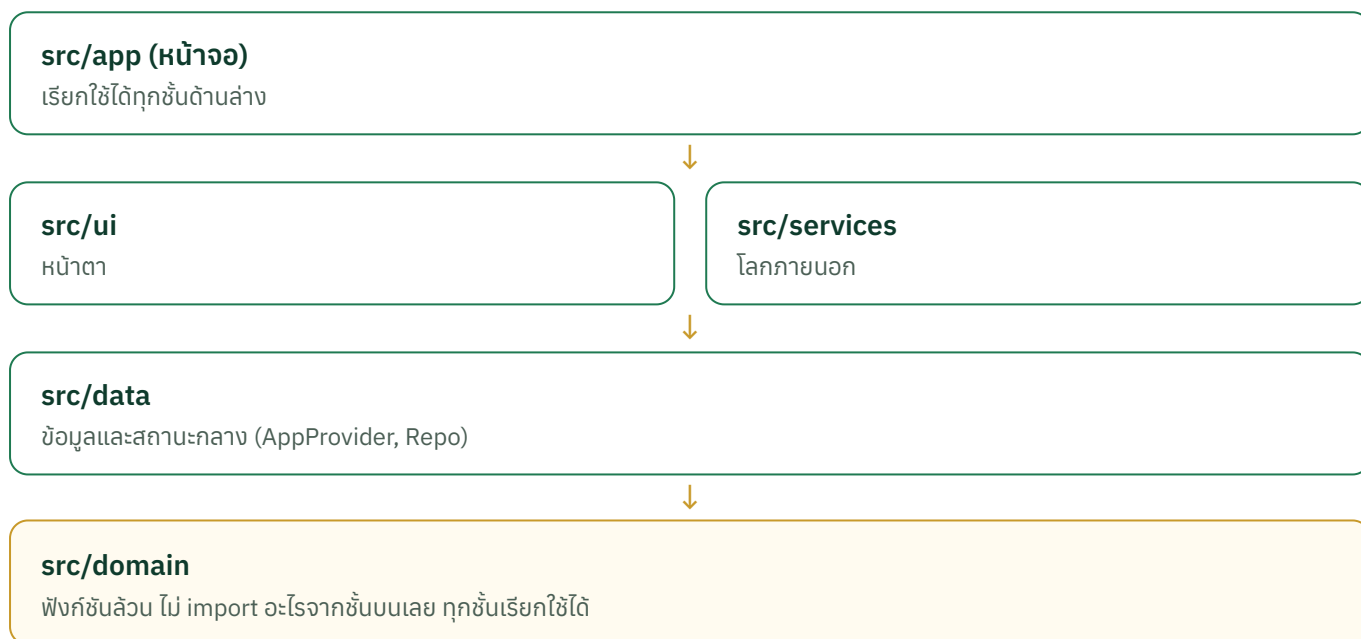
- เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
- เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
- สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
- กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	<code>src/app/transaction.tsx</code> หน้าจอดรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	<code>src/ui/components.tsx</code> , <code>src/ui/theme.ts</code>

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
กฎและการคำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แกลเลอรี่รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แค่นี้เดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน src/domain ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามมันไ้฿85" แล้วกด "บันทึกรายการ"

1. หน้าจอดรายการ [src/app/transaction.tsx](#)

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง [src/data/AppProvider.tsx](#)

`addTx` ส่งต่อให้ Repo บันทึก รวบรวมกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTx`)



3. Repo [src/data/repo.ts](#)

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase [supabase/migrations/20260927000000_init.sql](#)

ตรวจกฎ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน [src/data/AppProvider.tsx](#)

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละชั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนี้งูแค่ว่ามันต่อกันยังงี้)

ชั้น 1: หน้าจอดรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
```

```

113     await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ต)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนใต้ช่อง และหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหนวนรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการกดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385     async addTx(input) {
386         const r = need();
387         const tx = await r.insert(input);
388         if (repoRef.current !== r) return tx;
389         noteLocal(tx.id, tx);
390         setTxs((list) => [tx, ...list]);
391         return tx;
392     },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยาบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need () จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้ลบบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีการเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้ากำลังโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxS ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts	บรรทัด 217–222
<pre> 217 async insert(input) { 218 const { data, error } = await db.from('transactions').insert(toRow(input)).select().single(); 219 if (isUniqueViolation(error)) throw new DuplicateSlipError(); 220 if (error) throw error; 221 return fromRow(data as TxRow); 222 }, </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217	

บรรทัด	ความหมาย
218	อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
219	ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
220	error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
221	แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx	บรรทัด 442–459
<pre> 442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */ 443 export function useMoney() { 444 const { profile, txs, carrySatang, goals, today } = useApp(); 445 return useMemo(() => { 446 const opening = profile?.openingBalanceSatang ?? 0; 447 const floor = profile?.runwayFloorSatang ?? 50_000; 448 // Money from before the loaded window (over ~13 months ago) still counts. 449 const balance = computeBalance(opening + carrySatang, txs); 450 const average = averageDailyExpense(txs); 451 // Money in savings goals is set aside: it does not count as money to spend. 452 const reserved = reservedSatang(goals); 453 const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang: average.averageSatang }); 454 const drafts = txs.filter((t) => t.status === 'draft'); 455 return { balance, average, runway, drafts, reserved, floor }; </pre>	

```

456 // `today` makes the runway and averages start over at midnight.
457 // eslint-disable-next-line react-hooks/exhaustive-deps
458 }, [profile, txs, carrySatang, goals, today]);
459 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
443	useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
444	ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกลูก และวันนี้
445	useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
446–447	ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
449	คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
450–453	คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุก และ Money Runway (FR-6)
454	หารายการแบบร่างที่รอตรวจ (FR-4)
455	ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
458	รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่มเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จากรายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลบอกอย่างหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ [src/data/AppProvider.tsx](#) เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มือถือหารูปใหม่ในแคเลเลอริ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปไหนไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจว่าเข้าสู่ระบบจริง ตรวจโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ตรวจ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ตรวจ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขึ้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รันแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรันภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รัน SQL ใส่กุญแจ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

1. **VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows ตี๊ก "Add to PATH" และ "Open with Code" ไว้ด้วย)
2. **Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ node -v ต้องขึ้น v22 หรือสูงกว่า
3. **Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย git --version
4. **บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไข (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
5. **Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminalคำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน package.json แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ node_modules (ประมาณ 1-3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น .env ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ copy .env.example .env)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

.env.exampleบรรทัด 1-6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → .env.example:1

บรรทัด	ความหมาย
1-4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บท A9) ชื่อที่ขึ้นต้นด้วย EXPO_PUBLIC_ คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งลัดอะไรบ้าง

package.jsonบรรทัด 1-59

```

1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
17    "expo-haptics": "~57.0.3",
18    "expo-image-manipulator": "~57.0.20",
19    "expo-image-picker": "~57.0.20",
20    "expo-linear-gradient": "~57.0.2",
21    "expo-linking": "~57.0.11",
22    "expo-media-library": "~57.0.5",
23    "expo-router": "~57.0.23",
24    "expo-speech": "~57.0.3",
25    "expo-speech-recognition": "^57.1.0",
26    "expo-splash-screen": "~57.0.9",
27    "expo-sqlite": "~57.0.3",
28    "expo-status-bar": "~57.0.1",
29    "expo-system-ui": "~57.0.4",
30    "expo-updates": "~57.0.23",
31    "react": "19.2.3",
32    "react-dom": "19.2.3",
33    "react-native": "0.86.3",
34    "react-native-safe-area-context": "~5.7.0",
35    "react-native-screens": "~4.26.0",
36    "react-native-svg": "15.15.4",
37    "react-native-web": "~0.21.0"
38  },
39  "devDependencies": {
40    "@types/react": "~19.2.2",
41    "eslint": "^9.0.0",
42    "eslint-config-expo": "~57.0.2",
43    "playwright": "^1.56.0",
44    "typescript": "~6.0.3",
45    "vitest": "^5.0.2"
46  },
47  "scripts": {
48    "start": "expo start",
49    "android": "expo start --android",
50    "ios": "expo start --ios",
51    "web": "expo start --web",
52    "test": "vitest run",
53    "typecheck": "tsc --noEmit",
54    "lint": "expo lint",
55    "e2e:web": "EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist && EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56    "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-video dark android && node e2e/demo-video.mjs dist demo-video light web"
57  },

```

```
58 "private": true
59 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → package.json:1

บรรทัด	ความหมาย
2–3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5–38	dependencies แพ็กเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6–7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลอรีทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (บท A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขอมจ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าวกราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงผลบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้ลง: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)

บรรทัด	ความหมาย
47-57	scripts คำสั่งลัด พิมพ์ <code>npm run</code> ชื่อ (หรือ <code>npm test</code> , <code>npm start</code>) แทนคำสั่งยาว ๆ ทางขวา
58	<code>private</code> = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
<code>npm run web</code>	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก้โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
<code>npx expo start</code>	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
<code>npm test</code>	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
<code>npm run typecheck</code>	ให้ TypeScript ตรวจสอบข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
<code>npx expo lint</code>	ตรวจสอบรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
<code>npm run e2e:web</code>	build เว็บแล้วรันเทสต์ในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำ หรือค้าง = รอได้ หรือลบโฟลเดอร์ `node_modules` แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ `.env` (บรรทัด 4) · ห้ามใช้ `npm install` ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ `npx expo install` ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์ คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น <code>computeRunway</code> จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Terminal	ด้านล่าง (Ctrl+` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แถบข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบล่างสุด	บอก branch ของ Git, บรรทัด/คอมมิตที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อท้ายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หากทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเมนต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าจอหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด Ctrl+Shift+X แล้วค้นชื่อ กด Install

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ <code>npm run lint</code>
Expo Tools	ช่วยเติมคำและตรวจไฟล์ <code>app.json</code> / <code>eas.json</code>
Vitest	มีปุ่ม ▶ ข้างเกสต์แต่ละข้อ กดรันทีละข้อได้
GitLens (ไม่บังคับ)	ขีดบรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน commit อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ท้ายบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (`node_modules/typescript`) ตามไฟล์ `tsconfig.json`

ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แต่ชื่อไฟล์บนหัวโค้ดค่อนข้างยาวก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (Ctrl+Shift+F) กด F12 ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน `npm test` หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. Ctrl+P พิมพ์ `index.tsx` เลือกไฟล์ใน `src/app/` (tabs) (หน้าจอหลัก)
2. Ctrl+F ค้น `useMoney` แล้วกด F12 ที่คำนั้น จะแดงไป `src/data/AppProvider.tsx` บรรทัด 443
3. ที่บรรทัด 453 คลิก `computeRunway` แล้วกด F12 จะไปที่ `src/domain/runway.ts` บรรทัด 72 ซึ่งเป็นสูตรของ FR-6
4. กด Shift+F12 ที่ชื่อฟังก์ชัน จะเห็นว่ามีเกสต์เรียกใช้ใน `src/domain/__tests__/domain.test.ts` ด้วย
5. กด Alt+← สองครั้ง ย้อนกลับไปหน้าจอหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (บท A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React อยู่นะ (บท A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000;           // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0;                   // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500;            // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- `const` ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ `let` เฉพาะตอนต้องเปลี่ยนจริง ๆ
- `50_000` คือ 50000 เครื่องหมาย `_` ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง `//` คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน `/* ... */` และ `/** ... */` เป็นคอมเมนต์หลายบรรทัด (แบบ `/**` VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ camelCase:** คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น `balanceSatang`, `averageDailyExpense` คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น `RUNWAY_CAP_DAYS`

★ กฎการตั้งชื่อของทิมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากับ / ไม่เท่ากับ (แบบเคร่ง ชนิดต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'ลดรายการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่ขึ้นใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก `src/data/AppProvider.tsx:447` ซึ่งใช้ทั้ง ?. และ ??

`src/data/AppProvider.tsx`

บรรทัด 447-447

```
447 const floor = profile?.runwayFloorSatang ?? 50_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:447`

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เงื่อนไข if: ถ้า... ให้ทำ...

`if / else`

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 if (spendable <= 0) {
2   // ทำส่วนนี้เมื่อเงื่อนไขจริง
3 } else if (averageSatang <= 0) {
4   // ไม่จริง ถ้าเงื่อนไขนี้จริง ทำส่วนนี้
5 } else {
6   // ไม่ตรงอะไรเลย ทำส่วนนี้
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

src/domain/runway.ts

บรรทัด 83-88

```
83 if (spendable <= 0) {
84   return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85 }
86 if (averageSatang <= 0) {
87   return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:83

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด่านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ พารามิเตอร์ (ของที่ส่งเข้า) และ return ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 // แบบที่ 1: function
2 function toBaht(satang: number): number {
3   return satang / 100;
4 }
5
6 // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกัน เขียนสั้นกว่า
7 const toBaht2 = (satang: number) => satang / 100;
8
9 toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไม่ใช่ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปทำอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] ค้นด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0];           // 12000 (ตัวแรก)
3 amounts.length;       // 4 (มีกี่ตัว)
4 amounts.includes(0);   // true

```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกแต่ละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยัง (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```

9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คัดเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คำนวณเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว

บรรทัด	ความหมาย
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชั้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
<code>const { a, b } = obj;</code>	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น <code>const { balance, runway } = useMoney();</code>
<code>{ ...obj, status: 'confirmed' }</code>	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
<code>[tx, ...list]</code>	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ในบท A2)
<code>{ kind, title }</code>	shorthand	ย่อจาก <code>{ kind: kind, title: title }</code>

! ทำไมโค้ดชอบสร้าง object ใหม่แทนการแก้ไขเดิม

React (un A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ไขใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอก็ไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วนซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;       // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++)
ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดู un A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
Math.floor(20.69)	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
Math.round(680.2857)	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
Math.max(0, -500)	0 (ตัวที่มากกว่า)	กันเงินที่ใช้ได้ติดลบ
Math.min(7, 3)	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
Math.abs(-85)	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า `profile` เป็น `null` ค่าของ `profile?.runwayFloorSatang ?? 50_000` คืออะไร
2. `[100, 200, 300].filter((x) => x > 150).map((x) => x / 100)` ได้อะไร
3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 `computeBalance` คืนกี่สตางค์
4. `const t2 = { ...t, title: 'ข้าวผัด' }` แล้ว `t.title` เปลี่ยนไหม

i เฉลย

- 1) 50000 · 2) `[2, 3]` (กรองเหลือ 200, 300 แล้วหาร 100) · 3) $50,000 + 100,000 - 30,000 = 120,000$ สตางค์ (แบบร่างไม่นับ) · 4) ไม่เปลี่ยน `t2` เป็น object ใหม่ `t` ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatnag` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเพี้ยนตอนผู้ใช้งานใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด ("`strict`": `true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ตักกลับทันทีที่โต๊ะ ตักว่าไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกลบออก เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น number เอาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์นี้สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
```

```

11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
22   note: string | null;
23   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24   occurredAt: string;
25   source: TxSource;
26   status: TxStatus;
27   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28   slipRef: string | null;
29   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30   slipImageHash: string | null;
31   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32   ocrConfidence: number | null;
33   /** Fields the slip reader was unsure about, shown to the user during review. */
34   reviewFlags: string[];
35   createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด ความหมาย

- 1-7 คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3
- 9 **union type:** ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย | แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที

บรรทัด	ความหมาย
11–12	สถานะของรายการ: 'draft' (แบบร่าง ยังไม่นับ) หรือ 'confirmed' (ยืนยันแล้ว นับในยอด) คอมเมนต์อ้างอิงถึง Charter ของทีมว่าผลจากสลิปเป็นแบบร่างก่อน
14	รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) นับเป็น manual
16	interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร"
17	id สหสัมพันธ์ของรายการ (ฐานข้อมูลสร้างให้)
18	kind ใช้ชนิด TxKind จากบรรทัด 9
19	amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind)
20	categoryKey สหสัมพันธ์ เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts)
21–22	ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้)
23–24	เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย)
25–26	ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12
27–30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31–32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จดเองเป็น null
33–34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้นำมาตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38–39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทนโคช 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล่าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43–53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เงินสำรองของ FR-6 (บรรทัด 49), จบต่อเดือน (ไม่ตั้งก็ได้ = null), โทนโคช และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใส่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "ชื่อฉัน x เป็นชนิด Y" (ใช้น้อย ๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โด้ระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';
10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด	ความหมาย
6	นำ Storage มาจากไฟล์ ./storage (จุด = โฟลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์)
7	นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules
9-13	../domain/... = ถอยออกไป 1 โฟลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้
14	import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป)

11 type GoalInput ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว

ฝั่งไฟล์ต้นทางต้องเขียน export หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน export default ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจออกจาก default export)

งานที่ต้องรอ: async, await และ Promise

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครั้งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 async function save() {           // async = ฟังก์ชันนี้มีงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอจนจับได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }
```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ Promise.all ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออม พร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131-133

```
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
      r.listGoals().catch(() => [] as SavingsGoal[])]);
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:131

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แะใส่ตัวแปร p, list, g ส่วน .catch(() => []) ทำงานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ ใน Edge Function ก็ใช้ Promise.allSettled อ่านสลิป 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: try, catch, throw

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 try {
2   await addTx(input);           // ลองทำ
3   toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) {                  // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5   toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally {                    // ทำเสมอ ไม่ว่าสำเร็จหรือพลาด
7   setBusy(false);
8 }
```

throw คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น DuplicateSlipError (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

class ... extends Error = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา super(...) ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอลูกได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2      → 0.30000000000000004
19.99 * 100    → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกหลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผลบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ amount_satang bigint)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน parseBahtToSatang ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กร่อนจ่ายของ FR-6, กระทบยอด) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value).
6  */
7 export const MAX_SATANG = 100_000_000_000;
8
9 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
10 export function toAsciiDigits(s: string): string {
11   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
12 }
13
14 /**
15  * Parse what a user (or a slip) wrote into satang.
16  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
17  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
18  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
19  * Returns null for anything that is not a positive amount with at most 2 decimals, or above
20  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).

```

```

21 export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?:
    boolean } = {}): number | null {
22   if (input === null || input === undefined) return null;
23   const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero &&
    satang === 0) ? satang : null);
24   if (typeof input === 'number') {
25     if (!Number.isFinite(input) || input < 0) return null;
26     return ok(Math.round(input * 100));
27   }
28   let cleaned = toAsciiDigits(input)
29     .replace(/฿|บาท|THB|baht/gi, '')
30     .replace(/,/g, '');
31   if (/^\d+,\d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32   else if (cleaned.includes(',')) {
33     if (!/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/.test(cleaned)) return null;
34     cleaned = cleaned.replace(/,/g, '');
35   }
36   if (!/^\d+(\.\d{0,2})?$/.test(cleaned)) return null;
37   const [whole, frac = ''] = cleaned.split('.');
38   if (whole.length > 12) return null;
39   return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:1`

บรรทัด	ความหมาย
5–6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกัน กับค่าที่อ่านผิดใหญ่เกินจริง)
8–11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0e50) จะได้ 0–9
13–20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีมือถือบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ถูกต้อง
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24–27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา $19.99 \times 100 = 1998.999\dots$)
28–30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1–2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32–35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่ขึ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่ขึ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง

บรรทัด	ความหมาย
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท $\times 100$ + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์ชันจริง

ผู้ใช้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" $\rightarrow$ 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรรทัด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แค่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่มีก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙

สัญลักษณ์	แปลว่า
g, i (ท้าย /)	g = หากจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1-3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1-2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตริงกลับเป็นข้อความบาท

src/domain/money.ts

บรรทัด 42-63

```

42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
45   const negative = satang < 0;
46   const abs = Math.abs(Math.round(satang));
47   // Whole baht only: round to the nearest baht (฿99.99 reads ฿100, not ฿99).
48   const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49   const frac = abs % 100;
50   const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51   const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52   return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "฿1,234.50"; with sign: "+฿1,234.50" / "-฿1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}฿${body}`;
62   return `${satang < 0 ? '-' : ''}฿${body}`;
63 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:42

บรรทัด	ความหมาย
42-43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45-46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47-48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ ฿99.99 จึงแสดงเป็น ฿100 ไม่ใช่ ฿99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมาค้นหลักพันด้วย regex (หาตำแหน่งที่ด้านหลัมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51-52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55-63	formatBaht เติม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i ฅลจรง

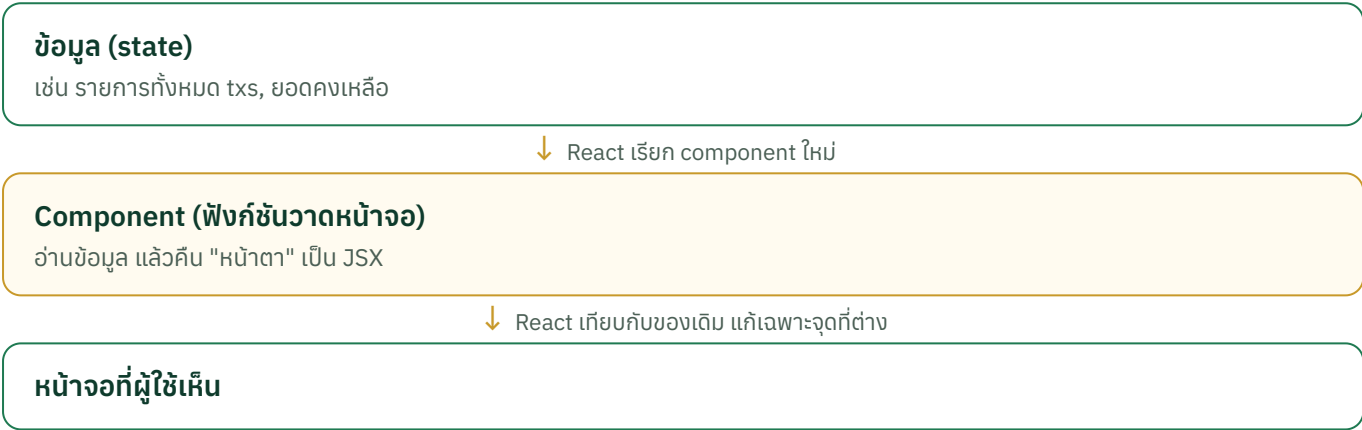
```
formatBaht(123450) → "฿1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·  
formatBaht(9999, { decimals: false }) → "฿100"
```

React: วาดหน้าจจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจอ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจอวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจอก็ขัดกันเอง React เปลี่ยนวิธีคิด: เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจอหน้าตาแบบนี้" แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน



Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจอทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {  
2   return <Text>สวัสดี {name}</Text>;  
3 }  
4  
5 // เรียกใช้ที่อื่น:  
6 <Hello name="มินท์" />    // แสดง "สวัสดี มินท์"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แท็กเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แกะเอาช่อง name ออกมา
<Hello name="มินท์" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

```

36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
40   h1: fonts.serif,
41   h2: fonts.sansBold,
42   h3: fonts.sansSemi,
43   body: fonts.sans,
44   small: fonts.sans,
45   micro: fonts.sansMedium,
46   label: fonts.sansSemi,
47 };
48
49 export function T({
50   v = 'body',
51   color,
52   style,
53   children,
54   numberOfLines,
55   center,
56   selectable,
57   accessibilityRole,
58 }: {
59   v?: Variant;
60   color?: string;
61   style?: StyleProp<TextStyle>;
62   children: ReactNode;
63   numberOfLines?: number;
64   center?: boolean;
65   selectable?: boolean;
66   accessibilityRole?: 'header' | 'text' | 'alert';
67 }) {
68   const theme = useTheme();
69   const size = v === 'label' ? type.micro : type[v];
70   return (
71     <Text
72       accessibilityRole={accessibilityRole}
73       numberOfLines={numberOfLines}
74       selectable={selectable}
75       style={[
76         size,
77         {
78           fontFamily: VARIANT_FONT[v],
79           color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80           textAlign: center ? 'center' : undefined,
81         },
82         // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below
4.5:1).
83         v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ??
theme.inkSoft },
84         style,
85       ]}
86     >
87       {children}
88     </Text>
89   );

```

บรรทัด	ความหมาย
36	ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
38–47	ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
49–57	ประกาศ component ชื่อ T พร้อมแกะ props ออกมา <code>v = 'body'</code> แปลว่าถ้าไม่ระบุระดับ ใช้ body
58–67	ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole บอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน
68	เรียก <code>useTheme()</code> เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ป่าทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จันข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: `<T v="h3">จดรายการ</T>` หรือ `<T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>`

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอลoadใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ `useState`

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount    = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
```

```

36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
    Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
    Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก้ไขรายการเดิม ใช้ประเภทเดิม ไม่จําเป็นดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จําเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) useState<TxKind> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจํานวนเงิน เก็บเป็น ข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก้ไขรายการเดิม เติมค่าเดิมให้
38–39	ชื่อรายการ และหมวด
40	จํานวนที่ผู้ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเดาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จํา state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จําค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจําผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney () คำนวณยอดเงินและ Runway

Hook	ทำอะไร	ตัวอย่างใน MindPay
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })
204     .catch(() => {});
205   const sub = AppState.addEventListener('change', (s) => {
206     if (s === 'active') check();
207   });
208   return () => {
209     clearTimeout(timer);
210     sub.remove();
211   };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array วางท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา

บรรทัด	ความหมาย
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกลั๊กตกกายและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกลูกาพิคาเดิม แล้วตั้งใหม่ให้ปลูกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ) msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งล่าสุดจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่หน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แชรข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบคลุมทั้งแอปไว้ใน `src/app/_layout.tsx` (un A7) ตัวสำคัญที่สุดคือ **AppProvider**

`src/data/AppProvider.tsx`

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the
96     companion). */
97 export function useAppMaybe() {
98   return useContext(AppContext);
99 }
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);

```

```

113 const todayRef = useRef(today);
114 const repoRef = useRef<Repo | null>(null);
115 /** The account whose data is loaded (or being loaded): prevents loading the same account
    twice. */
116 const activeUserRef = useRef<string | null>(null);
117 const handledLinks = useRef(new Set<string>());
118 /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119 const loadGen = useRef(0);
120 /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121 const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หิบบข้อมูลกลาง ถ้าเรียกนอก Provider จะ error กันที (กันลืมครอบ)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คลาวด์หรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โปรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกออม
108–110	ข้อความ error ตอนโหลด, คำสั่งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	รวันใหม่
113–121	ค่าที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme();           // สีของธีม
3   const { txs, addTx } = useApp();     // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6
7   return (
8     <Screen>

```

```

9      <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้ซัก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุปบท A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าจอ (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (un A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสิ้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (bussกัถ 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนกับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)

style	ความหมาย
gap: 8	ระยะห่างระหว่างลูกแต่ละตัว
padding / margin	ระยะห่างด้านใน / ด้านนอก
borderRadius	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

src/ui/components.tsx

บรรทัด 189–203

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }: {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap },
203     style] >{children}</View>;
204 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

บรรทัด 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวตั้ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่ามากมาย **Expo** ห่อทุกอย่างไว้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที่ Expo ให้กับเรา:

- **Expo Go + dev server:** แก่โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หวัข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ที่เราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน `src/app/` ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
<code>_layout.tsx</code>	-	โครงทั้งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
<code>welcome.tsx</code>	<code>/welcome</code>	สมัคร เข้าสู่ระบบ ลืมรหัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
<code>onboarding.tsx</code>	<code>/onboarding</code>	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, บง)	FR-2, FR-6
<code>(tabs)/_layout.tsx</code>	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
<code>(tabs)/index.tsx</code>	<code>/</code>	หน้าหลัก	FR-2 (+FR-6)
<code>(tabs)/transactions.tsx</code>	<code>/transactions</code>	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
<code>(tabs)/runway.tsx</code>	<code>/runway</code>	แท็บเงินพอถึง	FR-6
<code>(tabs)/coach.tsx</code>	<code>/coach</code>	แท็บโค้ชน้องกล้า	FR-5
<code>transaction.tsx</code>	<code>/transaction</code>	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
<code>voice.tsx</code>	<code>/voice</code>	พูดจด	FR-3
<code>scan.tsx</code>	<code>/scan</code>	สแกนสลิป	FR-4
<code>drafts.tsx</code>	<code>/drafts</code>	สลิปรอยืนยัน	FR-4
<code>settings.tsx</code>	<code>/settings</code>	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
<code>goals.tsx, goal.tsx</code>	<code>/goals, /goal</code>	กระปุกออม	FR-6
<code>recap.tsx</code>	<code>/recap</code>	สรุปเดือนแบบเรื่องเล่า	FR-2
<code>achievements.tsx, skins.tsx</code>	<code>/achievements, /skins</code>	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
<code>whatsnew.tsx</code>	<code>/whatsnew</code>	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น `(tabs)` เป็น "กลุ่ม" โมไฟล์ในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- `index.tsx` คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ `/`)
- ไฟล์ `_layout.tsx` ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
<code>router.push('/scan')</code>	เปิดหน้าจอสแกนซ้อนขึ้นมา (กดกลับได้)
<code>router.push({ pathname: '/transaction', params: { kind: 'income' } })</code>	เปิดหน้าจอรายการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
<code>useLocalSearchParams()</code>	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
<code>router.navigate('/runway')</code>	ไปที่แท็บเงินพอถึง
<code>goBack()</code>	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: `_layout.tsx` โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

`src/app/_layout.tsx`

บรรทัด 1-51

```
1 import { useFonts } from 'expo-font';
2 import { Stack } from 'expo-router';
3 import * as SplashScreen from 'expo-splash-screen';
4 import { StatusBar } from 'expo-status-bar';
5 import { useCallback, useState, type ReactNode } from 'react';
6 import { Platform, View } from 'react-native';
7 import { SafeAreaProvider } from 'react-native-safe-area-context';
8 import {
9   Anuphan_400Regular,
10  Anuphan_500Medium,
11  Anuphan_600SemiBold,
12  Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
```

```

35   });
36   return (
37     <SafeAreaProvider>
38       <WebFrame>
39         <CelebrationProvider>
40           <AppProvider>
41             <ToastProvider>
42               <AutoScanProvider>
43                 <Navigator ready={fontsLoaded || !!fontError} />
44               </AutoScanProvider>
45             </ToastProvider>
46           </AppProvider>
47         </CelebrationProvider>
48       </WebFrame>
49     </SafeAreaProvider>
50   );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด	ความหมาย
1–22	นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
24–25	สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
27	export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
28–35	โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
36–50	ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลูลอง → ข้อมูลกลาง (AppProvider) → ข้อความแจ้งเตือน (toast) → สแกนสลิปอัตโนมัติ (FR-4) → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
43	ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังกี่ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53–73

```

53  /** On a computer browser, show the app at phone width in the middle of the screen. */
54  function WebFrame({ children }: { children: ReactNode }) {
55    const theme = useTheme();
56    if (Platform.OS !== 'web') return <>{children}</>;
57    return (
58      <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg,
59        '#000000', 0.55) : theme.surfaceAlt }}>
60        <View
61          style={{
62            flex: 1,
63            width: '100%',
64            maxWidth: 440,
65            backgroundColor: theme.bg,
66            overflow: 'hidden',
67            boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
68          }}
69        >
70          {children}
71        </View>
72      </View>
73    );
74  }

```

```

70     </View>
71   </View>
72 );
73 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

53–54 WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือไว้ตรงกลางจอ

56 ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร ค้นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)

58 พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด

59–68 กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <>
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp=
{inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>
104           <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105         </Stack.Protected>
106         <Stack.Protected guard={needsOnboarding}>
107           <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108         </Stack.Protected>
109         <Stack.Protected guard={inApp}>
110           <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />

```

```

111         <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112         <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113         <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114         <Stack.Screen name="achievements" />
115         <Stack.Screen name="skins" />
116         <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117         <Stack.Screen name="goals" />
118         <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119         <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120         <Stack.Screen name="drafts" />
121         <Stack.Screen name="settings" />
122     </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo ใหม่ โปรไฟล์ตั้งค่าเสร็จใหม่ และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว้างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้าไฟล์) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่กับหน้าเก่า กดกลับก็หิบบหน้าบนออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, พื้นตามืด, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตูกุญแจ: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พุดจด, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ ผลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center',
137       justifyContent: 'center' }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3}
139         color={theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         />
166     </View>
167   </View>
```

```

164         <Ionicons name="scan" size={26} color={theme.heroAccent} />
165     </Pressable>
166 </Animated.View>
167 </View>
168     <Text style={[type.micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4
    ]}]>สแกนสลิป</Text>
169 </View>
170     {routes.slice(half).map((r, i) => tab(r, i + half))}
171 </View>
172 );
173 }
174
175 export default function TabsLayout() {
176 // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177 useKlaSync();
178 return (
179     <Tabs tabBar={props} => <TabBar {...props} /> screenOptions={{ headerShown: false }}>
180         <Tabs.Screen name="index" />
181         <Tabs.Screen name="transactions" />
182         <Tabs.Screen name="runway" />
183         <Tabs.Screen name="coach" />
184     </Tabs>
185 );
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด	ความหมาย
134	วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย
135–169	ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระเพื่อม (PulseRing) 3 รอบ ตอนเปิดแอป และกระเพื่อมตลอดระหว่างสแกนอัตโนมัติ
144–147	แตะแล้วมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน
170	วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา
175–186	component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แทนของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้บันทึกการกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app
  settings and demo data.
3  * Phone: SQLite-backed store (survives
  app restarts).
4  * Web: see storage.web.ts (browser
  localStorage).

```

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the
  browser's localStorage, wrapped so it
  never throws. */
2 export const Storage = {
3   async getItem(key: string):
  Promise<string | null> {
4     try {

```

```

5  */
6  import KV from 'expo-sqlite/kv-store';
7
8  export const Storage = {
9    getItem: (key: string) =>
10     KV.getItem(key),
11    setItem: (key: string, value: string) =>
12     KV.setItem(key, value),
13    removeItem: (key: string) =>
14     KV.removeItem(key),
15  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.ts:1

```

5      return
6      window.localStorage.getItem(key);
7    } catch {
8      return null;
9    },
10    async setItem(key: string, value:
11      string): Promise<void> {
12      try {
13        window.localStorage.setItem(key,
14          value);
15      } catch {
16        // Private mode or storage full:
17        // keep working in memory.
18      }
19    },
20    async removeItem(key: string):
21      Promise<void> {
22      try {
23        window.localStorage.removeItem(key);
24      } catch {
25        // ignore
26      }
27    },
28  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์โหมดส่วนตัวอาจห้ามเก็บข้อมูล คู่มือฉบับนี้มีทั้งหมด 5 คู่มือ:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้เลือกใช้รูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

src/services/speech.ts

บรรทัด 1-24

```

1  /**
2   * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech

```

```

3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15 interface NativeSpeech {
16   start(options: Record<string, unknown>): void;
17   stop(): void;
18   abort(): void;
19   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
20   isRecognitionAvailable(): boolean;
21   addListener(event: string, listener: (e: never) => void): Subscription;
22 }
23
24 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ `requireOptionalNativeModule` = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ให้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดไม่มีโมดูลนี้ แต่ยังได้รับโค้ดใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะเด็งปิดแบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไมค์

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง

โค้ดใน src/ (TypeScript + React Native)



เว็บ

`expo export --platform web` รวมโค้ดเป็นไฟล์ JavaScript + HTML ในโฟลเดอร์ dist แล้ววางบน GitHub Pages

APK (Android)

`expo prebuild` สร้างโปรเจกต์ Android จาก app.json แล้ว Gradle build เป็นไฟล์ .apk

EAS Update

ส่งเฉพาะโค้ด JavaScript ใหม่ แอปที่ลงแล้วโหลดเองตอนเปิดแอป ใช้ตอนเปิดครั้งถัดไป

ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที่เรา push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": {"policy": "appVersion"}	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	สหัสแอป com.socratesandskeletons.mindpay

ค่าใน app.json	ความหมาย
"blockedPermissions"	บล็อกสิทธิ์กล้อง วัตถุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอร์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็อัปเดตทุกครั้งที่เปิดแอป
"web": {"output": "single"}	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ `app.config.js` เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ `/Socrates-and-Skeletons - /` บน GitHub Pages จึงต้องตั้ง `baseUrl` แต่ห้ามใส่ค่านี้นในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

1. อ่านคอมเมนต์บนสุดของไฟล์ก่อน ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
2. ดูบรรทัด `import` บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า `import` จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
3. กด `Ctrl+Shift+O` ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
4. หาฟังก์ชันที่มี `export` นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี `export` คือผู้ช่วยภายใน
5. อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
6. หาเทสต์ของมัน กด `Shift+F12` ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย Satang	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น bkk เช่น <code>bkkDayKey</code> , <code>bkkToIso</code>	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ <code>'2026-10-09'</code>	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่า กันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
<code>useApp()</code>	หยิบข้อมูลกลาง (โปรไฟล์ รายการ คำสั่ง <code>addTx</code> ฯลฯ)
<code>useMoney()</code>	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย Runway แบบร่าง
<code>useTheme()</code> แล้วตามด้วย <code>theme.ink</code> , <code>theme.primary</code>	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<code><T v="h3"></code> , <code><Money satang={...}></code> , <code><Card></code> , <code><Button></code>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
<code>space.lg</code> , <code>radius.md</code>	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
<code>toast({ message })</code>	ข้อความแจ้งเตือนสั้น ๆ ด้านล่างจอ
<code>router.push('/xxx')</code>	เปิดหน้า <code>src/app/xxx.tsx</code>
<code>accessibilityLabel="..."</code>	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม

เห็นอะไร	แปลว่าอะไร
status === 'confirmed' หรือ confirmedOnly(tx)	นับเฉพาะรายการที่ยืนยันแล้ว
Reveal, usePressSpring, useReduceMotion	แอนิเมชัน (ค่อย ๆ โฟกัส, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส TC-12	รหัสเทสต์ ตรงกับในเอกสาร docs/TRACEABILITY.md
ไฟล์ชื่อ xxx.web.ts	ฉบับเว็บของไฟล์ xxx.ts
ชื่อแบบ snake_case เช่น amount_satang	ชื่อคอลัมน์ในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน src/data/repo.ts

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

src/domain/dates.ts

บรรทัด 1-51

```

1  /**
2   * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3   *
4   * Everything the user sees ("today", "this week", the date printed on a slip)
5   * is a Bangkok calendar day, so all grouping and range logic goes through
6   * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7   */
8  import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }

```

```

36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /\d{1,2}:\d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: ทุกอย่างที่ใช้ให้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลา ฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที ($86,400,000$)
13–17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบ สากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20–24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้ง นาฬิกาใน AppProvider (un A6)
26–28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30–35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือน ข้ามปีให้เอง
37–40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42–46	bkkToIso : เอาวันไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48–51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไร ไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะ อะไร	src/app/ชื่อหน้า.tsx (ตารางใน un A7)

อยากรู้	เปิดไฟล์
ตัวเลขนี้คิดยังไง	src/domain/: src/domain/summary.ts (ยอด), src/domain/runway.ts (Runway), src/domain/insights.ts (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	src/data/repo.ts , src/data/AppProvider.tsx
ตารางในฐานข้อมูลมีคอลัมน์อะไร	supabase/migrations/20260927000000_init.sql
AI ได้รับคำสั่งอะไร	supabase/functions/coach/index.ts , supabase/functions/parse-slip/index.ts
สี ขนาด ฟอนต์	src/ui/theme.ts , docs/DESIGN_SYSTEM.md
ฟีเจอร์นี้ตอบ requirement ข้อไหน มีทดสอบอะไร	docs/TRACEABILITY.md
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	docs/AI_USAGE_LOG.md
build / deploy อัตโนมัติ	.github/workflows/*.yaml

★ ถ้าอ่านแล้วงง

ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แถบ Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชีต แต่ละชีตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชั่น (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มินท์	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มินท์	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรกับเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล ลืมรหัสผ่าน

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ทให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ผังเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

โปรเจกต์ของทีมอยู่ภูมิภาค Singapore (ใกล้ไทย ตอบเร็ว) และใช้แพ็คเกจฟรี

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทีมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low
17   line, default #500
```

```

17 monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang >
0),
18 coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')),
-- FR-5
19 onboarded boolean not null default false,
20 created_at timestamptz not null default now(),
21 updated_at timestamptz not null default now()
22 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1–7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยคุณดูแลสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โพรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (฿500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมาฐานข้อมูลปฏิเสธเอง
17	งบท่อเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโค้ช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20–21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql

บรรทัด 24–53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
ocr_confidence <= 1)),

```

```

41 review_flags text[] not null default '{}',
42 created_at timestamptz not null default now(),
43 updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจสอบรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_uidx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_uidx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตางค์ ต้องมากกว่า 0
32	รหัสหมวด
33–34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอดรายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามือถือ 2 เครื่องสแกนสลิปใบเดียวกันพร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql

บรรทัด 55–65

```

55 -- -----
56 -- ai_usage: counts calls to the AI so one account cannot run up the bill.
57 -- Written only by the Edge Functions (service role), readable by the owner.
58 -- -----
59 create table if not exists public.ai_usage (
60   id bigserial primary key,
61   user_id uuid not null references auth.users (id) on delete cascade,

```

```

62 kind text not null check (kind in ('slip', 'coach')),
63 created_at timestamptz not null default now()
64 );
65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at
desc);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กันบัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ล็อกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครมีก็เดินเข้าห้องได้ แต่ละคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะโยมบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql

บรรทัด 67-87

```

67 -- -----
68 -- Row Level Security
69 -- -----
70 alter table public.profiles enable row level security;
71 alter table public.transactions enable row level security;
72 alter table public.ai_usage enable row level security;
73
74 drop policy if exists "own profile read" on public.profiles;
75 create policy "own profile read" on public.profiles
76   for select using (auth.uid() = id);
77 drop policy if exists "own profile update" on public.profiles;
78 create policy "own profile update" on public.profiles
79   for update using (auth.uid() = id) with check (auth.uid() = id);
80
81 drop policy if exists "own transactions" on public.transactions;
82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด	ความหมาย
70-72	เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง
74-76	policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รันไฟล์ซ้ำได้
77-79	แก้โปรไฟล์ได้เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ (กันแก้ id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น

บรรทัด	ความหมาย
81–83	รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง
85–87	ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรับตัวตรวจ Security Advisor ของ Supabase แล้วแก้ไขไม่เหลือคำเตือนเรื่องตาราง ไฟล์ `harden.sql` เปลี่ยน `auth.uid()` เป็น `(select auth.uid())` เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql

บรรทัด 1–25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7   new.updated_at := now();
8   return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17   for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21   for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25   for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql

บรรทัด 89–118

```

89 -- -----
90 -- Triggers
91 -- -----
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95   new.updated_at := now();
96   return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101   for each row execute function public.touch_updated_at();

```

```

102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104   for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110   insert into public.profiles (id, display_name)
111   values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112   on conflict (id) do nothing;
113   return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118   for each row execute function public.handle_new_user();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92–97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99–104	trigger = "ทุกครั้งก่อนแก้ไขในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107–114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116–118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใส่ให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคน que เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยิงใจ: src/data/supabase.ts

src/data/supabase.ts

บรรทัด 1-38

```
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: คุญแจมาจากตัวแปรสภาพแวดล้อม (.env) คุญแจ publishable ปลอดภัยเพราะ RLS และคุญแจ AI ไม่อยู่ที่นี่
10	ตั้งที่เก็บการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และคุญแจจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง

บรรทัด	ความหมาย
23–26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้ไลบรารีอ่านลิงก์ในแถบที่อยู่เอง (AppProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31–36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว [src/app/welcome.tsx](#)

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแถวโปรไฟล์ให้ทันที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ `SIGNED_IN` แล้วสร้าง CloudRepo โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

Stack.Protected พาไปหน้าตั้งค่าเงิน กรอกรายละเอียดตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

[src/app/welcome.tsx](#)

บรรทัด 148–172

```

148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);

```

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพากลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกินไป) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลิ้งรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;

```

```

31 update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32 remove(id: string): Promise<void>;
33 confirmMany(ids: string[]): Promise<void>;
34 listGoals(): Promise<SavingsGoal[]>;
35 insertGoal(input: GoalInput): Promise<SavingsGoal>;
36 updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37 removeGoal(id: string): Promise<void>;
38 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพรีเซนต์)
14–19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำอะไรเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23–24	อ่านและแก้ไขไฟล์
25–29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30–33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปรอยืนยัน)
34–37	คำสั่งของกระปุกออม (FR-6)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพรีเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44–93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;

```

```

58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:44

บรรทัด	ความหมาย
44–59	TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (คั่นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
61–76	fromRow: แถวฐานข้อมูล → Transaction ของแอป ใช้ Number(. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว่าง
78–93	toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นค่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();

```

```

172     if (error) throw error;
173     return profileFromRow(data as ProfileRow);
174 },
175 async updateProfile(patch) {
176     const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
userId).select().single();
177     if (error) throw error;
178     return profileFromRow(data as ProfileRow);
179 },
180 async listTransactions() {
181     // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182     const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183     const all: Transaction[] = [];
184     for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186             .from('transactions')
187             .select('*')
188             .gte('occurred_at', since)
189             .order('occurred_at', { ascending: false })
190             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
191             .order('id', { ascending: true })
192             .range(from, from + 999);
193         if (error) throw error;
194         all.push...(data as TxRow[]).map(fromRow);
195         if (!data || data.length < 1000) break;
196     }
197     // Older confirmed money still counts in the balance: add it up (two columns only).
198     let carrySatang = 0;
199     for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201             .from('transactions')
202             .select('kind,amount_satang')
203             .eq('status', 'confirmed')
204             .lt('occurred_at', since)
205             .order('occurred_at', { ascending: false })
206             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
207             .order('id', { ascending: true })
208             .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211             carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
Number(r.amount_satang);
212         }
213         if (!data || data.length < 1000) break;
214     }
215     return { txs: all, carrySatang };
216 },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้โปรไฟล์แล้วขอแถวใหม่กลับมา

บรรทัด	ความหมาย
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ีกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ยอดยกมา: รายการที่ ยืนยันแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้ใช้แอปมาหลายปี

โมดกทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };
302       await save();
303     }
304     return cache;

```

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ `mindpay.demo.v2` (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่) บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินตั้งต้น ฿3,000 เลื่อนสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยังไม่มีข้อมูล จะสร้างรายการตัวอย่าง (`buildSampleTransactions` ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้ แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as
300       // TOKEN_REFRESHED:
301       // that signs in too, but only while nothing is open (never over the demo or another
302       // account).
303       const signedIn =
304         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED'
305         && !activeUserRef.current);
306       if (signedIn && session && activeUserRef.current !== session.user.id) {
307         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
308         activateRepo(createCloudRepo(session.user.id), session.user.id);
309       }
310       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
311     }, 0);
312   });
313
314   const stopLinks = onIncomingLink((url) => {
315     handleAuthLink(url);
316   });

```

```

313     return () => {
314         cancelled = true;
315         sub?.data.subscription.unsubscribe();
316         stopLinks();
317     };
318 }, [activateRepo, clear, handleAuthLink]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:271`

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมา หลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย <code>setTimeout(..., 0)</code> เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

`src/data/AppProvider.tsx`

บรรทัด 127–167

```

127     const loadAll = useCallback(async (r: Repo) => {
128         const gen = ++loadGen.current;
129         localChanges.current = new Map();
130         setLoadError(null);
131         try {
132             // Goals never block the app: if they cannot be loaded, the rest still works.
133             const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
134             r.listGoals().catch(() => [] as SavingsGoal[])]);
135             if (gen !== loadGen.current || repoRef.current !== r) return;
136             // A slip saved by the automatic scan during the load is not lost when the list arrives.
137             const changes = localChanges.current;
138             const fresh = list.txs.filter((t) => !changes.has(t.id));
139             const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140             setProfile(p);
141             setTxs([...kept, ...fresh]);
142             setCarry(list.carrySatang);
143             setGoals(g);
144             setStatus('ready');
145         } catch (e) {
146             if (gen !== loadGen.current || repoRef.current !== r) return;
147             setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148             setStatus('ready');
149         }
150     }, []);
151
152     const activateRepo = useCallback(
153         async (r: Repo, id: string) => {

```

```

153     if (activeUserRef.current !== id) {
154         // Another account (or leaving the demo): never show or use the previous one's data
           meanwhile.
155         setProfile(null);
156         setTxs([]);
157         setCarry(0);
158         setGoals([]);
159     }
160     activeUserRef.current = id;
161     repoRef.current = r;
162     setRepo(r);
163     setUserId(id);
164     await loadAll(r);
165 },
166 [loadAll],
167 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง error เก่า
133	โหลด 3 อย่างพร้อมกัน (un A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านบน ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอวาดใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจํา Repo และโหลดข้อมูล

★ สรุป un A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโค้ชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี API key (กุญแจ) ไว้ระบุว่าใครเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น {"text": "สวัสดี"}

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้เราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน **Edge Functions** เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโค้ชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI ลืมแปลง)
5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ข้อยิ่งน้องกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ชื่อรองเท้า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังชื่อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ คุกกี้แวลส์ (Secret)

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้าขยับปากพร้อมซับไต้เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10  if (error) {
11    if (error instanceof FunctionsHttpError) {
12      const body = await (error.context as Response).json().catch(() => null);
13      if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่');
14      if (body?.error?.code === 'not_configured') {
15        throw new CoachError('โค้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย');
16      }
17      if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่');
18      if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19    }
20    throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้ง');
21  }
22  const text = (data as { text?: string })?.text;
```

```

23 if (!text) throw new CoachError('น้องกล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24 return text;
25 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/coach.ts:1

บรรทัด	ความหมาย
2	นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมาว่าผิดพลาด" มาใช้
5	สร้าง error ชนิดของโค้ชเอง ข้อความของมันคือสิ่งที่จะแสดงให้ผู้ใช้เห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้ว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวราวีแบบ token การเข้าสู่ระบบให้เอง
10–21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่ กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22–24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้หน้าโค้ช

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts

บรรทัด 1–26

```

1 // FR-5: the AI coach is น้องกล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องกล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "กล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22
23 Rules:
24 - Reply in Thai, at most 90 words.

```

- 22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot answer, say what is missing.
- 23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to the user.
- 24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
- 25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and after.
- 26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:1`

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก <code>_shared</code> มาใช้ (นามสกุล <code>.ts</code> ต้องเขียนเต็มใน Deno)
8	โควตาต่อวัน อ่านจาก <code>Secret COACH_DAILY_LIMIT</code> ถ้าไม่ได้ตั้งใช้ 60
10	ขนาดคำขอสองสูงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)
12-26	System prompt (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง · โครงสร้างคำตอบ: ตัวเลขบอกอะไร → สำคัญยัง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · ห้ามตำหนิ · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข <code>purchaseCheck</code> บอก Runway ก่อนและหลัง · ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน กู้ยืม ทรัพย์สิน

`supabase/functions/coach/index.ts`

บรรทัด 28-76

```

28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
'Request is too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({

```

```

53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่คว
60       }`,
61     ],
62   });
63   return json({ text: plainText(text) });
64 } catch (e) {
65   if (e instanceof NotConfiguredError) {
66     console.error('coach: AI key missing or refused', e.message);
67     return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68   }
69   if (e instanceof BusyError) {
70     console.warn('coach: AI busy', e.message);
71     return fail('busy', 'The AI service is busy, try again shortly', 503);
72   }
73   console.error('coach failed', e);
74   return fail('coach_error', 'The coach is unavailable right now', 502);
75 }
76 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32–33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36–42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49–51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52–62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของเงินช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64–75	จัดการ error ตามชนิด: กุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวอย่างกลาง: supabase/functions/_shared/common.ts

บรรทัด 17-83

```
17 /** Read one environment variable; a name the runtime refuses counts as "not set". */
18 const env = (name: string) => {
19   try {
20     return Deno.env.get(name);
21   } catch {
22     return undefined;
23   }
24 };
25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES);
26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES);
27
28 export const corsHeaders = {
29   'Access-Control-Allow-Origin': '*',
30   'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
31   'Access-Control-Allow-Methods': 'POST, OPTIONS',
32 };
33
34 export function json(body: unknown, status = 200) {
35   return new Response(JSON.stringify(body), {
36     status,
37     headers: { ...corsHeaders, 'Content-Type': 'application/json' },
38   });
39 }
40
41 /** Error body the app understands: { error: { code, message } } */
42 export function fail(code: string, message: string, status: number) {
43   return json({ error: { code, message } }, status);
44 }
45
46 /** Returns the signed-in user's id, or null when the request has no valid session. */
47 export async function requireUser(req: Request): Promise<string | null> {
48   const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49   if (!token) return null;
50   const { data, error } = await admin().auth.getUser(token);
51   if (error || !data.user) return null;
52   return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
57     {
58       auth: { persistSession: false },
59     });
60 }
61
62 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
63 export class NotConfiguredError extends Error {}
64
65 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
66 export class BusyError extends Error {}
67
68 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
69 export function envLimit(name: string, fallback: number): number {
70   const n = Number(Deno.env.get(name));
```

```

70 return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind,
81     p_limit: dailyLimit });
82   if (error) throw error;
83   return data === true;
84 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/_shared/common.ts:17`

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเชิร์ฟเวอร์ (<code>Deno.env.get()</code>) ถ้าชื่อแปลจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ยอมรับ (ดู <code>readKey</code> ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	<code>json()</code> สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	<code>fail()</code> สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง <code>{ "error": { "code", "message" } }</code> ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ <code>coach.ts</code> ผังแอป)
46–53	<code>requireUser</code> : ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	<code>admin()</code> : client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเชิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านไควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	<code>takeQuota</code> : เรียกฟังก์ชัน SQL <code>take_ai_quota</code> ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในขั้นตอนเดียว

ทำไมนับไควตาต้อง "ทำในขั้นตอนเดียว"

`supabase/migrations/20261001000000_atomic_ai_quota.sql`

บรรทัด 1–44

```

1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''

```

```

12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21         from public.ai_usage
22         where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon,
    authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37     drop constraint if exists transactions_amount_max,
38     add constraint transactions_amount_max check (amount_satang <= 1000000000000);
39 alter table public.profiles
40     drop constraint if exists profiles_amounts_max,
41     add constraint profiles_amounts_max check (
42         abs(opening_balance_satang) <= 1000000000000
43         and (monthly_budget_satang is null or monthly_budget_satang <= 1000000000000)
44     );

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด	ความหมาย
1-5	คอมเมนต์เล่าบ๊วกเดิม: เดิมนับจำนวนครั้งวันนี้นับกับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กุญแจล็อก
7-14	สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิตงาน (slip/coach) และเพดาน คำนวณจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer)
16-18	ค่าที่ส่งมาผิด → ไม่อนุญาต
19	ล็อก ต่อผู้ใช้ต่อชนิตงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ล็อกปลดเองเมื่อจบคำสั่ง)
20-22	นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด
23-25	ถึงเพดานแล้ว → ไม่อนุญาต
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเพดานจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]):
  string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\"']|['\"]$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini:
  boolean }): AiProvider | null {
39   const forced = env.forced?.trim().toLowerCase();
40   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
41   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
42   if (env.hasClaude) return 'claude';
43   if (env.hasGemini) return 'gemini';
44   return null;
45 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด ความหมาย

1-2 โฟกัสนี้เป็นฟังก์ชันล้วน ไม่ใช้คำสั่งของ Deno จึงเอาไปเทสต์ด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนถือปวงออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีคีย์แจด้วย) ไม่จันมีคีย์แจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.ts

บรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req,
133     deadline);
134   // Metadata only: never log images, questions or replies.
135   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant:
136     req.variant ?? 'primary', ms: Date.now() - started }));
137   return result;
138 }
139
140 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and
141   retries). */
142 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
```

```

140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS,
144   left)) });
145   } catch (e) {
146     const name = (e as Error)?.name;
147     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed
148   out (${url.split('/')[2]})`);
149     throw e;
150   }
151 }
152
153 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
154
155 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model:
156   string }> {
157   const key = claudeKey();
158   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ??
159   CLAUDE_DEFAULT_MODEL;
160   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
161   const body: Record<string, unknown> = {
162     model,
163     max_tokens: req.maxTokens,
164     system: req.system,
165     messages: [
166       {
167         role: 'user',
168         content: req.content.map((c) =>
169           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type:
170   c.mediaType, data: c.data } },
171       ),
172     ],
173   };
174   if (req.schema) {
175     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
176   }
177   const res = await timedFetch(
178     'https://api.anthropic.com/v1/messages',
179     {
180       method: 'POST',
181       headers: {
182         'x-api-key': key,
183         'anthropic-version': '2023-06-01',
184         'content-type': 'application/json',
185       },
186       body: JSON.stringify(body),
187     },
188     deadline,
189   );
190   if (!res.ok) {
191     const detail = await res.text();
192     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
193     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
194     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
195     throw new Error(msg);
196   }
197   const data = await res.json();
198   const text = (data.content ?? []).
199     .filter((b: { type: string }) => b.type === 'text')

```

```

196     .map((b: { text: string }) => b.text)
197     .join('');
198     if (!text) throw new Error('Claude API returned no text');
199     return { text, model };
200 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:114

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลคำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูปคำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็น
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบถูกใจใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: คุกกี้แฉิด → NotConfigured, 429/529 (ส่งถี่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทั่วไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คืนพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับคุกกี้แฉิดข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.ts

บรรทัด 75–93

```

75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';

```

บรรทัด	ความหมาย
87	กุญแจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กุญแจนี้ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

1. สร้างกุญแจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
2. เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)
3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีการขอ
5. ถ้าแก้ไขโค้ดใน `supabase/functions` ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ `parse-slip`)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้ผู้ใช้เสือก (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้ง่ายไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำอย่างไรในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจได้, ปุ่มลัด + ฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	บอกล่าพุดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามันใจแค่นี้	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังงี้" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง

สี: Green + Gold

ทิศทางแบรนด์คือ **สีเขียว** (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ **สีทอง** ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

`src/ui/theme.ts`

บรรทัด 1-21

1 /**

```

2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9  import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

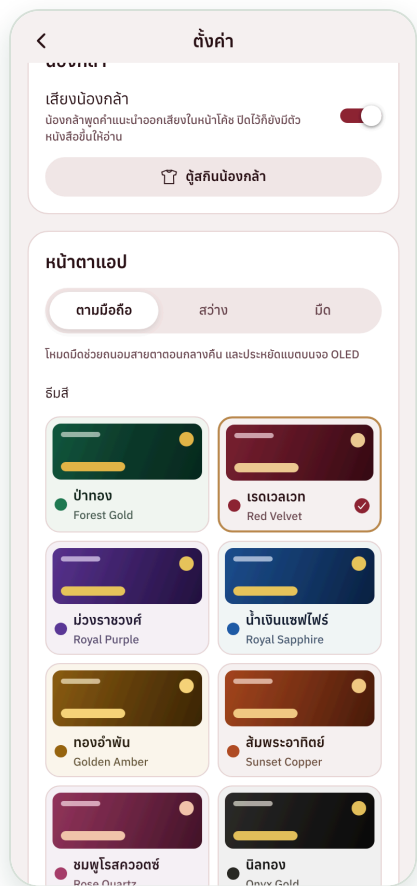
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ป่าทอง สว่าง	ป่าทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียม ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่ง que เลือกอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, ระวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดงอิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), เรดเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, สัมพระอาทิตย์, ชมพูโรสควอตซ์, นิลทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts บรรทัด 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน `buildTheme` สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (`onAccent`) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทงนั้น

หน้าจอเรียกแค่ `const theme = useTheme()` ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล **WCAG AA** กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย **4.5 : 1** (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาสั้นหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token <code>accentInk</code> ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน `contrast()` ในแอป

src/ui/theme.tsบรรทัด 261–279

```
261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
```

```

264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266 };
267 return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272     const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273     return (hi + 0.05) / (lo + 0.05);
274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278     return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35     for (const dark of [false, true]) {
36         it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is
used on`, () => {
37             const t = buildTheme(spec.key, dark);
38             const pairs: [string, string, string][] = [
39                 ['inkSoft on bg', t.inkSoft, t.bg],
40                 ['inkSoft on surface', t.inkSoft, t.surface],
41                 ['onPrimary on primary', t.onPrimary, t.primary],
42                 ['onAccent on accent', t.onAccent, t.accent],
43                 ['accentInk on surface', t.accentInk, t.surface],
44                 ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45                 ['good on goodSoft', t.good, t.goodSoft],
46                 ['watch on watchSoft', t.watch, t.watchSoft],
47                 ['critical on criticalSoft', t.critical, t.criticalSoft],
48                 ['good on bg', t.good, t.bg],
49                 ['watch on bg', t.watch, t.bg],
50                 ['critical on bg', t.critical, t.bg],
51                 ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52                 ['heroInk on heroTop', t.heroInk, t.heroTop],
53             ];
54             const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([n, a, b]) => `${n}
${contrast(a, b).toFixed(2)}`);
55             expect(low).toEqual([]);
56         });
57     }
58 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ TC-78 ตรวจสอบทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่มือ (บรรทัด 38–53) ถ้าคู่มือต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่มือพร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```
301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {
308   serif: 'NotoSerifThai_600SemiBold',
309   serifBold: 'NotoSerifThai_700Bold',
310   sans: 'Anuphan_400Regular',
311   sansMedium: 'Anuphan_500Medium',
312   sansSemi: 'Anuphan_600SemiBold',
313   sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below.
320  */
321 export const type = {
322   display: { fontSize: 38, lineHeight: 52 },
323   h1: { fontSize: 26, lineHeight: 38 },
324   h2: { fontSize: 20, lineHeight: 30 },
325   h3: { fontSize: 17, lineHeight: 26 },
326   body: { fontSize: 15, lineHeight: 24 },
327   small: { fontSize: 13, lineHeight: 20 },
328   micro: { fontSize: 11, lineHeight: 16 },
329 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:301

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว กันสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

src/ui/components.tsx

บรรทัด 261–338

```
261 export function Button({
262   label,
```

```

263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 } : {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 }) {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },
291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down
305       only,
306       // so buttons side by side never share a touch area).
307       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
308       accessibilityRole="button"
309       aria-disabled={!off}
310       aria-busy={!loading}
311       style={[
312         {
313           backgroundColor: colors.bg,
314           borderColor: colors.border,
315           borderWidth: 1.5,
316           borderRadius: radius.pill,
317           paddingVertical: small ? 8 : 12,
318           paddingHorizontal: small ? 14 : 20,
319           flexDirection: 'row',
320           alignItems: 'center',
321           justifyContent: 'center',
322           gap: 8,
323           opacity: off ? 0.5 : 1,

```

```

323         overflow: shine ? 'hidden' : undefined,
324     },
325     style,
326     { transform: [{ scale: press.scale }] },
327   ]}
328   >
329   {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ?
'rgba(255,255,255,0.45)' : 'rgba(255,255,255,0.28)'} /> : null}
330   {loading ? (
331     <ActivityIndicator color={colors.fg} />
332   ) : icon ? (
333     <Ionicons name={icon} size={small ? 16 : 19} color={colors.fg} />
334   ) : null}
335     <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color:
colors.fg }]}>{label}</Text>
336   </AnimatedPressable>
337 );
338 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/components.tsx:261`

บรรทัด	ความหมาย
261–283	props ของปุ่ม: ข้อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
284–285	สีของธีม และแอนิเมชันยุบตัวตอนกด (เหลือ 95%)
286–293	ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาวตัวขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
294	ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
297–300	กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นิ้วคนกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอว่าเป็นปุ่ม และสถานะปิด/คำสั่งโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	คำสั่งโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาถึงหนา

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่คำสั่งพิมพ์ให้พิมพ์บอร์ด
Card	กลุ่มเนื้อหา มี 3 โท: ปกติ, พิเศษ (alt), ท่องอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ

ชิ้นส่วน	ใช้เมื่อ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินลด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ชุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: น้องกล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำยังไง
คนตาบอด/สายตาเลือนราง	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"
แยกลีไม่ออก (ตาบอดสี)	สถานะไม่ใช้สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมารถจากการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มแดง, ตันไม้เงินโยก, พลุตอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

- ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นแม่มือถือราคาประหยัด
- ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
- เคส "ลดการเคลื่อนไหว"
- ไม่ใช้ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

- ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha()
- ตัวหนังสือใช้ <T> และเงินใช้ <Money>
- ปุ่ม ชิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
- ลองเปิดหน้านั้น 2–3 ธีม ทั้งสว่างและมืด

5. รับ npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web

i ทบทวนศึกษา UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จดให้") และแนวการออกแบบของ Hugging Face (มาสคอตตัวเดียวหลายอารมณ์, สีอุ้งบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพลุดอนทำสำเร็จ) แล้วออกแบบตัวละครน้องกล้ายของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเรามีการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทส

ทำไมต้องเขียนเทส

เวลาแก้ไขจุดหนึ่ง อาจทำให้จุดอื่นพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แย่ จะใช้เวลาหลายชั่วโมงและลืมบางจุดแน่นอน เทสอัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)

ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)

เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)

คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสอยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสของ FR-6:

src/domain/__tests__/domain.test.ts

บรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35–36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสต์วันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสต์จะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38–58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายจ่าย หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

src/domain/__tests__/domain.test.ts บรรทัด 184–210

```

184 it('TC-14 computes days and depletion date', () => {
185     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000,
now: NOW });
186     expect(r.days).toBe(15);
187     expect(r.depletionDay).toBe('2026-10-12');
188     expect(r.status).toBe('healthy');
189 });
190
191 it('TC-15 handles zero average without dividing by zero', () => {
192     const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now:
NOW });
193     expect(r.status).toBe('no_spending');
194     expect(r.days).toBeNull();
195     expect(r.depletionDay).toBeNull();
196 });
197
198 it('TC-16 flags balance at or below the low line', () => {
199     const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000,
now: NOW });
200     expect(r.status).toBe('below_floor');
201     expect(r.days).toBe(0);
202 });
203
204 it('TC-17 status thresholds', () => {
205     const at = (days: number) =>
206         computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang:
10_000, now: NOW }).status;
207     expect(at(6)).toBe('critical');
208     expect(at(7)).toBe('watch');
209     expect(at(14)).toBe('healthy');
210 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เงินสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186–188	Assert: expect (ค่าจริง) .toBe (ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191–196	TC-15 ทดสอบ กรณีพิเศษ: ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198–202	TC-16 ทดสอบ ขอบ: ยอดเท่ากับเงินสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"

บรรทัด	ความหมาย
204–210	TC-17 ทดสอบ ขอบของเกณฑ์ : 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเผลอแก้ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

บ๊ีกชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทิมจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือนปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บ๊ีกที่เจอตอนตรวจทั้ง repo (1–3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเซิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ [e2e/web-auth.mjs](#) (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เซิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน

`e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไม่ได้อยู่ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (`page.route`) เหมือนซ้อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29–60

```
29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)}):` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 },
46     deviceScaleFactor: 2, locale: 'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50     m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request
52     failed`, r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }
58 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
59 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false
60   }).first().waitFor({ state: 'visible', timeout }).then(() => true, () => false);
61 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false
62   }).first().waitFor({ state: 'hidden', timeout }).then(() => true, () => false);
63 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached',
64   timeout: 8000 }).catch(() => {});
65 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
66 /** Waits (up to `timeout`) until the element is visible. */
67 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(()
68   => true, () => false);
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `e2e/web-auth.mjs:29`

บรรทัด ความหมาย

30–36 `check` (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions

บรรทัด	ความหมาย
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดักคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแบบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the
    coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has
    fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000
    }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x)
    < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
599     ? await page.evaluate(({ x, y }) => {
600       const el = document.elementFromPoint(x, y);
601       return el ? `${el.tagName} role=${el.getAttribute('role')}`
        label=${el.getAttribute('aria-label')}` : 'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';
604   if (box) {
605     // hover() waits until the slider itself receives the pointer (nothing on top of it).
606     await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607     await page.mouse.down();

```

```

608     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps:
8 });
609     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps:
8 });
610     await page.mouse.up();
611 }
612 const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613 await page.waitForTimeout(900);
614 await shot(page, '28-runway-slider');
615 check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' &&
(await visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under}
introWaitMs=${introMs}`);
616 // The arrow keys move it too (keyboard users on the web).
617 await slider.focus();
618 await page.keyboard.press('ArrowRight');
619 const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620 check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0)
+ 5), `value=${keyed}`);
621 await page.keyboard.press('Home');
622 await page.fill('#runway-price', '2000');
623 check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ
฿2,000')) && (await visible(page, 'ตอนนี้')));
624 await button(page, 'ถามโค้ชเรื่องนี้').click();
625 const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626 check('Runway: "ask the coach" opens the coach with the question ready',
asked.includes('฿2,000'), asked);
627 await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574–579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589–597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่เงินจะลากพลาด
604–611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612–615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617–620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622–623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624–626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ข้อผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน [docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ตำหนิ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแกลเลอรี แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → ทดสอบ

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจสอบคุณภาพอื่น ๆ

คำสั่ง	ตรวจสอบอะไร
npm run typecheck	TypeScript ตรวจสอบนิพจน์ข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจสอบรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจสอบความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แจ้งเตือน 0 ปัญหาเรื่องตาราง)

คำสั่ง	ตรวจสอบอะไร
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก่ครบพร้อมทดสอบ

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

วงจรทำงาน 1 รอบ

```

1 git pull                                # 1. ดึงของล่าสุดก่อนเริ่ม
2 git checkout -b fix-runway-text         # 2. แยก branch ใหม่ (ไม่บังคับ)
3 # ... แก้โค้ดใน VS Code ...
4 npm test                                # 3. รันเทสต์ให้ผ่านก่อน
5 git status                              # 4. ดูว่าแค่ไฟล์ไหนไปบ้าง
6 git add src/domain/runway.ts            # 5. เลือกไฟล์ที่จะเซฟ
  
```

```
7 git commit -m "Runway: say 'ประมาณ' in the date" # 6. เซฟพร้อมข้อความ
8 git push -u origin fix-runway-text # 7. ส่งขึ้น GitHub แล้วเปิด Pull Request
```

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่เก็บ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก
พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: *"Runway: arrow keys on the web; steadier drag test"*, *"Audit follow-up: fix regressions found by independent review"* อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ github.com/2550expo-tech/Socrates-and-Skeletons - (สาธารณะ ใครก็อ่านได้) branch main มี **66 commit** ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บไซต์ GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, บัญชี, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาคุกกี้ AI, อ่าน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจด (FR-3), บัญชีพุดจด, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แกะบีก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้ง push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่ได้เปลี่ยน ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ 2550expo-tech.github.io/Socrates-and-Skeletons/
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป

ไฟล์	ทำอะไร	ผลลัพธ์
e2e-webkit.yml	E2E ชัดเดียวกันบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่บล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ต่ออยู่

android-apk.yml

เทสหรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret EXPO_TOKEN และเทสผ่าน

e2e-webkit.yml

ตรวจสอบ Safari เพิ่ม

อ่าน web.yml ที่ละบรรทัด

[.github/workflows/web.yml](#)

บรรทัด 1-76

```

1 # Publishes the web version of MindPay to GitHub Pages on every push to main:
2 # https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3 # The web version shows the app at phone width, works with real accounts and
4 # demo data, and reads slips from photos the user uploads. Scanning the whole
5 # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6 # Add to Home Screen, and it opens full screen with the MindPay icon.
7 name: Deploy web app
8
9 on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21 jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env

```

```

37
38   - name: Export web build
39     env:
40       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41     run: npx expo export --platform web --output-dir dist
42
43   - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44     env:
45       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46     run: node scripts/web-home-screen.mjs dist
47
48   - name: Single-page app fallback for deep links
49     run: cp dist/index.html dist/404.html
50
51   - name: Unit tests
52     run: npm test
53
54   # Browser test of every sign-in path (sign up, email code, links, reset
55   # password, sign out) against a local stand-in for Supabase. If anything
56   # fails, the website is NOT published and the screenshots are attached.
57   - name: Install the test browser
58     run: npx playwright install --with-deps chromium
59
60   - name: Browser tests (sign-in flows)
61     run: node e2e/web-auth.mjs dist e2e-shots
62
63   - name: Keep screenshots when the browser tests fail
64     if: failure()
65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด	ความหมาย
1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอโฮมบน iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สิทธิ์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่มาระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รับบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดีงโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)

บรรทัด	ความหมาย
28–31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35–36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38–41	build เว็บลงโพลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43–46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจอล็อกอิน" แล้วเปิดเต็มจอเหมือนแอป
48–49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51–52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทั้งงาน
57–61	ติดตั้งเบร่าวเซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63–68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวนโหลดดูว่าพังตรงไหน
70–76	ผ่านทุกด่านแล้วจึงเผยแพร่โพลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

<code>.github/workflows/android-apk.yml</code>	บรรทัด 12–90
<pre> 12 on: 13 push: 14 branches: [main] 15 # Editing documents only does not need a new APK. 16 paths-ignore: ['**/*.md', 'docs/**'] 17 workflow_dispatch: 18 19 permissions: 20 contents: write 21 22 # A build that already started runs to the end; newer pushes wait in line 23 # (only the newest waiting push is kept). 24 concurrency: 25 group: android-apk 26 cancel-in-progress: false 27 28 jobs: 29 apk: 30 runs-on: ubuntu-latest 31 timeout-minutes: 45 32 steps: 33 - uses: actions/checkout@v4 34 35 - uses: actions/setup-node@v4 36 with: 37 node-version: 22 38 cache: npm 39 40 - uses: actions/setup-java@v4 41 with: 42 distribution: temurin </pre>	

```

43     java-version: 17
44
45   - name: Install dependencies
46     run: npm ci
47
48   - name: Unit tests
49     run: npm test
50
51   - name: Typecheck
52     run: npx tsc --noEmit
53
54   - name: App settings (public Supabase URL and publishable key)
55     run: cp .env.example .env
56
57   - name: Generate the native Android project
58     run: npx expo prebuild --platform android --no-install
59
60   - name: Give Gradle enough memory
61     # The default Metaspace ran out during Android lint and the build hung until the job
timeout.
62     run: |
63       sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64       # The generated file has no trailing newline: start with one so the setting is not
glued onto the last line.
65       printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
        android/gradle.properties
66       tail -n 3 android/gradle.properties
67
68   - name: Build release APK
69     working-directory: android
70     # arm64 only: covers practically every Android phone from the last 7 years and builds
much faster.
71     # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to
produce the APK.
72     run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
        lintVitalAnalyzeRelease --stacktrace
73
74   - name: Rename APK
75     run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
76
77   - uses: actions/upload-artifact@v4
78     with:
79       name: mind-pay-apk
80       path: mind-pay.apk
81
82   - name: Publish to the "latest-apk" release
83     env:
84       GH_TOKEN: ${GITHUB_TOKEN}
85     run: |
86       gh release delete latest-apk --yes --cleanup-tag || true
87       gh release create latest-apk mind-pay.apk \
88         --title "MindPay (latest APK)" \
89         --notes "Built from ${GITHUB_SHA::7} on $(date -u +%Y-%m-%d %H:%M UTC'). Open this
        page on an Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90         --prerelease

```

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างโฟลเดอร์ android ทั้งโปรเจกต์จาก app.json (ทีมไม่เก็บโฟลเดอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์เล่าว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหน เมื่อไร

i APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและพีซีเซ็นได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

ค่าลับที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

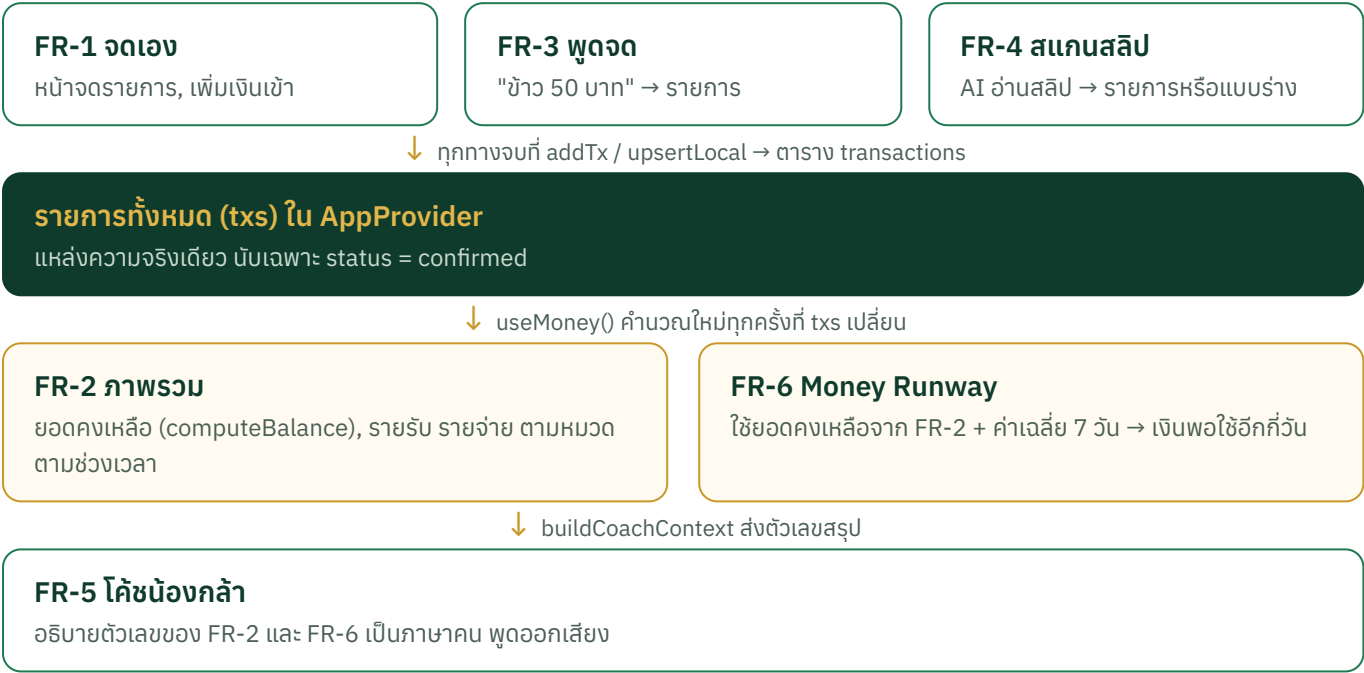
ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)

ไฟล์	มีอะไร
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ



ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จด/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจดยรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจดยรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
<code>src/domain/types.ts</code>	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
<code>src/domain/money.ts</code>	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่
<code>src/domain/dates.ts</code>	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
<code>src/domain/summary.ts</code>	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
<code>src/domain/categories.ts</code>	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
<code>src/data/AppProvider.tsx</code>	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
<code>src/app/transaction.tsx</code>	FR-1 และ FR-4 (ตรวจสอบ)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสต์ก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

- Q1** ทำไมเลือก Expo / React Native?

โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
- Q2** ทำไมเลือก Supabase?

ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในที่เดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
- Q3** ข้อมูลผู้ใช้ปลอดภัยยัง?

ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แวลบและคุกกี้ AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ยอดรวม
- Q4** ทำไมเก็บเงินเป็นสตางค์?

ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
- Q5** ทำไมต้องจัดวันตามเวลาไทย?

ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเข้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
- Q6** ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม?

ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่ข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
- Q7** แบบร่าง (draft) คืออะไร ทำไมต้องมี?

รายการจากสลิปที่ AI อ่านไม่มั่นใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
- Q8** ใช้ AI ตรงไหน และป้องกัน AI ผิดยังไง?

2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เหนือความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว

Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? กฎแฉ AI จะหลุดไปกับแอป Edge Function ถือกฎแฉไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยังไง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รันเทสต์ก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ทหลุดระหว่างฟรีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยังไง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และสหัสเลขต์ (TC/MT) ที่พิสูจน์
Q13	ใช้ AI ช่วยเขียนโค้ดไหม? ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกแถวบอกจุดประสงค์ ผลลัพธ์ สิ่งที่ทีมแก้หรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอนตามจริงว่าตัวเองตรวจอะไรไปแล้วบ้าง)
Q14	ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม? การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง
Q15	อะไรที่ยังไม่ได้ทำ (นอก MVP)? Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ท) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเซฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AuthProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.ink
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่

คำ	ความหมายแบบง่าย
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่ที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้โครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น ไมค์ แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใช้ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบยังไ้
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-2 Overview Dashboard

หน้าหลัก ภาพรวมรายรับ รายจ่าย ยอดคงเหลือ ภาคนี้เขียนในมุมมองของคนสร้าง FR-2
เอง ตั้งแต่ปัญหาที่ยากแก่ หน้าจอ หลักการคำนวณ โค้ดจริงที่ละบรรทัด การทดสอบ ไป
จนถึงคำถามที่อาจารย์น่าจะถาม

FR-2 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-2 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-2 **Overview Dashboard** คือ "หน้าปัดรถยนต์" ของ MindPay เปิดแอปมาแล้วเห็นทันทีว่า **ตอนนี้มีเงินเท่าไร, ช่วงที่เลือกเงินเข้า-ออกเท่าไร, เงินไปอยู่หมวดไหน, 7 วันที่ผ่านมาใช้วันละเท่าไร และ เดือนนี้ใช้ไปเท่าไรเทียบกับงบ** นอกจากนี้ยังมี **ปฏิทินการใช้จ่าย** ในแท็บรายการ และ **สรุปเดือน** แบบเรียงเล่าเติมจ่อ

ยอดคงเหลือ = ยอดตั้งต้น + ยอดยกมา + รายรับที่ยืนยันแล้ว - รายจ่ายที่ยืนยันแล้ว

ตัวเลขทุกตัวของ FR-2 นับเฉพาะรายการที่ "ยืนยันแล้ว" (แบบร่างจากสลิปไม่นับ) · จัดวันตามเวลาไทย · ช่วง "วันนี้ / 7 วัน / 1 เดือน / 6 เดือน / 1 ปี" มีความหมายเดียวกันทั้งแอป

อยู่ตรงไหนในแอป

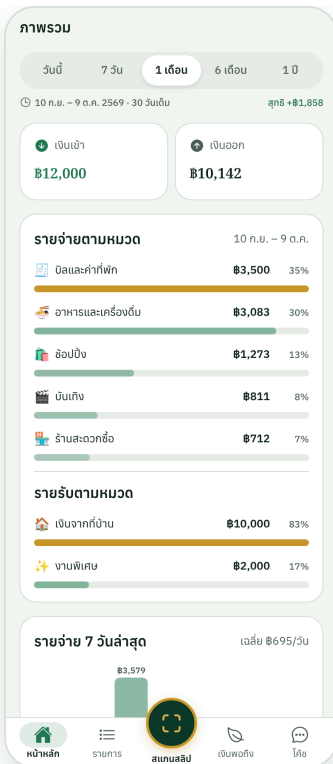
ที่	เห็นอะไร
หน้าหลัก การ์ดเขียวด้านบน	ยอดคงเหลือ ตัวใหญ่ (นับขึ้นตอนเปิด) ป้ายสถานะ และต้นไม้เงิน (ส่วนจำนวนวันเป็นของ FR-6)
หน้าหลัก ส่วน "ภาพรวม"	ปุ่มเลือกช่วง 5 แบบ, ช่วงเวลาที่นับจริง, ยอดสุทธิ, การ์ดเงินเข้า/เงินออก, รายจ่ายตามหมวด และรายรับตามหมวด
หน้าหลัก การ์ด "รายจ่าย 7 วันล่าสุด"	กราฟแท่ง 7 วัน วันนี้สีทอง เส้นประ = ค่าเฉลี่ยต่อวัน และป้ายยอดของวันที่ใช้มากที่สุด
หน้าหลัก การ์ด "งบเดือนนี้"	ใช้ไปแล้วเทียบกับงบที่ตั้งไว้ (แถบเปลี่ยนสีเมื่อใกล้/เกินงบ)
หน้าหลัก "รายการล่าสุด"	5 รายการล่าสุดที่ยืนยันแล้ว และลิงก์ "ดูทั้งหมด"
แท็บรายการ ปฏิทินการใช้จ่าย	แต่ละวันไล่สีทองตามยอดใช้ จุดเขียว = มีเงินเข้า ตะวันเพื่อดูเฉพาะวันนั้น
การ์ด "สรุปเดือน..." บนหน้าหลัก	เปิดเรียงเล่า 7 หน้าจ่อ: ใช้ไปเท่าไร เงินไปไหน วันที่ใช้เยอะสุด จดด้วยวิธีไหน เทียบเดือนก่อน

3 ข้อที่ต้องจำ

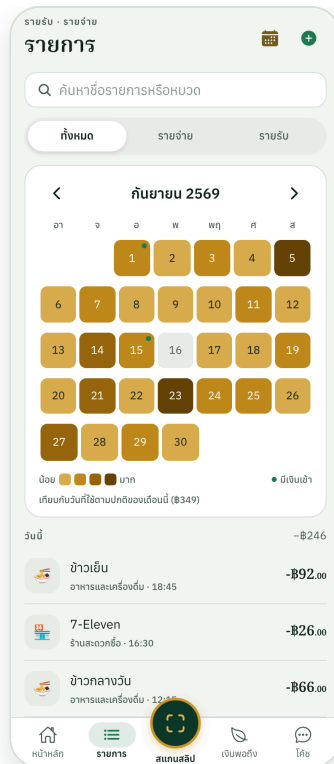
- นับเฉพาะที่**ยืนยันแล้ว** แบบร่างจากสลิป (FR-4) ไม่เปลี่ยนตัวเลขใดที่ผู้ใช้เห็น จนกว่าจะกดยืนยัน ฟังก์ชัน confirmedOnly ตัวเดียวถูกใช้ทุกที่
- วันไทย ไม่ใช่วัน UTC** รายการตอน 00:30 คืนวันที่ 9 ต้องอยู่ "วันที่ 9" แม้เวลามาตรฐาน (UTC) ยังเป็นวันที่ 8 ทุกการจัดกลุ่มผ่าน bkkDayKey
- ช่วงเวลามีความหมายเดียว** "1 เดือน" = 30 วันเต็มนับรวมวันนี้ ทั้งในภาพรวม (FR-2) และหน้าสแกน (FR-4) เพราะใช้ตาราง RANGE_DAYS เดียวกัน และแอปเขียนบอกช่วงจริงใต้ปุ่มเสมอ เช่น "10 ก.ย. – 9 ต.ค. 2569 · 30 วันเต็ม"



หน้าหลัก: ยอดคงเหลือ
฿11,113.00



ภาพรวม 1 เดือน: เงินเข้า เงินออก
และรายจ่ายตามหมวด



ปฏิทินการใช้จ่าย เดือนกันยายน

1 ภาพหน้าจอในเล่มนี้ถ่ายตอนไหน

ทุกภาพของ FR-2 ถ่ายจากเว็บเวอร์ชันจริงใน โหมทดกลง (ข้อมูลตัวอย่าง) โดยตั้งนาฬิกาเป็น 19:00 น. วันศุกร์ที่ 9 ตุลาคม 2569 ตัวเลขทุกตัวในเล่มคำนวณเข้าด้วยฟังก์ชันจริงของแอปแล้วตรงกับภาพ ถ้าเปิดแอปเองวันอื่น ตัวเลขจะต่างไป เพราะข้อมูลตัวอย่างสร้างย้อนหลังจาก "วันนี้" เสมอ

ปัญหาที่ฉันอยากแก้

ยอดในแอปธนาคารบอกแค่ 'เหลือเท่าไร' แต่ไม่บอกว่า 'หายไปไหน'

จากการสัมภาษณ์ตอนทำ SRS นักศึกษาส่วนใหญ่ดูแค่ยอดเงินในแอปธนาคาร เห็นตัวเลขลดลงแต่ตอบไม่ได้ว่า **เดือนนี้ใช้ไป** **กี่บาทมากที่สุด** หรือ **สัปดาห์นี้ใช้มากกว่าปกติไหม** ข้อมูลรายการ (FR-1, FR-3, FR-4) มีครบแล้ว แต่ถ้าไม่สรุปให้เห็นภาพ ผู้ใช้ก็ต้องไล่ดูทีละรายการเอง

FR-2 จึงตอบคำถาม 5 ข้อนี้ภายในหน้าเดียว โดยเรียงตามลำดับที่คนอยากรู้ (คอมเมนต์บรรทัดแรกของ `src/app/(tabs)/index.tsx`): มีเท่าไร → พอใช้กี่วัน → วันนี้ใช้ได้เท่าไร → เงินไปไหน

คำถามของผู้ใช้	FR-2 ตอบด้วย
ตอนนี้มีเงินเท่าไร	ยอดคงเหลือตัวใหญ่สุดบนการ์ดเขียว
ช่วงนี้เงินเข้า-ออกเท่าไร	การ์ดเงินเข้า/เงินออก + ยอดสุทธิ เลือกช่วงได้ 5 แบบ
เงินไปไหนมากที่สุด	แถบรายจ่ายตามหมวด เรียงมากไปน้อย พร้อมเปอร์เซ็นต์
สัปดาห์นี้ใช้มากผิดปกติไหม	กราฟ 7 วัน มีเส้นประค่าเฉลี่ย และป้ายวันที่ใช้มากที่สุด
เดือนนี้เกินงบหรือยัง	แถบงบเดือนนี้ เขียว → เหลือง (เกิน 85%) → แดง (เกินงบ)
วันไหนใช้เยอะ	ปฏิทินไล่สี แถวๆ ละวัน
เดือนนี้เป็นยังไงเมื่อเทียบกับเดือนก่อน	สรุปเดือน เทียบจำนวนวันเท่ากัน

เป้าหมายที่ฉันตั้งไว้ตอนออกแบบ

เป้าหมาย	ทำยังไงในแอป
ตัวเลขต้องเชื่อถือได้	นับเฉพาะที่ยืนยัน, เงินเป็นสตางค์, วันไทย, ทุกสูตรอยู่ใน <code>src/domain/summary.ts</code> ซึ่งมี unit test
ไม่มีความกำกวม	เขียนช่วงเวลาจริงได้ปุ่มเลือกช่วงเสมอ เช่น "9 ต.ค. 2569 · 00:00–23:59"
อ่านเข้าใจใน 3 วินาที	ตัวเลขสำคัญตัวใหญ่, กราฟเรียบง่าย ทุกป้ายบอกค่าที่กราฟแสดงจริง, หมวดเรียงมากไปน้อย
ไม่ล้าสมัยข้ามคืน	เที่ยงคืนเวลาไทย "วันนี้" เริ่มใหม่เอง แม้เปิดแอปค้างไว้
สนุกพอให้กลับมาดู	ยอดเงินนับขึ้น, กราฟโตขึ้น, สรุปเดือนแบบเรื่องเล่า แต่ทั้งหมดหยุดเองและเคารพ "ลดการเคลื่อนไหว"

Requirement และเงื่อนไขผ่าน

จาก docs/TRACEABILITY.md: Acceptance criteria → โค้ด → เทสต์

i Requirement FR-2

"แสดงรายรับ รายจ่าย ยอดคงเหลือ และยอดตามหมวด"

Acceptance criteria	โค้ด	เทสต์
ยอดคงเหลือ = ยอดตั้งต้น + รายรับ – รายจ่าย ที่ยืนยันแล้ว (แบบร่างไม่นับ)	summary.ts computeBalance	TC-09
เลือกช่วง วันที่ / 7 วัน / 1 เดือน / 6 เดือน / 1 ปี และได้ยอดแยกหมวด	summarizeRange, dates.ts RANGE_DAYS	TC-08, TC-10
กราฟ 7 วัน แสดงวันที่ไม่มีรายจ่ายเป็น 0	dailyTotals	TC-11
ปฏิทินการใช้จ่ายในแท็บรายการ: ไล่สีทองตามยอดใช้เทียบวันปกติของเดือน (สีเดียว อ่อน→เข้ม ตรวจสอบด้วยตัวตรวจสอบสีทั้งโหมดสว่าง/มืด), จุดเขียว = มีเงินเข้า, และวันเพื่อดูเฉพาะวันนั้น, เลื่อนดูเดือนก่อนได้	recap.ts spendCalendar, ui/SpendCalendar.tsx, transactions.tsx	TC-70, E2E calendar
จัดวันตามเวลาไทย ไม่ใช่ UTC	bkkDayKey	TC-03
สรุปเดือนแบบเรียงเล่า: ยอด หมวดที่ใช้มาก วันที่ใช้มาก นิสัยการจด และเทียบเดือนก่อนแบบจำนวนวันเท่ากัน	recap.ts monthRecap, app/recap.tsx	TC-69, E2E recap
(ตรวจ repo 1–3 ต.ค.) ยอดเงินรวมรายการที่ต่ำกว่า 400 วันด้วย · สลับบัญชีไม่ค้างข้อมูลบัญชีเก่า · รีเฟรชไม่ทำรายการที่เพ่งบันทึกหาย	data/AppProvider.tsx, data/repo.ts	TC-81, TC-82, E2E
เที่ยงคืนเวลาไทย "วันนี้" เริ่มใหม่เอง	AppProvider (timer + AppState), msUntilNextBkkMidnight	E2E midnight

ตัวเลขทุกตัวบนหน้าหลักมาจากไหน

แผนที่ระหว่าง 'สิ่งที่เห็น' กับ 'ฟังก์ชันที่คำนวณ' ใช้ตอบคำถาม 'ตัวเลขนี้มาจากไหน' ได้ทุกตัว

สิ่งที่เห็น (ภาพ 9 ต.ค. 19:00)	ฟังก์ชัน	ไฟล์	FR
ยอดคงเหลือ ฿11,113.00	computeBalance	src/domain/summary.ts:12	FR-2
ป้าย "สบาย ๆ"	RUNWAY_BADGE[runway.status]	src/app/(tabs)/index.tsx:54	FR-6
เงินพอใช้อีก 15 วัน · ถึง 24 ต.ค.	computeRunway	src/domain/runway.ts:72	FR-6
วันนี้ใช้ได้ประมาณ ฿461 / วัน	safeDailySpend	src/domain/runway.ts:96	FR-6
"10 ก.ย. – 9 ต.ค. 2569 · 30 วันเต็ม"	formatRangeSpan	src/domain/dates.ts:117	FR-2
สุทธิ +฿1,858 · เงิน เข้า ฿12,000 · เงิน ออก ฿10,142	summarizeRange	src/domain/summary.ts:66	FR-2
แถบรายจ่ายตามหมวด (บิล ฿3,500 35% ...)	totalsByCategory + CategoryBars	src/domain/summary.ts:40 , src/ui/charts.tsx:133	FR-2
กราฟ 7 วัน (สูงสุด ฿3,579)	dailyTotals + DayBars	src/domain/summary.ts:91 , src/ui/charts.tsx:47	FR-2
เฉลี่ย ฿695/วัน (เส้น ประ)	averageDailyExpense	src/domain/runway.ts:36	FR-6 (ใช้ ร่วม)
งบเดือนนี้ ฿5,091 / ฿10,000	monthExpense	src/domain/summary.ts:129	FR-2
รายการล่าสุด 5 รายการ	recent (useMemo)	src/app/(tabs)/index.tsx:191	FR-2 + FR-1
การ์ด "สรุปเดือน ตุลาคม 2569"	recapToOffer	src/domain/recap.ts:113	FR-2
ปฏิทินในแถบรายการ	spendCalendar + SpendCalendar	src/domain/recap.ts:147 , src/ui/SpendCalendar.tsx:20	FR-2

★ **ฝึกตอบ**

ถ้าอาจารย์ชี้ตัวเลขใดบนหน้าจอแล้วถามว่า "มาจากไหน" ให้ตอบเป็น 3 ท่อน: **ข้อมูล** (รายการที่ยืนยันแล้วใน txs) → **ฟังก์ชัน** (เช่น summarizeRange ใน summary.ts) → **หน้าจอ** (index.tsx เรียกผ่าน useMemo แล้วส่งให้ Money/CategoryBars วาด)

ทัวร์หน้าจอ: หน้าหลักส่วนบน

ยอดคงเหลือคือตัวเลขที่สำคัญที่สุด จึงอยู่บนสุด ตัวใหญ่สุด บนการ์ดที่เด่นที่สุด



หน้าหลักเมื่อเปิดแอป (9 ต.ค. 2569 19:00 โหมดทดลอง)

- 1 วันที่แบบเต็ม "วันศุกร์ที่ 9 ตุลาคม 2569" จาก formatThaiDayLong (today) เปลี่ยนเองตอนเที่ยงคืน
- 2 คำทักทาย "สวัสดิ์ ผู้ทดลองใช้" (ชื่อจากโปรไฟล์) โฟกัสขวา คือ จำนวนวันที่จดต่อเนื่อง
- 3 นื่องกล่าวพูด สรุปสถานะเงินเป็นประโยคเดียว (FR-5/FR-6 ใช้ร่วม) แตะแล้วให้เคล็ดลับ
- 4 แลบทโหมดทดลอง บอกว่าข้อมูลเป็นตัวอย่าง อยู่ในเครื่องนี้
- 5 การ์ดเขียว (hero): ป้ายเล็ก "ยอดคงเหลือ" และ **฿11,113.00** ตัวใหญ่สุดของหน้า นับขึ้นจาก 0 ตอนเปิด (ถ้าเปิด "ลดการเคลื่อนไหว" จะแสดงทันที)
- 6 ป้าย "สบาย ๆ" และ **ต้นไม้เงิน** สะท้อนสถานะ Runway (FR-6)
- 7 เงินพอใช้อีก 15 วัน - ถึง 24 ต.ค. ตัวเลขทอง (FR-6) กด "ดูรายละเอียด" ไปเก็บเงินพอถึง
- 8 วันนี้ใช้ได้ประมาณ **฿461 / วัน** ถ้าอยากให้เงินพอถึงสิ้นเดือน (FR-6)

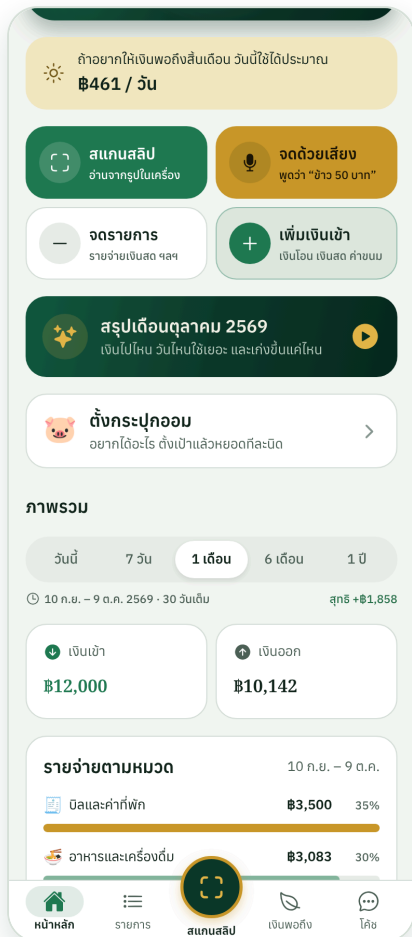
i FR-2 กับ FR-6 อยู่บนการ์ดเดียวกัน

การ์ดเขียวแสดงทั้ง ยอดคงเหลือ (FR-2) และ จำนวนวันที่เงินพอใช้ (FR-6) เพราะ FR-6 เอายอดคงเหลือของ FR-2 ไปคำนวณต่อ ทั้งสองตัวเลขมาจาก useMoney () ฟังก์ชันเดียว จึงไม่มีทางที่เลขสองตัวจะใช้อยอดคนละค่า

บทที่ 6

ทัวร์หน้าจอ: กลางหน้าหลัก

สิ่งที่ทำได้ทันที และทางลัดไปสรุปเดือนกับกระปุกออม



ใต้การ์ดเขียว: ใช้ได้วันละเท่าไร ปุ่มลัด 4 ปุ่ม สรุปเดือน และกระปุกออม

- 1 การ์ดทองอ่อน "วันนี้ใช้ได้ประมาณ 461 / วัน" (FR-6)
- 2 ปุ่มลัด 4 ปุ่ม: สแกนสลิป (FR-4), จดด้วยเสียง (FR-3), จดย่อยรายการ และเพิ่มเงินเข้า (FR-1)
- 3 การ์ด "สรุปเดือนตุลาคม 2569" พื้นเขียวไล่สี มีปุ่มเล่น ▶ กดแล้วเปิดเรื่องเล่าเต็มจอ แสดงเมื่อเดือนนี้มีรายการยืนยันแล้วอย่างน้อย 3 รายการ (หรือช่วง 7 วันแรกของเดือนใหม่ จะเสนอเดือนที่แล้ว)
- 4 การ์ดกระปุกออม (FR-6) ยังไม่มีกระปุก จึงชวน "ตั้งกระปุกออม"
- 5 เริ่มส่วน "ภาพรวม" ของ FR-2 (หน้าถัดไป)

ทัวร์หน้าจอ: ภาพรวม (เลือกช่วงได้ 5 แบบ)

ส่วนที่เป็น FR-2 โดยตรง: แต่ละปุ่มช่วง ตัวเลขทุกตัวในส่วนนี้คำนวณใหม่ทันที



- 1 ปุ่มเลือกช่วง** วันนี้ | 7 วัน | 1 เดือน | 6 เดือน | 1 ปี (เริ่มที่ 1 เดือน)
- 2 ช่วงเวลาจริง** 🕒 "10 ก.ย. - 9 ต.ค. 2569 · 30 วันเต็ม" ไม่ต้องเดาว่า "1 เดือน" นับตั้งแต่เมื่อไร
- 3 สุทธิ +฿1,858** สีเขียวเพราะเงินเข้ามากกว่าออก (ติดลบจะเป็นสีแดง)
- 4 เงินเข้า ฿12,000** (สีเขียว) และ **เงินออก ฿10,142** (สีแดง) นับขึ้นเมื่อเปลี่ยนช่วง
- 5 รายจ่ายตามหมวด** มุมขวาดอกวันแรก-วันสุดท้าย "10 ก.ย. - 9 ต.ค." แสดง 5 หมวดที่มากที่สุด แยกละ: ไอคอน ชื่อหมวด ยอด และเปอร์เซ็นต์ แถบแรกสีทอง ที่เหลือสีเขียวจางลงเรื่อย ๆ
- 6 รายรับตามหมวด** (สูงสุด 3 หมวด) คั่นด้วยเส้น

ภาพรวม ช่วง '1 เดือน' (ค่าเริ่มต้นเมื่อเปิดแอป)

ตัวเลขในภาพ (ช่วง 1 เดือน) เมื่อคำนวณด้วยฟังก์ชันจริง:

หมวดรายจ่าย	ยอด	ส่วนแบ่ง	จำนวนรายการ
📄 บิลและค่าที่พัก	฿3,500	35%	1
🍽️ อาหารและเครื่องดื่ม	฿3,083	30%	51
🛒 ช้อปปิ้ง	฿1,273	13%	4
🎬 บันเทิง	฿811	8%	4
🏪 ร้านสะดวกซื้อ	฿712	7%	13
🚗 เดินทาง (ไม่แสดง เพราะเกิน 5 หมวด)	฿609	6%	17
📚 การเรียน (ไม่แสดง)	฿154	2%	4
รวม	฿10,142	100%	94

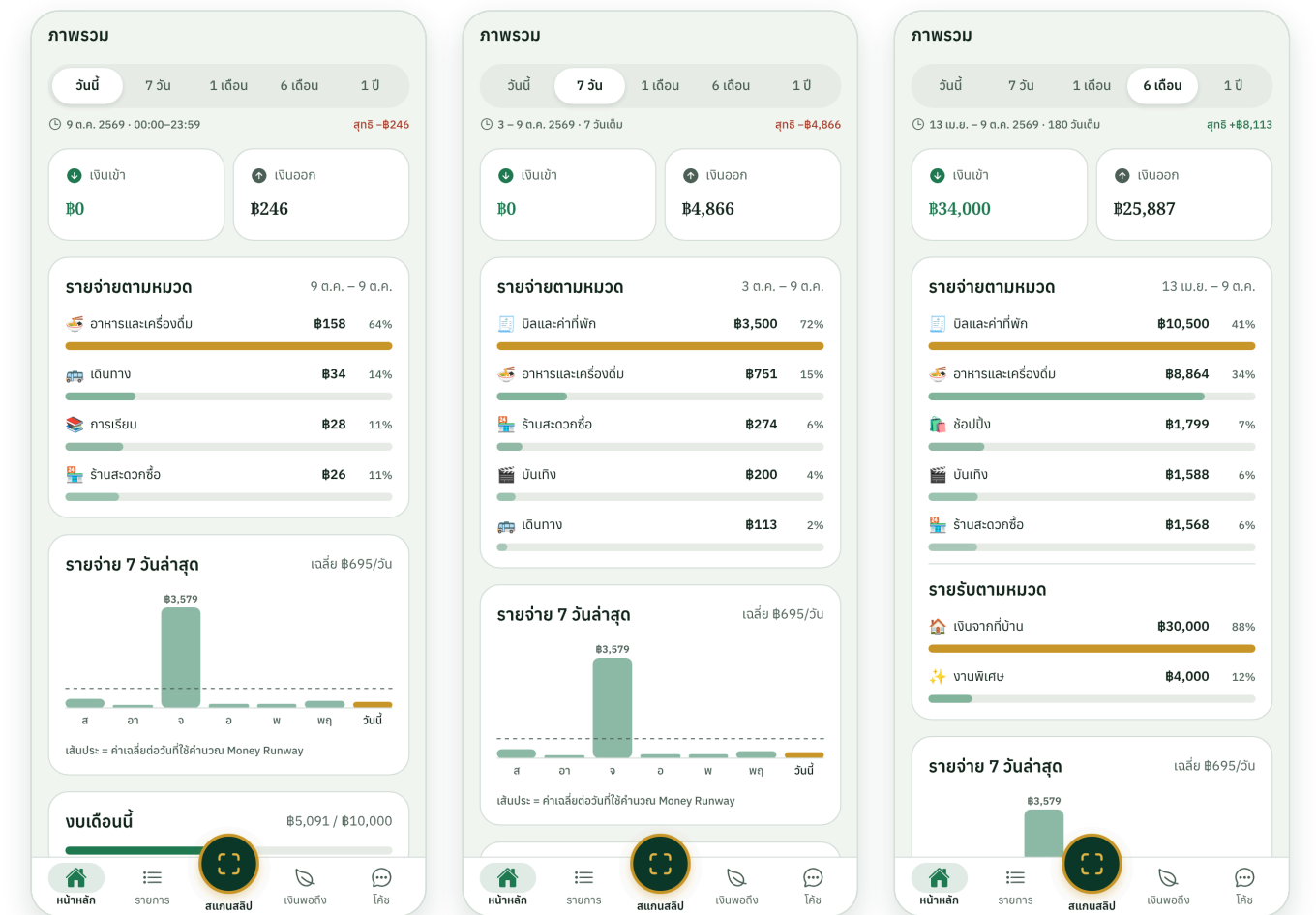
รายรับ: 🏠 เงินจากที่บ้าน ฿10,000 (83%) และ ✨ งานพิเศษ ฿2,000 (17%) รวม ฿12,000

! สังเกต: 2 หมวดสุดท้ายไม่ขึ้นบนจอ

CategoryBars แสดงแค่ 5 หมวดแรก (ค่า limit = 5) เดินทาง ฿609 และการเรียน ฿154 จึงไม่ปรากฏ

เปอร์เซ็นต์ที่เห็นบนจอรวมกันได้ 93% ไม่ใช่ 100% ฉะนั้นก็เรื่องนี้ไว้ในหัวข้อข้อจำกัดท้ายเล่ม

เปลี่ยนช่วง: ตัวเลขเปลี่ยนยังงี้



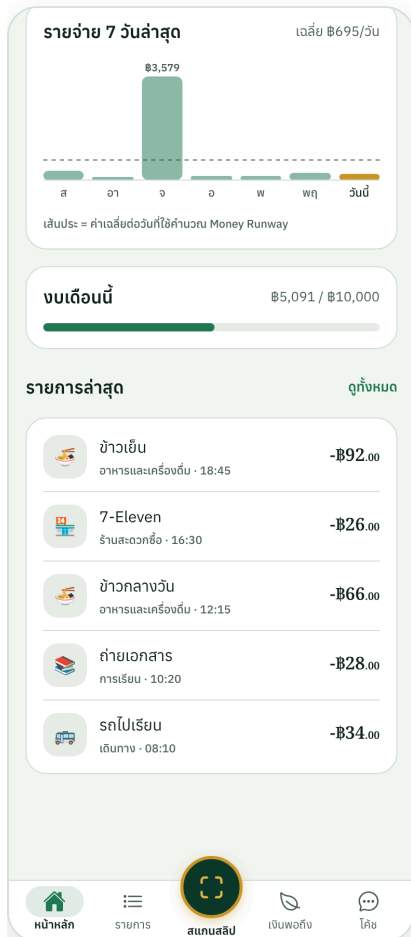
ช่วง	ข้อความใต้ปุ่ม	เงินเข้า	เงินออก	สุทธิ	หมวดรายจ่ายอันดับ 1
1 ปี	10 ต.ค. 2568 – 9 ต.ค. 2569 · 365 วัน เต็ม	฿34,000	฿25,887	+ ฿8,113	บิลและค่าที่พัก ฿10,500 (41%)

i ทำไม 6 เดือนกับ 1 ปีได้ตัวเลขเท่ากัน

ข้อมูลตัวอย่างสร้างขึ้นหลังแค่ 75 วัน (ตั้งแต่ 27 ก.ค.) ทุกรายการจึงอยู่ในทั้งสองช่วง ถ้าเป็นผู้ใช้จริงที่ใช้แอปมานานกว่า 6 เดือน ตัวเลข 1 ปีจะมากกว่า สังเกตด้วยว่าปีในข้อความ "10 ต.ค. 2568" แสดงเพราะวันแรกกับวันสุดท้ายอยู่คนละปี

ทัวร์หน้าจอ: กราฟ 7 วัน งบเดือน และรายการล่าสุด

สามการ์ดสุดท้ายของหน้าหลัก

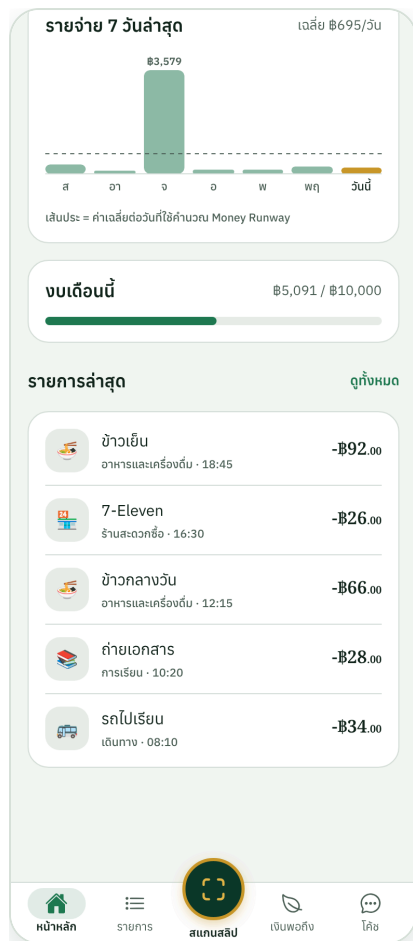


- 1 หัวการ์ด "รายจ่าย 7 วันล่าสุด" มุมขวา "เฉลี่ย ฿695/วัน"
- 2 แท่ง 7 แท่ง เรียงจากเก่าไปใหม่ ชื่อวันใต้แท่ง (ส อา จ อ พ พก) แท่งสุดท้ายเขียนว่า "วันนี้" และเป็น **สีทองเข้ม** แท่งอื่นสีเขียวจาง
- 3 ป้าย **฿3,579** เหนือแท่งที่สูงที่สุดเท่านั้น (วันจันทร์ 5 ต.ค. วันจ่ายค่าหอ)
- 4 **เส้นประ** = ค่าเฉลี่ยต่อวันที่ FR-6 ใช้คำนวณ Money Runway คำอธิบายเขียนใต้กราฟ
- 5 **งบเดือนนี้** ฿5,091 / ฿10,000 (51%) แถบสีหลักเพราะยังไม่เกิน 85% ของงบ

รายจ่าย 7 วันล่าสุด และงบเดือนนี้

วัน	3 ต.ค. (ส)	4 (อา)	5 (จ)	6 (อ)	7 (พ)	8 (พก)	9 (วันนี้)	รวม
รายจ่าย	฿335	฿124	฿3,579	฿154	฿167	฿261	฿246	฿4,866

ค่าเฉลี่ย = $฿4,866 \div 7 = ฿695.14$ แสดงเป็น ฿695 · สังเกตว่ารวม 7 วันตรงกับ "เงินออก" ของช่วง 7 วันพอดี เพราะทั้งสองใช้นิยาม "7 วันรวมวันนี้" เหมือนกัน

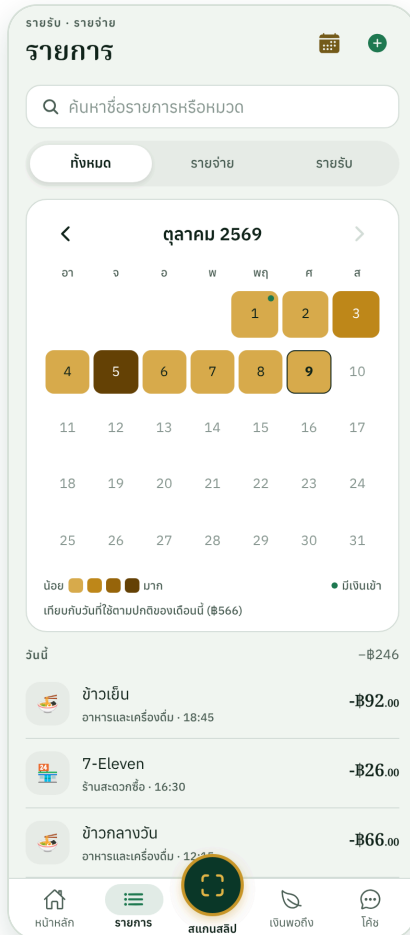


รายการล่าสุด 5 รายการ

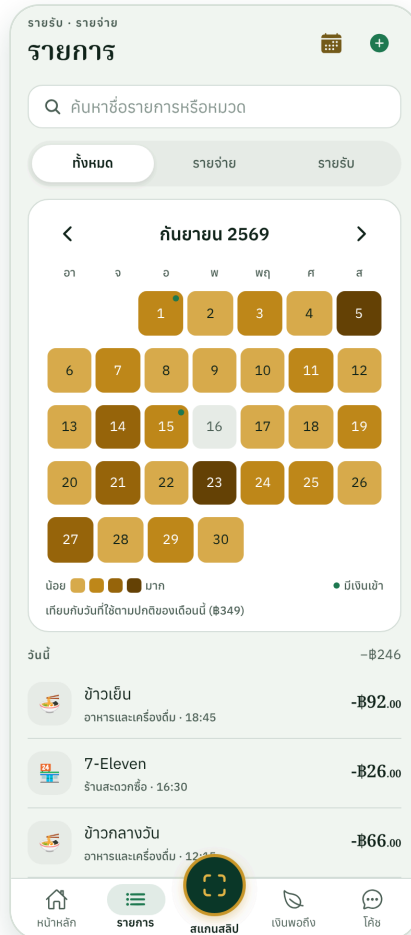
- 1 หัวข้อ "รายการล่าสุด" และลิงก์ "ดูทั้งหมด" ไปเก็บรายการ
- 2 5 รายการล่าสุดที่ **ยืนยันแล้ว** เรียงจากเวลาใหม่ไปเก่า: ข้าวเย็น -฿92 (18:45), 7-Eleven -฿26 (16:30), ข้าวกลางวัน -฿66 (12:15), ถ่ายเอกสาร -฿28 (10:20), รถไปเรียน -฿34 (08:10) รวม ฿246 = เงินออกวันนี้
- 3 แต่ละแถวคือ TxRow ตัวเดียวกับเก็บรายการ (FR-1) และเพื่อแก้ไขได้

ทิวทัศน์หน้าจอ: ปฏิทินการใช้จ่าย

อยู่บนสุดของแท็บรายการ เห็นทั้งเดือนในครั้งเดียวว่าวันไหนใช้เยอะ



ตุลาคม 2569: มีข้อมูลถึงวันที่ 9 (วันนี้มีรอบ) วันที่ 10 เป็นต้นไปยังไม่มาถึง



กวดเดือนก่อน: กันยายน 2569 ทั้งเดือน



และวันที่ 5 ก.ย.: แถบล่างบอก 'ใช้ ฿3,760 • 6 รายการ' และรายการเหลือเฉพาะวันนั้น

- หัวปฏิทิน:** ปุ่ม < เดือนก่อน, ชื่อเดือนแบบไทย, ปุ่ม > เดือนถัดไป ปุ่ม < จะจางและกดไม่ได้เมื่อถึงเดือนเก่าสุดที่มีรายการ ส่วนปุ่ม > จางเมื่อเป็นเดือนนี้ (ดูอนาคตไม่ได้)
- หัวคอลัมน์** อา จ อ พ พก ศ ส (สัปดาห์เริ่มวันอาทิตย์) 1 ต.ค. 2569 ตรงกับวันพฤหัสบดี ช่องแรก 4 ช่องจึงว่าง
- สีของแต่ละวัน** ไล่จากทองอ่อน (ใช้น้อย) ถึงน้ำตาลเข้ม (ใช้มาก) โดย **เทียบกับ "วันปกติ" ของเดือนนั้น** (ค่าเฉลี่ยของวันที่มีรายจ่าย) วันที่ไม่ใช้เงินเป็นสีเทาอ่อน
- จุดเขียว** มุมขวบน = วันนั้นมีเงินเข้า (1 ก.ย. ค่าขนม, 15 ก.ย. ค่าจ้างสอนพิเศษ)
- คำอธิบายสี** น้อย □□□□ มาก • • มีเงินเข้า และบรรทัด "เทียบกับวันที่ใช้ตามปกติของเดือนนี้ (฿349)"
- แตะวัน** → กรอบสีหลัก แถบสรุปวันนั้น และปุ่ม "ดูทุกวัน" แตะวันเดิมซ้ำหรือกดดูทุกวันเพื่อยกเลิก

≈ เหมือนกราฟการเข้าใช้งานของ GitHub

ใครเคยเห็นตารางสีเหลี่ยมสีเขียวในหน้าโปรไฟล์ GitHub (ยังเขียนโค้ดเยอะยังเข้ม) ปกติมันใช้ไอเดียเดียวกัน เรียกว่า **heatmap** แต่ใช้สีทองและเทียบกับ "วันปกติของเดือน" แทนตัวเลขตายตัว เพราะคนใช้เงินวันละ ฿100 กับคนใช้วันละ ฿1,000 ควรเห็นภาพแบบเดียวกันเมื่อใช้ "มากกว่าปกติ"

ทัวร์หน้าจอ: สรุประดับ (เรื่องเล่า 7 หน้าจอ)

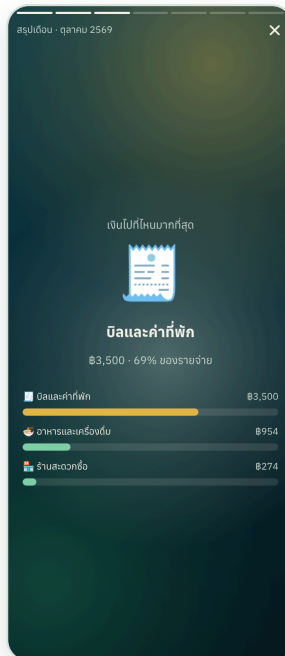
แต่ละการ์ดสรุปเดือนบนหน้าหลัก แล้วแอปเล่าเรื่องเดือนนั้นทีละหน้าจอเหมือนสตอรี่



1 เปิดเรื่อง: 'ตุลาคม 2569 · 9 วันแรก'



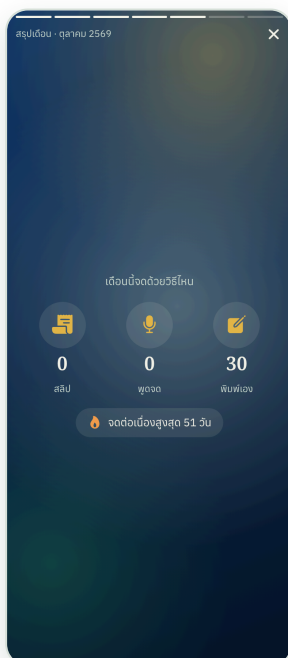
2 ใช้จ่ายไปแล้ว ฿5,091 จาก 30 รายการ เฉลี่ยวันละ ฿566 · เงินเข้า ฿10,000



3 เงินไปไหน: บิลและค่าที่พัก ฿3,500 (69%)



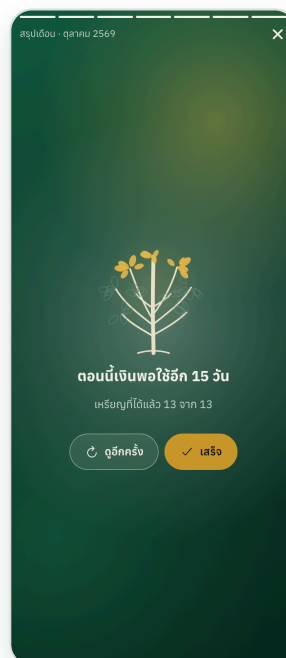
4 วันที่ใช้เยอะสุด 5 ต.ค. ฿3,579 · 0 วันไม่ใช้เงิน · จ่ายบ่อยสุด กาแฟยามเช้า 7 ครั้ง



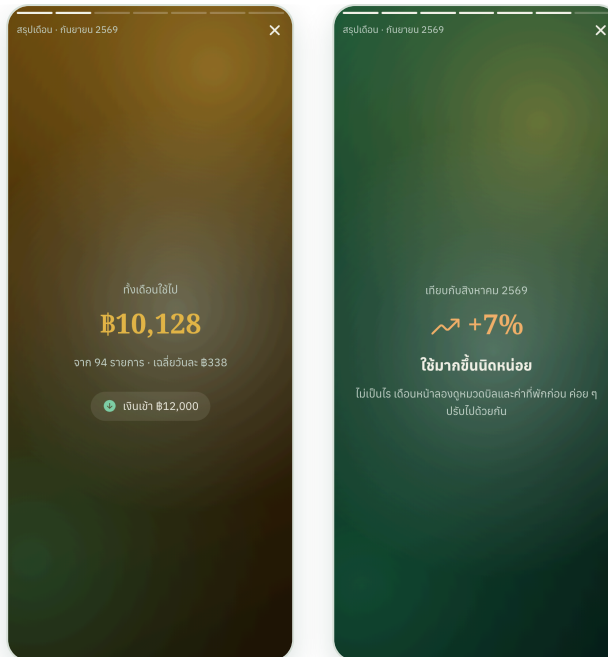
5 จัดด้วยวิธีไหน: สลิป 0 · พูดจด 0 · พิมพ์เอง 30 · ต่อเนื่องสูงสุด 51 วัน



6 เทียบกันยายน (9 วันแรกเท่ากัน): 0% 'เท่าเดิมพอดี'



7 ปิดเรื่อง: 'ตอนนี้เงินพอใช้อีก 15 วัน' ปุ่มดูอีกครั้ง/เสร็จ



สรุปเดือนกันยายน (ทั้งเดือน): ใช้ไป ฿10,128 จาก 94 รายการ

กันยายนเทียบกับสิงหาคม: +7% 'ใช้มากกว่าเดือนก่อน'

วิธีใช้

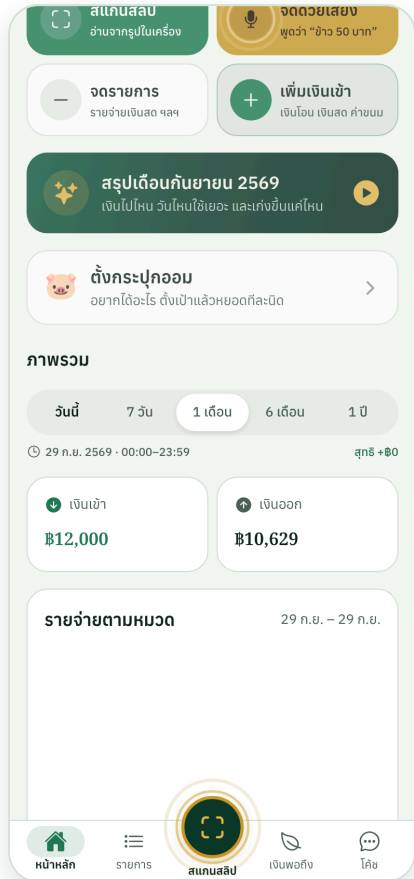
- แถบบางด้านบน 7 ช่อง = 7 หน้าจอ ช่องปัจจุบันค่อย ๆ เติบโตใน 6.5 วินาที แล้วไปหน้าถัดไปเอง
- **แถบด้านขวา (70% ของจอ) = แถบด้านซ้าย (30%) = ย้อนกลับ · × มุมขวาบน = ปิด**
- หน้าสุดท้ายมีฟลลอป และปุ่ม "ดูอีกครั้ง" กับ "เสร็จ"
- ตัวเลขทุกตัวมาจาก monthRecap ตัวเดียว (ส่วน "15 วัน" มาจาก FR-6 และ "51 วัน", "13 จาก 13" มาจากระบบเหรียญ)

★ ทำไม 0% ทั้งที่ ฿5,091 ไม่เท่ากับเดือนก่อนพอดี

9 วันแรกของกันยายนใช้ไป ฿5,077 ส่วน 9 วันแรกของตุลาคมใช้ ฿5,091 ต่างกัน 0.28% เมื่อปิดเป็นจำนวนเต็มจึงเป็น 0% แอปจึงขึ้นว่า "เท่าเดิมพอดี" (ตรวจสอบด้วยการรับโค้ดจริงแล้ว)

ทิวทัศน์หน้าจอ: เมื่อข้ามเที่ยงคืน

เปิดแอปค้างไว้ข้ามคืน หน้าจอต้องไม่โกหกกว่า 'วันนี้' ยังเป็นเมื่อวาน



ภาพจาก E2E: นาฬิกาจำลอง 23:59:40 ของ 28 ก.ย. แล้วเดินผ่านเที่ยงคืน

เทสต์ E2E ข้อ "A new day" ตั้งนาฬิกาของเบราร์เซอร์ เป็น 23:59:40 วันที่ 28 ก.ย. 2569 (เวลาไทย) เปิดแอป แล้วเร่งเวลาไป 30 วินาที ผลที่ต้องเห็น:

1. วันที่บนหน้าหลักเปลี่ยนจาก "วันจันทร์ที่ 28 กันยายน 2569" เป็น "วันอังคารที่ 29 กันยายน 2569" เอง โดยไม่ต้องรีเฟรช
2. นื่องคล้าย "วันใหม่แล้ว!" พร้อมยอดที่ใช้เมื่อวาน
3. เลือกช่วง "วันนี้" แล้วข้อความเป็น "29 ก.ย. 2569 - 00:00-23:59"

กลไกอยู่ใน AppProvider (อ่านโค้ดในบท "ข้อมูลกลาง") ซึ่งตั้งเวลาปลุกไว้ที่เที่ยงคืนเวลาไทยถัดไป และเช็กอีกครั้ง เมื่อแอปกลับมาอยู่หน้าจอ

หลักการคำนวณ: ยอดคงเหลือ

สูตรเดียวที่สำคัญที่สุดของ FR-2 คิดตามได้ด้วยเครื่องคิดเลข

ยอดคงเหลือ = (ยอดตั้งต้น + ยอดยกมา) + Σ รายรับที่ยืนยัน - Σ รายจ่ายที่ยืนยัน

Σ (ซิกมา) แปลว่า "รวมทุกตัว" · ยอดตั้งต้น = เงินที่ผู้ใช้มีตอนเริ่มใช้แอป (ตั้งในหน้าตั้งค่าเริ่มต้น) · ยอดยกมา = ผลสุกของรายการที่เก่ากว่า 400 วัน (ไม่ได้ไหลมาเป็นรายการ แต่ยังนับในยอด)

ตัวอย่างที่ 1: ข้อมูลตัวอย่างในภาพหน้าจอ

ส่วนประกอบ	จำนวน
ยอดตั้งต้นของโหมดทดลอง (DEMO_OPENING_BALANCE_SATANG = 300,000 สตางค์)	฿3,000
ยอดยกมา (โหมดทดลองเก็บทุกอย่างในเครื่อง ไม่มีของเก่าเกิน 400 วัน)	฿0
+ รายรับที่ยืนยันทั้งหมด (ค่าขนม ฿10,000 × 3 เดือน + ค่าสอนพิเศษ ฿2,000 × 2 ครั้ง)	+฿34,000
- รายจ่ายที่ยืนยันทั้งหมด (232 รายการ ตั้งแต่ 27 ก.ค.)	-฿25,887
ยอดคงเหลือ	฿11,113 ✓ ตรงกับภาพ

ตัวอย่างที่ 2: แบบร่างไม่นับ (TC-09)

ยอดตั้งต้น ฿5,000 มีรายการ: รายรับ ฿9,000 (1 ก.ย.), รายจ่าย ฿100 (27 ก.ย.), รายจ่าย ฿50 (25 ก.ย.) และ แบบร่าง ฿999 (27 ก.ย.)

$5,000 + 9,000 - 100 - 50 = ฿13,850$ (ไม่ใช่ ฿12,851)

≈ นึกถึงกระเป๋าออมสินจริง

ยอดตั้งต้นคือเงินในกระปุกวันแรก ทุกรายรับคือหยอดเหรียญ ทุกรายจ่ายคือเคาะเหรียญออก ส่วนแบบร่างคือ "ใบเสร็จที่ยังไม่แน่ใจว่าเป็นของเรา" วางไว้ข้างกระปุก ยังไม่หยิบเงินออกจนกว่าจะตรวจแล้ว

หลักการคำนวณ: ช่วงเวลา และการสรุปตามช่วง

'1 เดือน' คือ 30 วันเต็มนับรวมวันนี้ ไม่ใช่ 'ตั้งแต่วันที่ 1'

วันสุดท้าย = วันนี้ (เวลาไทย) · วันแรก = วันนี้ - (จำนวนวันของช่วง - 1)

เหตุผล 1: ช่วง 7 วันที่รวมวันนี้ เริ่มจาก 6 วันก่อน ไม่ใช่ 7 วันก่อน (นับนิ้ว: 3, 4, 5, 6, 7, 8, 9 ค.ค. = 7 วัน)

ปุ่ม	จำนวนวัน (RANGE_DAYS)	วันนี้ = 9 ค.ค. 2569 ได้ช่วง	วันนี้ = 27 ก.ย. 2569 (ใน TC-08)
วันนี้	1	9 ค.ค. – 9 ค.ค.	27 ก.ย. – 27 ก.ย.
7 วัน	7	3 – 9 ค.ค.	21 – 27 ก.ย.
1 เดือน	30	10 ก.ย. – 9 ค.ค.	29 ส.ค. – 27 ก.ย.
6 เดือน	180	13 เม.ย. – 9 ค.ค.	1 เม.ย. – 27 ก.ย.
1 ปี	365	10 ค.ค. 2568 – 9 ค.ค. 2569	28 ก.ย. 2568 – 27 ก.ย. 2569

ขั้นตอนของ summarizeRange (ทำทุกครั้งที่แตะปุ่มช่วง)

- 1. หาวันแรก-วันสุดท้ายของช่วง (ตารางข้างบน)
- 2. เก็บเฉพาะรายการที่ **ยืนยันแล้ว** และ **วันไทย** ของรายการอยู่ในช่วง
- 3. รวมยอดรายรับ และรวมยอดรายจ่าย แยกกัน
- 4. สุทธิ = รายรับ – รายจ่าย
- 5. แยกตามหมวด: รวมยอดและนับจำนวนของแต่ละหมวด แล้วคิดส่วนแบ่ง = ยอดหมวด ÷ ยอดรวมของประเภทนั้น เรียงมากไปน้อย

ตัวอย่างที่ 3: ทำตาม TC-10 ด้วยมือ (วันนี้ = 27 ก.ย. 2569)

รายการ	อยู่ในช่วง 7 วัน (21–27 ก.ย.)?	อยู่ในช่วง 1 เดือน (29 ส.ค.–27 ก.ย.)?
27 ก.ย. รายจ่าย อาหาร \$100	✓	✓
25 ก.ย. รายจ่าย เดินทาง \$50	✓	✓
1 ก.ย. รายรับ เงินจากที่บ้าน \$9,000	× (ก่อน 21 ก.ย.)	✓
27 ก.ย. แบบร่าง \$999	× ไม่นับ (ยังไม่ยืนยัน)	× ไม่นับ

ผล 7 วัน: เงินเข้า \$0 · เงินออก \$150 · อาหาร \$100 = 100 ÷ 150 = **66.7%** · เดินทาง \$50 = **33.3%**
ผล 1 เดือน: เงินเข้า \$9,000 · เงินออก \$150 · สุทธิ + \$8,850

i ส่วนแบ่งของรายจ่ายกับรายรับคิดแยกกัน

เปอร์เซ็นต์ของหมวดรายจ่ายเทียบกับ "รายจ่ายรวม" เท่านั้น ไม่ใช่เทียบกับเงินทั้งหมด ในภาพ 1 เดือน บิล ฿3,500 จึงเป็น 35% ของ ฿10,142 (ไม่ใช่ของ ฿22,142) ส่วนรายรับ เงินจากที่บ้าน ฿10,000 เป็น 83% ของรายรับ ฿12,000

ความยาวแถบหมวด

แถบของ CategoryBars ไม่ได้ยาวตามเปอร์เซ็นต์ แต่ยาว **เทียบกับหมวดที่มากที่สุดที่แสดงอยู่** หมวดอันดับ 1 จึงเต็มแถบเสมอ (บิล ฿3,500 = เต็ม, อาหาร ฿3,083 = $3,083 \div 3,500 = 88\%$ ของแถบ) ทำให้เห็นความต่างระหว่างหมวดชัด แม้หมวดแรกจะมีส่วนแบ่งแค่ 35%

หลักการคำนวณ: กราฟ 7 วัน และงบเดือน

วันที่ไม่ได้ใช้เงินก็ต้องมีแท่ง (สูง 0) ไม่ใช่หายไป

dailyTotals: 7 ช่องเสมอ

ถ้าสร้างกราฟจาก "วันที่มีรายการ" อย่างเดียว วันที่ไม่ได้ใช้เงินจะหายไป กราฟจะดูเหมือนใช้เงินทุกวัน และแท่งจะไม่ตรงกับชื่อวัน พังค์ชันจึง **สร้างช่องครบ 7 วันก่อน** (ค่าเป็น 0) แล้วค่อยเติมยอดเข้าไป (TC-11 ทดสอบว่าวันที่ว่างได้ 0)

ขั้น	ผลลัพธ์ (ตัวอย่าง TC-11 วันที่ 27 ก.ย.)
1. สร้าง 7 ช่อง จากค่าไปใหม่	21, 22, 23, 24, 25, 26, 27 ก.ย. ทุกช่อง = ฿0
2. เติมรายจ่ายที่ยืนยันแล้ว	25 ก.ย. = ฿50, 27 ก.ย. = ฿100
3. ผล	[0, 0, 0, 0, 50, 0, 100] — ช่องที่ 6 (26 ก.ย.) = 0 ตามที่เกสต์ต้องการ

ความสูงของแท่ง

ความสูง = ยอดของวัน ÷ ค่าสูงสุด × ความสูงพื้นที่กราฟ โดย **ค่าสูงสุด = ค่ามากที่สุดระหว่าง (แท่งที่สูงที่สุด, ค่าเฉลี่ย, 1)** เพื่อให้เส้นประค่าเฉลี่ยอยู่ในกราฟเสมอ และไม่หารด้วยศูนย์ถ้าทั้งสปีดาร์ไม่ได้ใช้เงิน แท่งที่มีรายจ่ายแต่น้อยมากจะสูงอย่างน้อย 3 จุดให้มองเห็น

งบเดือนนี้

ใช้ไปเดือนนี้ = Σ รายจ่ายที่ยืนยัน ตั้งแต่วันที่ 1 ของเดือนปฏิทินนี้ (เวลาไทย)

สีแถบ: ≤ 85% ของงบ = สีส้ม · เกิน 85% = เหลือง (watch) · เกินงบ = แดง (critical) · การ์ดนี้แสดงเฉพาะเมื่อผู้ใช้ตั้งงบไว้

งบ ฿10,000 ใช้ไป	สัดส่วน	สีแถบ
฿5,091 (ภาพหน้าจอ)	51%	สีส้ม
฿8,500	85% พอดี	สีส้ม (ต้อง "เกิน" 85%)
฿8,600	86%	เหลือง
฿10,001	100.01%	แดง

! 'งบเดือนนี้' กับ 'ช่วง 1 เดือน' นับไม่เหมือนกัน

งบเดือนนี้นับ **ตามเดือนปฏิทิน** (1–9 ต.ค. = ฿5,091) เพราะงบของคนส่วนใหญ่ตั้งเป็นรายเดือน ส่วนปุ่ม "1 เดือน" นับ **30 วันย้อนหลัง** (10 ก.ย.–9 ต.ค. = ฿10,142) เพราะใช้เปรียบเทียบช่วงยาวเท่ากันได้ทุกวัน สองตัวเลขนี้จึงต่างกันได้ และแอปเขียนชื่อต่างกันชัดเจน

หลักการคำนวณ: ปฏิทินไล่สี

สีของวันไม่ได้ขึ้นกับจำนวนบาทตายตัว แต่เทียบกับ 'วันปกติ' ของเดือนนั้น

วันปกติ = (รวมรายจ่ายของวันที่มีรายจ่าย) ÷ (จำนวนวันที่มีรายจ่าย) · อัตราส่วน =

รายจ่ายของวัน ÷ วันปกติ

ระดับ 0 = ไม่มีรายจ่าย · ระดับ 1 = อัตราส่วน < 0.5 · ระดับ 2 = < 1 · ระดับ 3 = < 2 · ระดับ 4 = ตั้งแต่ 2 เท่าขึ้นไป

ตัวอย่างที่ 4: TC-70 (ก.ย. 2569, วันนี้ = 15 ก.ย.)

รายจ่ายยืนยัน: 1 ก.ย. ฿100, 2 ก.ย. ฿40, 3 ก.ย. ฿400, 4 ก.ย. ฿60 (และเงินเข้า ฿0.50 วันที่ 4, แบบร่าง ฿999 วันที่ 5)

วันปกติ = (100 + 40 + 400 + 60) ÷ 4 = ฿150

วัน	รายจ่าย	อัตราส่วน	ระดับ
1 ก.ย.	฿100	0.67	2
2 ก.ย.	฿40	0.27	1
3 ก.ย.	฿400	2.67	4
4 ก.ย.	฿60 (+ เงินเข้า)	0.40	1 และมีจุดเขียว
5 ก.ย.	แบบร่าง ฿999	—	0 (แบบร่างไม่นับ)

ตัวอย่างที่ 5: ปฏิทินกันยายนในภาพหน้าจอ

กันยายนมีรายจ่าย 29 วัน รวม ฿10,128 วันปกติ = 10,128 ÷ 29 = ฿349.24 (บนจอปัดเป็น ฿349)

วัน	รายจ่าย	อัตราส่วน	ระดับ
13 ก.ย.	฿44	0.13	1 (ทองอ่อนสุด)
1 ก.ย.	฿182	0.52	2
14 ก.ย.	฿456	1.31	3
23 ก.ย.	฿911	2.61	4 (เข้มสุด)
5 ก.ย.	฿3,760 (ค่าหอ ฿3,500 + อีก 5 รายการ)	10.77	4
16 ก.ย.	฿0	—	0 (เทา)

i ทำไมตุลาคมเกือบทุกวันเป็นสีอ่อนสุด

ตุลาคม (ถึงวันที่ 9) มีค่าหอ ๓3,500 วันที่ 5 ทำให้วันปกติสูงถึง ๓565.67 วันอื่นที่ใช้ ๓97-๓261 จึงต่ำกว่าครึ่งของวันปกติ (ระดับ 1) ทั้งหมด มีเพียงวันที่ 3 (๓335 = 0.59 เท่า) ที่เป็นระดับ 2 เป็นข้อจำกัดที่ฉับพลันที่กไว้: วันที่จ่ายก่อนใหญ่ครึ่งเดียวถึงค่าปกติขึ้น

หลักการคำนวณ: สรุปเดือน

ทุกตัวเลขในเรื่องเล่า และกติกาการเทียบเดือนก่อนแบบยุติธรรม

ตัวเลข	วิธีคิด	ตุลาคม (ถึง 9 ต.ค.)
วันที่นับ	เดือนที่ยังไม่จบ = ถึงวันนี้ · เดือนที่จบแล้ว = ทั้งเดือน	9 วัน ("9 วันแรก")
ใช้ไป / เงินเข้า / จำนวนรายการ	รายการที่ยืนยันในเดือน และไม่เกินวันนี้	฿5,091 / ฿10,000 / 30
เฉลี่ยวันละ	ใช้ไป ÷ วันที่นับ (ปิด)	5,091 ÷ 9 = ฿566
เงินไปไหน (3 อันดับ)	รวมตามหมวด เรียงมากไปน้อย ส่วนแบ่ง = หมวด ÷ ใช้ไป	บิล ฿3,500 (69%), อาหาร ฿954, สะดวกซื้อ ฿274
วันที่ใช้เยอะสุด	รวมตามวัน เอาวันที่มากที่สุด (เท่ากับเอาวันก่อน)	5 ต.ค. ฿3,579
วันที่ไม่ใช้เงินเลย	วันที่นับ – จำนวนวันที่มีรายจ่าย	9 – 9 = 0
จ่ายบ่อยที่สุด	นับตามชื่อ (ตัดช่องว่าง ไม่สนตัวใหญ่เล็ก) ต้องอย่างน้อย 2 ครั้ง	กาแฟยามเช้า 7 ครั้ง
จดด้วยวิธีไหน	สลิป = source 'slip' · พุดจด = บันทึกเป็นเครื่องหมายเสียง · พิมพ์เอง = ที่เหลือ	0 / 0 / 30
เทียบเดือนก่อน	เทียบกับ จำนวนวันแรกเท่ากัน ของเดือนก่อน	9 วันแรกของ ก.ย. = ฿5,077 → +0.28% → 0%

ทำไมต้องเทียบ "จำนวนวันเท่ากัน"

วันที่ 9 ต.ค. ถ้าเอายอด 9 วันของตุลาคม (฿5,091) ไปเทียบกับกันยายนทั้งเดือน (฿10,128) จะได้ -50% แอปจะชมว่า "ใช้น้อยลง เก่งมาก!" ทั้งที่จริงใช้พอ ๆ กัน การเทียบ 9 วันแรกกับ 9 วันแรกจึงยุติธรรม ถ้าเดือนก่อนสั้นกว่า (เช่น ก.พ. 28 วัน) จะใช้จำนวนวันเท่าที่เดือนก่อนมี

สรุปเดือน	ยอดเดือนนี้	ยอดเดือนก่อน (จำนวนวันเท่ากัน)	ผล
ตุลาคม (9 วัน)	฿5,091	1–9 ก.ย. ฿5,077	0% "เท่าเดิมพอดี"
กันยายน (30 วัน)	฿10,128	1–30 ส.ค. ฿9,504	+7% "ใช้มากขึ้นนิดหน่อย"
สิงหาคม (31 วัน)	฿9,723	1–31 ก.ค. ฿945 (ข้อมูลเริ่ม 27 ก.ค.)	+929%

เมื่อไหร่การ์ดสรุปเดือนขึ้นบนหน้าหลัก (recapToOffer)

- ถ้าวันนี้เป็นวันที่ 1–7 ของเดือน และเดือนที่แล้วมีรายการ → เสนอ **เดือนที่แล้ว** (ต้นเดือนเหมาะกับการสรุปเดือนที่เพิ่งจบ)
- ไม่จิ้น ถ้าเดือนนี้มีรายการยืนยันแล้วอย่างน้อย 3 รายการ → เสนอ **เดือนนี้**
- ไม่จิ้น → ไม่แสดงการ์ด (ยังมีเรื่องไม่พอเล่า)

ถ้าวันนี้คือ	การ์ดที่เสนอ (ข้อมูลตัวอย่าง)
3 ต.ค. 2569	สรุปเดือนกันยายน
7 ต.ค. 2569	สรุปเดือนกันยายน
8 ต.ค. 2569	สรุปเดือนตุลาคม

บทที่ 17

อ่านโค้ด: src/domain/summary.ts ทั้งไฟล์

ไฟล์หัวใจของ FR-2 มี 134 บรรทัด 6 ฟังก์ชัน เป็นฟังก์ชันล้วน (รับข้อมูล คำนวณ ไม่แตะหน้าจอหรือเน็ต) จึงทดสอบได้ง่าย

ส่วนที่ 1: หัวไฟล์ confirmedOnly และ computeBalance (บรรทัด 1-17)

src/domain/summary.ts

บรรทัด 1-17

```
1 /**
2  * FR-2 Overview Dashboard: balance, income, expense and totals by category.
3  * Only CONFIRMED transactions count. Drafts never change a number the user sees.
4  */
5 import { getCategory } from '../categories';
6 import { addDays, bkkDayKey, isDayInRange, rangeDays } from '../dates';
7 import type { RangeKey, Transaction, TxKind } from '../types';
8
9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:1

บรรทัด	ความหมาย
1-4	คอมเมนต์: FR-2 Overview Dashboard: ยอดคงเหลือ รายรับ รายจ่าย และยอดตามหมวด นับเฉพาะรายการที่ยืนยันแล้ว แบบร่างจะไม่เปลี่ยนตัวเลขใดที่ผู้ใช้เห็น (นี่คือกฎข้อที่ 1 ของ FR-2 เขียนไว้บนสุดของไฟล์)
5	นำ getCategory มาใช้ (หาชื่อไทยและไอคอนของหมวดจากรหัส)
6	ฟังก์ชันวันที่ 4 ตัว: บวกวัน, วันไทย, วันอยู่ในช่วงไหม, หาช่วงจากปุ่ม
7	ชนิดข้อมูล: RangeKey ('today' '7d' '1m' '6m' '1y'), Transaction, TxKind
9	confirmedOnly ฟังก์ชันลูกศรสั้น ๆ: รับรายการ คำนวณเฉพาะตัวที่ status เป็น 'confirmed' (filter = คัดกรอง) มี export เพราะ FR-6 (runway.ts) ก็ใช้ตัวเดียวกัน นิยาม "ยืนยันแล้ว" จะได้ตรงกันทั้งแอป
11	คอมเมนต์สูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยันทั้งหมด - รายจ่ายที่ยืนยันทั้งหมด
12	computeBalance รับยอดตั้งต้น (สตางค์) และรายการ คำนวณยอดคงเหลือ (สตางค์)
13	กรองเฉพาะที่ยืนยันแล้ว ใช้ reduce ทยอยทีละรายการ
14	ทุกรายการ: ถ้ารายรับ บวกจำนวน ถ้ารายจ่าย บวกจำนวนติดลบ (= ลบ) เข้ากับผลรวมที่สะสมมา (sum)
15	ค่าเริ่มต้นของผลรวม = ยอดตั้งต้น (ไม่ใช่ 0) จึงไม่ต้องบวกยอดตั้งต้นแยกอีกที

บรรทัด	ความหมาย
--------	----------

16–17	ปิด reduce และปิดฟังก์ชัน
-------	---------------------------

≈ reduce = ทบยอดในสมุด

นี่คือการจดยอดในสมุดบัญชีที่ละบรรทัด: เริ่มจากยอดยกมา (ค่าเริ่มต้น) แล้วบรรทัดละรายการ บวกหรือลบ จดยอดใหม่ไว้ใช้บรรทัดต่อไป จนสมุดก็ได้ยอดสุดท้าย reduce ทำแบบนี้ทุกประการ ตัวแปร sum คือ "ยอดบรรทัดก่อน" และค่าที่ฟังก์ชันคืนคือ "ยอดบรรทัดนี้"

รอบ	รายการ	sum ก่อน	sum หลัง
เริ่ม	—	—	500,000 (ยอดตั้งต้น ฿5,000)
1	รายจ่าย 10,000 (฿100)	500,000	490,000
2	รายจ่าย 5,000 (฿50)	490,000	485,000
3	รายรับ 900,000 (฿9,000)	485,000	1,385,000
—	แบบร่าง 99,900 ถูกกรองทิ้งตั้งแต่บรรทัด 13	ผล = 1,385,000 สตางค์ = ฿13,850 (TC-09)	

ส่วนที่ 2: รูปร่างของผลสรุป (บรรทัด 19–38)

src/domain/summary.ts

บรรทัด 19–38

```

19 export interface CategoryTotal {
20   key: string;
21   label: string;
22   glyph: string;
23   kind: TxKind;
24   totalSatang: number;
25   /** Share of this kind's total, 0..1 */
26   share: number;
27   count: number;
28 }
29
30 export interface RangeSummary {
31   from: string;
32   to: string;
33   incomeSatang: number;
34   expenseSatang: number;
35   netSatang: number;
36   count: number;
37   byCategory: { income: CategoryTotal[]; expense: CategoryTotal[] };
38 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:19

บรรทัด	ความหมาย
--------	----------

19	CategoryTotal = ยอดของ 1 หมวด
----	-------------------------------

บรรทัด	ความหมาย
20–23	รหัสหมวด, ชื่อไทย, ไอคอน, ประเภท (รายรับ/รายจ่าย)
24	ยอดรวมของหมวด (สตางค์)
25–26	ส่วนแบ่ง 0 ถึง 1 ของยอดรวมประเภทนั้น ($0.35 = 35\%$)
27	จำนวนรายการในหมวด
30	RangeSummary = ผลสรุปของ 1 ช่วง
31–32	วันแรกและวันสุดท้ายของช่วง ("YYYY-MM-DD") ใช้แสดงมุมมองของการ์ดหมวด
33–35	รายรับรวม, รายจ่ายรวม, สุทธิ (สตางค์)
36	จำนวนรายการที่นับ
37	ยอดตามหมวด แยกเป็นสองรายการ: ของรายรับ และของรายจ่าย

ส่วนที่ 3: totalsByCategory – รวมตามหมวด (บรรทัด 40–64)

src/domain/summary.ts

บรรทัด 40–64

```

40 function totalsByCategory(txs: Transaction[], kind: TxKind): CategoryTotal[] {
41   const map = new Map<string, { total: number; count: number }>();
42   for (const t of txs) {
43     if (t.kind !== kind) continue;
44     const cur = map.get(t.categoryKey) ?? { total: 0, count: 0 };
45     cur.total += t.amountSatang;
46     cur.count += 1;
47     map.set(t.categoryKey, cur);
48   }
49   const grand = [...map.values()].reduce((s, v) => s + v.total, 0);
50   return [...map.entries()]
51     .map(([key, v]) => {
52       const c = getCategory(key);
53       return {
54         key,
55         label: c.label,
56         glyph: c.glyph,
57         kind,
58         totalSatang: v.total,
59         count: v.count,
60         share: grand > 0 ? v.total / grand : 0,
61       };
62     })
63     .sort((a, b) => b.totalSatang - a.totalSatang);
64 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:40

บรรทัด	ความหมาย
40	ฟังก์ชันภายในไฟล์ (ไม่มี export) รับรายการและประเภทที่ต้องการ
41	Map จากรหัสหมวด → { ยอดรวม, จำนวน }

บรรทัด	ความหมาย
42–43	วนทุกรายการ ถ้าไม่ใช่ประเภทที่ต้องการ ข้าม (continue = ไปตัวถัดไป)
44	หับยอดเดิมของหมวดนี้ ถ้ายังไม่มี เริ่มที่ 0, 0
45–46	บวกยอด และนับเพิ่ม 1
47	เก็บกลับ
49	ยอดรวมทุกหมวดของประเภทนี้ (ใช้คิดส่วนแบ่ง)
50–51	แปลง Map เป็นรายการ แต่ละตัวมี [รหัส, ข้อมูล]
52	หาข้อมูลหมวด (ชื่อ ไอคอน)
53–61	สร้าง CategoryTotal: ส่วนแบ่ง = ยอดหมวด ÷ ยอดรวม ถ้ายอดรวมเป็น 0 ใช้ 0 (กันหารด้วยศูนย์)
63	เรียงมากไปน้อย (b - a = ใหญ่ก่อน)

ส่วนที่ 4: summarizeRange – สรุปตามช่วง (บรรทัด 66–82)

src/domain/summary.ts

บรรทัด 66–82

```

66 export function summarizeRange(txs: Transaction[], range: RangeKey, now: Date = new Date()):
    RangeSummary {
67   const days = rangeDays(range, now);
68   const inRange = confirmedOnly(txs).filter((t) => isDayInRange(bkkDayKey(t.occurredAt), days));
69   const income = inRange.filter((t) => t.kind === 'income').reduce((s, t) => s + t.amountSatang,
    0);
70   const expense = inRange.filter((t) => t.kind === 'expense').reduce((s, t) => s +
    t.amountSatang, 0);
71   return {
72     ...days,
73     incomeSatang: income,
74     expenseSatang: expense,
75     netSatang: income - expense,
76     count: inRange.length,
77     byCategory: {
78       income: totalsByCategory(inRange, 'income'),
79       expense: totalsByCategory(inRange, 'expense'),
80     },
81   };
82 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:66

บรรทัด	ความหมาย
66	รับรายการ, ช่วง (เช่น '1m') และ "ตอนนี้" (ค่าเริ่มต้นคือเวลาจริง แต่ทดสอบส่งเวลาตายตัวได้ จึงผลเหมือนเดิมทุกครั้ง)
67	หาวันแรก–วันสุดท้าย เช่น { from: '2026-09-10', to: '2026-10-09' }
68	รายการในช่วง: ยืนยันแล้ว และวันที่ของรายการอยู่ในช่วง
69	รวมรายรับ: กรองเฉพาะรายรับ แล้วลบยอดจาก 0

บรรทัด	ความหมาย
70	รวมรายจ่าย แบบเดียวกัน
71–72	คืน object: ...days ทาง from และ to ใส่งไปเลย
73–76	รายรับ รายจ่าย สุทธิ (รับ – จ่าย) และจำนวนรายการ
77–80	ยอดตามหมวด: เรียก totalsByCategory สองครั้ง (รายรับ และรายจ่าย) กับรายการในช่วง

★ ลองเองใน VS Code

เปิด `src/domain/__tests__/domain.test.ts:147` ดู TC-10 ที่เรียก `summarizeRange` ด้วยข้อมูล 4 รายการ แล้วรัน `npm test -- domain` ในเทอร์มินัล ถ้าลองแค่บรรทัด 68 ให้ลืม `confirmedOnly` เทสต์ TC-10 จะแดงทันที เพราะแบบร่าง ฿999 จะถูกนับ (อย่าลืมแก้กลับ)

ส่วนที่ 5: dailyTotals — ยอดรายวันสำหรับกราฟ (บรรทัด 84–108)

src/domain/summary.ts บรรทัด 84–108

```

84 export interface DayTotal {
85   day: string;
86   incomeSatang: number;
87   expenseSatang: number;
88 }
89
90 /** One entry per Bangkok day, oldest first, including days with no activity. */
91 export function dailyTotals(txs: Transaction[], days: number, now: Date = new Date()): DayTotal[]
92 {
93   const today = bkkDayKey(now);
94   const out: DayTotal[] = [];
95   const index = new Map<string, DayTotal>();
96   for (let i = days - 1; i >= 0; i--) {
97     const day = addDays(today, -i);
98     const row = { day, incomeSatang: 0, expenseSatang: 0 };
99     out.push(row);
100    index.set(day, row);
101  }
102  for (const t of confirmedOnly(txs)) {
103    const row = index.get(bkkDayKey(t.occurredAt));
104    if (!row) continue;
105    if (t.kind === 'income') row.incomeSatang += t.amountSatang;
106    else row.expenseSatang += t.amountSatang;
107  }
108  return out;
109 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:84

บรรทัด	ความหมาย
84–88	DayTotal = 1 วัน: วันที่, รายรับ, รายจ่าย
90	คอมเมนต์: วันละ 1 ช่องตามวันไทย เถ้าไปใหม่ รวมวันที่ไม่มีอะไรเลย

บรรทัด	ความหมาย
91	รับรายการ จำนวนวัน (หน้าหลักส่ง 7) และตอนนี้
92	วันนี้ (เวลาไทย)
93–94	รายการผลลัพธ์ และ Map สำหรับหาช่องของวันได้เร็ว
95	วนจาก i = 6 ลงไปถึง 0 (ถ้า days = 7) จะได้วันเก่าก่อน
96	วันที่ของช่อง = วันนี้ - i วัน
97–99	สร้างช่องค่าเป็น 0 ใส่ทั้งในรายการและใน Map (object เดียวกัน อยู่สองที่ แก่ที่หนึ่งอีกที่เห็นด้วย)
101–102	วนรายการที่ยืนยันแล้ว หาช่องของวันนั้น
103	ไม่มีช่อง = รายการอยู่นอก 7 วัน ข้าม
104–105	บวกเข้ารายรับหรือรายจ่ายของช่องนั้น (แก้ผ่าน Map แต่รายการผลลัพธ์เปลี่ยนตามเพราะเป็น object เดียวกัน)
107	คืนรายการ 7 ช่อง

ส่วนที่ 6: groupByDay และ monthExpense (บรรทัด 110–134)

src/domain/summary.ts

บรรทัด 110–134

```

110 /** Group transactions by Bangkok day, newest day first, newest item first within a day. */
111 export function groupByDay(txs: Transaction[]): { day: string; items: Transaction[]; netSatang:
    number }[] {
112   const map = new Map<string, Transaction[]>();
113   for (const t of [...txs].sort((a, b) => b.occurredAt.localeCompare(a.occurredAt))) {
114     const day = bkkDayKey(t.occurredAt);
115     const list = map.get(day) ?? [];
116     list.push(t);
117     map.set(day, list);
118   }
119   return [...map.entries()].map(([day, items]) => ({
120     day,
121     items,
122     netSatang: items
123       .filter((t) => t.status === 'confirmed')
124       .reduce((s, t) => s + (t.kind === 'income' ? t.amountSatang : -t.amountSatang), 0),
125   }));
126 }
127
128 /** This calendar month's confirmed expense (for the budget meter). */
129 export function monthExpense(txs: Transaction[], now: Date = new Date()): number {
130   const month = bkkDayKey(now).slice(0, 7);
131   return confirmedOnly(txs)
132     .filter((t) => t.kind === 'expense' && bkkDayKey(t.occurredAt).startsWith(month))
133     .reduce((s, t) => s + t.amountSatang, 0);
134 }

```

▶ เปิดใน VS Code (లేు) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:110

บรรทัด	ความหมาย
110–126	groupByDay แยกรายการตามวันไทยสำหรับเก็บรายการ (อธิบายละเอียดในเล่ม FR-1): เรียงเวลาใหม่ไปเก่า, ใส่ลงกลุ่มของวัน, ยอดสุทธิของวันนับเฉพาะที่ยืนยัน
128	คอมเมนต์: รายการที่ยืนยันของ เดือนปฏิทินนี้ (สำหรับแถบงบ)
129	รับรายการและตอนนี้
130	เดือนนี้ = 7 ตัวแรกของวันไทย เช่น "2026-10"
131–132	ยืนยันแล้ว + เป็นรายจ่าย + วันไทยขึ้นต้นด้วย "2026-10" (startsWith)
133	ทบยอดจาก 0 → ภาพหน้าจอได้ 509,100 สตางค์ = ฿5,091

บทที่ 18

อ่านโค้ด: ช่วงเวลาและการแสดงวันแบบไทย (dates.ts บรรทัด 53–157)

ครั้งแรกของไฟล์ (บรรทัด 1–51: bkkDayKey, addDays ฯลฯ) อธิบายไว้แล้วในบท A8 บทนี้อ่านครั้งที่ FR-2 ใช้

ส่วนที่ 1: ตารางช่วงเวลา (บรรทัด 53–89)

src/domain/dates.ts

บรรทัด 53–89

```
53 /**
54  * Number of calendar days (including today) covered by each range.
55  * The same definition is used by the dashboard (FR-2) and the slip scanner (FR-4),
56  * so "1 เดือน" always means the same 30 days everywhere in the app.
57  */
58 export const RANGE_DAYS: Record<RangeKey, number> = {
59   today: 1,
60   '7d': 7,
61   '1m': 30,
62   '6m': 180,
63   '1y': 365,
64 };
65
66 export const RANGE_LABEL: Record<RangeKey, string> = {
67   today: 'วันนี้',
68   '7d': '7 วัน',
69   '1m': '1 เดือน',
70   '6m': '6 เดือน',
71   '1y': '1 ปี',
72 };
73
74 export const RANGE_ORDER: RangeKey[] = ['today', '7d', '1m', '6m', '1y'];
75
76 /** First and last Bangkok day of a range ending today. */
77 export function rangeDays(range: RangeKey, now: Date = new Date()): { from: string; to: string } {
78   {
79     const to = bkkDayKey(now);
80     return { from: addDays(to, -(RANGE_DAYS[range] - 1)), to };
81   }
82 }
83
84 /** UTC instant where a range starts (for querying the photo library by creation time). */
85 export function rangeStartMs(range: RangeKey, now: Date = new Date()): number {
86   return bkkDayStart(rangeDays(range, now).from).getTime();
87 }
88
89 export function isDayInRange(dayKey: string, range: { from: string; to: string }): boolean {
90   return dayKey >= range.from && dayKey <= range.to;
91 }
```

▶ เปิดใน VS Code (ใหม่) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:53

บรรทัด	ความหมาย
53–57	คอมเมนต์: จำนวนวัน (รวมวันนี้) ของแต่ละช่วง ใช้นิยามเดียวกันทั้งแดชบอร์ด (FR-2) และตัวสแกนสลิป (FR-4) "1 เดือน" จึงหมายถึง 30 วันเดียวกันทุกที่
58–64	RANGE_DAYS: today 1, 7d 7, 1m 30, 6m 180, 1y 365 ชนิด Record<RangeKey, number> บังคับว่าต้องมีครบทุกช่วง ขาดตัวไหน TypeScript ฟ้อง
66–72	RANGE_LABEL: ชื่อบนปุ่ม วันนี้ / 7 วัน / 1 เดือน / 6 เดือน / 1 ปี
74	RANGE_ORDER: ลำดับปุ่มจากซ้ายไปขวา หน้าหลักใช้ map สร้างปุ่มจากรายการนี้
76–80	rangeDays: วันสุดท้าย = วันนี้ไทย, วันแรก = วันนี้ - (จำนวนวัน - 1)
82–85	rangeStartMs: เวลาเริ่มของช่วงเป็นมิลลิวินาที UTC (FR-4 ใช้คั่นรูปในแคลเลอรีตามเวลาสร้าง) คือ 00:00 เวลาไทยของวันแรก
87–89	isDayInRange: วันอยู่ในช่วงถ้า $\geq$ วันแรก และ $\leq$ วันสุดท้าย เปรียบเทียบข้อความได้ตรง ๆ เพราะรูปแบบ "YYYY-MM-DD" เรียงตัวอักษรได้เหมือนเรียงวัน

i ทำไมเทียบข้อความวันที่ได้

"2026-09-30" กับ "2026-10-01" เทียบทีละตัวจากซ้าย: "2026-" เท่ากัน, "0" กับ "1" → "1" มากกว่า จึงสรุปว่า 1 ต.ค. มาทีหลังถูกต้อง ใช้ได้เพราะเดือนและวันเต็ม 0 ให้ครบ 2 หลักเสมอ ถ้าเขียน "2026-9-30" วิธีนี้จะผิด

ส่วนที่ 2: ชื่อเดือนและวันแบบไทย (บรรทัด 91–110)

src/domain/dates.ts

บรรทัด 91–110

```

91 // -----
92 // Thai display
93 // -----
94
95 export const THAI_MONTHS_SHORT = [
96   'ม.ค.', 'ก.พ.', 'มี.ค.', 'เม.ย.', 'พ.ค.', 'มิ.ย.',
97   'ก.ค.', 'ส.ค.', 'ก.ย.', 'ต.ค.', 'พ.ย.', 'ธ.ค.',
98 ];
99 const THAI_MONTHS_LONG = [
100  'มกราคม', 'กุมภาพันธ์', 'มีนาคม', 'เมษายน', 'พฤษภาคม', 'มิถุนายน',
101  'กรกฎาคม', 'สิงหาคม', 'กันยายน', 'ตุลาคม', 'พฤศจิกายน', 'ธันวาคม',
102 ];
103 const THAI_WEEKDAYS = ['อาทิตย์', 'จันทร์', 'อังคาร', 'พุธ', 'พฤหัสบดี', 'ศุกร์', 'เสาร์'];
104
105 /** "2026-09-27" -> "27 ก.ย. 2569" (Buddhist Era year, as Thai users read it). */
106 export function formatThaiDay(dayKey: string, opts: { year?: boolean } = {}): string {
107   const [y, m, d] = dayKey.split('-').map(Number);
108   const base = `${d} ${THAI_MONTHS_SHORT[m - 1]}`;
109   return opts.year === false ? base : `${base} ${y + 543}`;
110 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:91

บรรทัด	ความหมาย
95–98	ชื่อเดือนย่อ 12 เดือน (ม.ค. ... ธ.ค.) ตำแหน่ง 0 = มกราคม
99–102	ชื่อเดือนเต็ม (ใช้ในปฏิทินและสรุปเดือน)
103	ชื่อวันในสัปดาห์ ตำแหน่ง 0 = อาทิตย์ (ตรงกับ getUTCDay)
105–106	<code>formatThaiDay</code> : "2026-09-27" → "27 ก.ย. 2569" แยกด้วย "-" แล้วแปลงเป็นตัวเลข
108	วัน + ชื่อเดือนย่อ (เดือน - 1 เพราะรายการเริ่มนับที่ 0)
109	ถ้าส่ง <code>year: false</code> ไม่ใส่ปี ไม่ขึ้นต่อท้ายด้วยปี พ.ศ. (ค.ศ. + 543)

ส่วนที่ 3: ข้อความช่วงเวลาไต่ปุม (บรรทัด 112–124)

src/domain/dates.ts

บรรทัด 112–124

```
112 /**
113  * Exact span of a period, so users see which hours count:
114  * today -> "28 ก.ย. 2569 · 00:00–23:59"
115  * 7d -> "22 – 28 ก.ย. 2569 · 7 วันเต็ม (00:00 วันแรก ถึง 23:59 วันนี้)"
116 */
117 export function formatRangeSpan(range: RangeKey, now: Date = new Date()): string {
118   const { from, to } = rangeDays(range, now);
119   if (range === 'today') return `${formatThaiDay(to)} · 00:00–23:59`;
120   const sameYear = from.slice(0, 4) === to.slice(0, 4);
121   const sameMonth = sameYear && from.slice(5, 7) === to.slice(5, 7);
122   const start = sameMonth ? String(Number(from.slice(8, 10))) : formatThaiDay(from, { year:
!sameYear });
123   return `${start} – ${formatThaiDay(to)} · ${RANGE_DAYS[range]} วันเต็ม`;
124 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:112

บรรทัด	ความหมาย
112–116	คอมเมนต์: บอกช่วงให้ชัดว่าชั่วโมงไหนนับ เช่น วันนี้ → "28 ก.ย. 2569 · 00:00–23:59"
117–118	<code>formatRangeSpan</code> หาวันแรก–วันสุดท้าย
119	ช่วงวันนี้: "9 ต.ค. 2569 · 00:00–23:59"
120	วันแรกกับวันสุดท้ายปีเดียวกันไหม
121	เดือนเดียวกันไหม (ต้องปีเดียวกันด้วย)
122	ข้อความวันแรก: เดือนเดียวกัน → แค่เลขวัน ("3") · ต่างเดือน → "10 ก.ย." · ต่างปี → "10 ต.ค. 2568" (ใส่ปีเมื่อจำเป็นเท่านั้น)
123	"3 – 9 ต.ค. 2569 · 7 วันเต็ม"

ช่วง (วันนี้ 9 ต.ค. 2569)	บรรทัดที่ทำงาน	ผล
today	119	9 ต.ค. 2569 · 00:00–23:59

ช่วง (วันที่ 9 ต.ค. 2569)	บรรทัดที่ทำงาน	ผล
7d	122 (เดือนเดียวกัน)	3 – 9 ต.ค. 2569 · 7 วันเต็ม
1m	122 (ต่างเดือน)	10 ก.ย. – 9 ต.ค. 2569 · 30 วันเต็ม
1y	122 (ต่างปี)	10 ต.ค. 2568 – 9 ต.ค. 2569 · 365 วันเต็ม

ส่วนที่ 4: เดือน และป้ายวันแบบสัมพัทธ์ (บรรทัด 126–157)

src/domain/dates.tsบรรทัด 126–157

```
126 /** "2026-09" -> "กันยายน 2569" */
127 export function formatThaiMonth(monthKey: string): string {
128   const [y, m] = monthKey.split('-').map(Number);
129   return `${THAI_MONTHS_LONG[m - 1]} ${y + 543}`;
130 }
131
132 /** "2026-09" -> "2026-08" (and "2026-01" -> "2025-12"). */
133 export function previousMonth(monthKey: string): string {
134   const [y, m] = monthKey.split('-').map(Number);
135   return m === 1 ? `${y - 1}-12` : `${y}-${String(m - 1).padStart(2, '0')}`;
136 }
137
138 /** Number of days in a month ("2026-02" -> 28). */
139 export function daysInMonth(monthKey: string): number {
140   const [y, m] = monthKey.split('-').map(Number);
141   return new Date(Date.UTC(y, m, 0)).getUTCDate();
142 }
143
144 /** "2026-09-27" -> "วันอาทิตย์ที่ 27 กันยายน 2569" */
145 export function formatThaiDayLong(dayKey: string): string {
146   const [y, m, d] = dayKey.split('-').map(Number);
147   const weekday = new Date(Date.UTC(y, m - 1, d)).getUTCDay();
148   return `วัน${THAI_WEEKDAYS[weekday]}ที่ ${d} ${THAI_MONTHS_LONG[m - 1]} ${y + 543}`;
149 }
150
151 /** Friendly label for a day relative to today: "วันนี้", "เมื่อวาน", or "25 ก.ย." */
152 export function relativeDayLabel(dayKey: string, now: Date = new Date()): string {
153   const today = bkkDayKey(now);
154   if (dayKey === today) return 'วันนี้';
155   if (dayKey === addDays(today, -1)) return 'เมื่อวาน';
156   return formatThaiDay(dayKey, { year: dayKey.slice(0, 4) !== today.slice(0, 4) });
157 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:126

บรรทัด	ความหมาย
126–130	formatThaiMonth: "2026-09" → "กันยายน 2569" (หวัปฏิทินและสรุปเดือน)
132–136	previousMonth: เดือนก่อน ถ้าเป็นมกราคมย้อนไปธันวาคมปีก่อน ไม่ขึ้นลดเดือน 1 และเติม 0 ให้ครบ 2 หลัก
138–142	daysInMonth: ใช้เคล็ดลับ "วันที่ 0 ของเดือนถัดไป = วันสุดท้ายของเดือนนี้" (<code>Date.UTC(y, m, 0)</code>) ซึ่ง <code>m</code> ที่นี้คือเดือนถัดไปเพราะ <code>Date</code> นับเดือนจาก 0) ได้ ค.พ. 2028 = 29 วัน (ปอธิศสุรทิน) ตามที่ TC-69 ทดสอบ
144–149	formatThaiDayLong: "วันศุกร์ที่ 9 ตุลาคม 2569" หาวันในสัปดาห์จาก <code>getUTCDay</code> (0 = อาทิตย์)

บรรทัด	ความหมาย
--------	----------

151-157	relativeDayLabel: วันนี้ → "วันนี้", เมื่อวาน → "เมื่อวาน", นอกนั้น "25 ก.ย." (ใส่ปีถ้าต่างปีกับวันนี้)
---------	--

อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 1 — ส่วนประกอบและข้อมูล

ไฟล์ src/app/(tabs)/index.tsx ยาว 629 บรรทัด เป็นหน้าที่หลาย FR มาอยู่รวมกัน บทนี้อ่านครบทุกบรรทัด และบอกว่าแต่ละส่วนเป็นของ FR ไหน

i แผนที่ไฟล์ก่อนอ่าน

บรรทัด 1–52 import · 54–60 ตารางป้ายสถานะ · 62–141 ปุ่มลัด ActionTile · 143–178 ไฟกดต่อเนื่อง · 180–250 ข้อมูลของหน้า · 252–443 ส่วนบน (การ์ดต่าง ๆ และ hero) · 445–527 วันนี้ ปุ่มลัด สรุปเดือน กระปุก · 529–605 ภาพรวม กราฟ จบ (FR-2 โดยตรง) · 607–629 รายการล่าสุด

ส่วนที่ 1: คอมเมนต์และ import (บรรทัด 1–52)

src/app/(tabs)/index.tsx

บรรทัด 1–52

```

1  /**
2   * Home (FR-2 Overview Dashboard + FR-6 at a glance).
3   * Order of information: how much I have -> how long it lasts -> what to do today
4   * -> where the money went.
5   */
6  import * as Haptics from 'expo-haptics';
7  import { LinearGradient } from 'expo-linear-gradient';
8  import { router } from 'expo-router';
9  import { useEffect, useMemo, useRef, useState } from 'react';
10 import { Animated, Pressable, RefreshControl, View } from 'react-native';
11 import { useApp, useMoney } from '../data/AppProvider';
12 import { buddyLine, buddyPoke, spentOnDay, treeLine, type BuddyLine } from '../domain/buddy';
13 import { addDays, formatRangeSpan, formatThaiDay, formatThaiDayLong, formatThaiMonth,
14   RANGE_LABEL, RANGE_ORDER } from '../domain/dates';
15 import { formatBaht } from '../domain/money';
16 import { goalLine, goalProgress } from '../domain/goals';
17 import { recapToOffer } from '../domain/recap';
18 import { skinById } from '../domain/skins';
19 import { HALLOWEEN_LINES, halloweenOn } from '../domain/halloween';
20 import { HalloweenBunting, HalloweenCard, HalloweenInvite, SpookyHeroDecor, useHalloween } from
21   '../ui/halloween';
22 import { endOfMonthDay, safeDailySpend } from '../domain/runway';
23 import { dailyTotals, monthExpense, summarizeRange } from '../domain/summary';
24 import type { RangeKey } from '../domain/types';
25 import { ContourLines, MoneyTree, treeHealth } from '../ui/art';
26 import { CategoryBars, DayBars } from '../ui/charts';
27 import {
28   Badge,
29   Button,
30   Card,
31   Divider,
32   EmptyState,
33   IconButton,
34   Ionicons,

```

```

33 Money,
34 ProgressBar,
35 Row,
36 Screen,
37 SectionTitle,
38 Segmented,
39 T,
40 } from '../ui/components';
41 import { useKla } from '../services/kla';
42 import { useAchievements } from '../services/useAchievements';
43 import { markWhatsNewSeen, useWhatsNewPending } from '../services/whatsNew';
44 import { AutoScanBanner } from '../ui/AutoScanBanner';
45 import { Buddy, BuddySays } from '../ui/Buddy';
46 import { Aurora, PulseRing, Reveal, Shine, Sparkles, useCelebrate, usePressSpring } from
  '../ui/effects';
47 import { Jar } from '../ui/Jar';
48 import { KlaPicture } from '../ui/kla/KlaPicture';
49 import { Medal } from '../ui/Medal';
50 import { useCountUp, useReduceMotion } from '../ui/motion';
51 import { TxRow } from '../ui/TxRow';
52 import { alpha, fonts, radius, readableOn, space, useTheme } from '../ui/theme';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:1

บรรทัด	ความหมาย
1–5	คอมเมนต์: หน้าหลัก = FR-2 Overview Dashboard + FR-6 แบบมองแวบเดียว ลำดับข้อมูล: มีเท่าไร → อยู่ได้นานแค่ไหน → วันนี้ทำอะไร → เงินไปไหน
6	การสั่นของเครื่อง (haptics) ตอนกดปุ่ม
7	พื้นโล่สี (การ์ดเขียว)
8	router เปลี่ยนหน้า
9	hook ของ React: effect, memo, ref, state
10	ชิ้นส่วน React Native: ภาพเคลื่อนไหว, ปุ่มกด, ดึงรีเฟรช, กล้อง
11	ข้อมูลกลาง: useApp (รายการ โปรไฟล์ วันนี้) และ useMoney (ยอด Runway ค่าเฉลี่ย แบบร่าง)
12	ประโยชน์ของน้องกล้า (buddy.ts) และยอดที่ใช้ในวันหนึ่ง
13	วันที่ของ FR-2: บวกวัน, ข้อความช่วง, วันไทยสั้น/ยาว, ชื่อเดือน, ชื่อปุ่มช่วง และลำดับปุ่ม
14	แสดงเงิน
15	กระปุกออม (FR-6)
16	เลือกเดือนที่จะเสนอสรุป (FR-2)
17–19	สกินน้องกล้า และของธีมฮาโลวีน
20	สูตร FR-6: วันสิ้นเดือน และใช้ได้วันละเท่าไร
21	สูตร FR-2 สามตัว: ยอดรายวัน, รายจ่ายเดือนนี้, สรุปตามช่วง
22	ชนิดของช่วง

บรรทัด	ความหมาย
23	ภาพวาด: เส้นลาย ตันไม้เงิน ความสมบูรณ์ของตันไม้
24	กราฟของ FR-2: แถบหมวด และแท่งรายวัน (charts.tsx)
25–40	ชั้นส่วน Design System 15 ตัว: ป้าย ปุ่ม การ์ด เส้นคั่น หน้าว่าง ปุ่มไอคอน ไอคอน Money แถบความคืบหน้า แถว หน้าจอ หัวข้อส่วน ปุ่มสลับ ข้อความ
41–43	บริการ: ตูสกิน, เหรียญความสำเร็จ, "มีอะไรใหม่"
44–51	ชั้นส่วนอื่น: แบบเนอร์สแกนอัตโนมัติ (FR-4), นื่องกล้า, เอฟเฟกต์ (ออโรรา วงจักร ค่อย ๆ โผล่ แสงวิ่ง ประกาย พลุ่ ปุ่มเด็ก), กระปุก, รูปนื่องกล้า, เหรียญ, ตัวนับเลข/ลดการเคลื่อนไหว, แถวรายการ
52	ค่าการออกแบบ: สีโปรง ฟอนต์ มุมโค้ง สีที่อ่านออกบนพื้น ระยะห่าง อิม

ส่วนที่ 2: ป้ายสถานะ และชนิดปุ่มลัด (บรรทัด 54–62)

src/app/(tabs)/index.tsx	บรรทัด 54–62
<pre> 54 const RUNWAY_BADGE = { 55 healthy: { label: 'สบาย ๆ', tone: 'good' }, 56 watch: { label: 'เริ่มต้องระวัง', tone: 'watch' }, 57 critical: { label: 'ใกล้เส้นสำรอง', tone: 'critical' }, 58 below_floor: { label: 'ต่ำกว่าเงินสำรอง', tone: 'critical' }, 59 no_spending: { label: 'ยังไม่มีรายจ่าย 7 วัน', tone: 'neutral' }, 60 } as const; 61 62 type TileTone = 'primary' 'gold' 'plain' 'income'; </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:54	

บรรทัด	ความหมาย
54–60	ตารางแปลงสถานะ Runway (FR-6) → ข้อความและโทนสีของป้ายบนการ์ดเขียว: สบาย ๆ / เริ่มต้องระวัง / ใกล้เส้นสำรอง / ต่ำกว่าเงินสำรอง / ยังไม่มีรายจ่าย 7 วัน as const ทำให้ค่าเป็นค่าคงที่ตายตัว
62	ชนิดโทนสีของปุ่มลัดมี 4 แบบ

ส่วนที่ 3: ปุ่มลัด ActionTile (บรรทัด 64–141)

src/app/(tabs)/index.tsx	บรรทัด 64–141
<pre> 64 /** 65 * One of the four quick actions on home: scan slips, speak, write an expense, 66 * add money in (money that arrives without a slip has to be added by hand). 67 */ 68 function ActionTile({ 69 icon, 70 label, 71 sub, 72 tone, 73 onPress, 74 pulse, 75 }): { </pre>	

```

76 icon: keyof typeof Ionicons.glyphMap;
77 label: string;
78 sub: string;
79 tone: TileTone;
80 onPress: () => void;
81 pulse?: boolean;
82 }) {
83   const theme = useTheme();
84   const press = usePressSpring(0.95);
85   // Every colour from the theme; sub-text at full strength so it reads on the fill (AA) in every
   theme.
86   const look = {
87     primary: { bg: theme.primary, border: theme.primary, ink: theme.onPrimary, soft:
   theme.onPrimary, dot: alpha(theme.onPrimary, 0.16), icon: theme.onPrimary },
88     gold: { bg: theme.accent, border: theme.accent, ink: theme.onAccent, soft: theme.onAccent,
   dot: alpha(theme.onAccent, 0.12), icon: theme.onAccent },
89     plain: { bg: theme.surface, border: theme.line, ink: theme.ink, soft: theme.inkSoft, dot:
   theme.surfaceAlt, icon: theme.ink },
90     income: {
91       bg: alpha(theme.income, theme.dark ? 0.12 : 0.1),
92       border: alpha(theme.income, 0.35),
93       ink: theme.ink,
94       soft: theme.inkSoft,
95       dot: theme.income,
96       icon: readableOn(theme.income, '#FFFFFF', theme.bg),
97     },
98   }[tone];
99   return (
100     <Animated.View style={{ flex: 1, transform: [{ scale: press.scale }] }}>
101       <Pressable
102         onPress={() => {
103           Haptics.selectionAsync().catch(() => {});
104           onPress();
105         }}
106         onPressIn={press.onPressIn}
107         onPressOut={press.onPressOut}
108         accessibilityRole="button"
109         accessibilityLabel={label}
110         style={{
111           flexDirection: 'row',
112           alignItems: 'center',
113           gap: space.sm,
114           padding: space.md,
115           minHeight: 70,
116           borderRadius: radius.lg,
117           backgroundColor: look.bg,
118           borderWidth: 1,
119           borderColor: look.border,
120           overflow: 'hidden',
121         }}
122       >
123         {tone === 'primary' ? <Shine times={1} delay={900} color="rgba(255,255,255,0.25)" /> :
   null}
124         <View style={{ width: 40, height: 40, alignItems: 'center', justifyContent: 'center' }}>
125           {pulse ? <PulseRing size={40} times={2} color={tone === 'gold' ? '#FFF4D6' :
   theme.accent} /> : null}
126           <View style={{ width: 40, height: 40, borderRadius: 20, backgroundColor: look.dot,
   alignItems: 'center', justifyContent: 'center' }}>
127             <Ionicons name={icon} size={21} color={look.icon} />
128           </View>
129         </View>

```

```

130     <View style={{ flex: 1 }}>
131       <T v="body" color={look.ink} style={{ fontFamily: fonts.sansSemi }} numberOfLines={1}>
132         {label}
133       </T>
134       <T v="micro" color={look.soft} numberOfLines={1}>
135         {sub}
136       </T>
137     </View>
138   </Pressable>
139 </Animated.View>
140 );
141 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/index.tsx:64`

บรรทัด	ความหมาย
64–67	คอมเมนต์: 1 ใน 4 ปุ่มสลับบนหน้าหลัก (เงินที่เข้ามาไม่มีสลิปต้องเพิ่มเอง)
68–82	props: ไอคอน (ชื่อต้องอยู่ในชุด Ionicons), ป้าย, คำอธิบายเล็ก, โทน, ทำอะไรเมื่อกด, มิวiczพจรโหม
83–84	สัริม และสปริงตอนกด (หดเหลือ 95%)
85–98	ตารางสีของแต่ละโทน (พื้น ขอบ ตัวหนังสือ ตัวรอง จุดหลังไอคอน ไอคอน) ทุกสีมาจากธีม คำอธิบายใช้สีเต็มเพื่อให้่านออกบนพื้น (ผ่านเกณฑ์ AA) แล้วหีบด้วย [tone]
100	กล่องเคลื่อนไหวที่ข่อยขยายตามสปริง
101–105	กดแล้ว สั่นเบา ๆ (ถ้าเครื่องไม่รองรับก็ไม่เป็นไร) แล้วทำงานตาม onPress
106–109	กดลง/ปล่อยเรียกสปริง และบอกโปรแกรมอ่านจั่วว่าเป็นปุ่มชื่ออะไร
110–121	หน้าตา: แนวนอน จัดกลาง ระยะห่าง สูงอย่างน้อย 70 มุมโค้งใหญ่ พื้นและขอบตามโทน
123	โทนหลัก (สแกนสลิป) มีแสงวิ่งผ่าน 1 ครั้ง
124–129	วงกลมไอคอน 40×40 ถ้ามีชีพจร (จดด้วยเสียง) มิวiczกระพือม 2 ครั้ง
130–137	ป้าย (ตัวหนา) และคำอธิบายเล็ก บรรทัดเดียว

ส่วนที่ 4: ไฟจดต่อเนื่อง StreakPill (บรรทัด 143–178)

`src/app/(tabs)/index.tsx`

บรรทัด 143–178

```

143 /** Badges already celebrated in this session (the home screen can mount again). */
144 const celebrated = new Set<string>();
145
146 /** The recording streak: a flame and the number of days; opens the achievements. */
147 function StreakPill({ days, today }: { days: number; today: boolean }) {
148   const theme = useTheme();
149   const press = usePressSpring(0.92);
150   return (
151     <Animated.View style={{ transform: [{ scale: press.scale }] }}>
152       <Pressable
153         onPress={() => router.push('/achievements')}
154         onPressIn={press.onPressIn}
155         onPressOut={press.onPressOut}
156         hitSlop={6}

```

```

157     accessibilityRole="button"
158     accessibilityLabel={`จัดต่อเนื่อง ${days} วัน ดูความสำเร็จ`}
159     style={{
160       flexDirection: 'row',
161       alignItems: 'center',
162       gap: 4,
163       paddingHorizontal: 12,
164       height: 36,
165       borderRadius: 18,
166       backgroundColor: theme.dark ? alpha(theme.accent, 0.14) : theme.accentSoft,
167       borderWidth: 1,
168       borderColor: today ? theme.accent : 'transparent',
169     }}
170   >
171     <Icons name="flame" size={18} color={today ? '#E07B2E' : theme.inkFaint} />
172     <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi }}>
173       {days}
174     </T>
175   </Pressable>
176 </Animated.View>
177 );
178 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/index.tsx:143`

บรรทัด	ความหมาย
143–144	Set ของเหรียญที่ลดลงไปแล้วในรอบนี้ (อยู่นอก component จึงไม่หายเมื่อนำหลักถูกสร้างใหม่) กันพลุซ้ำ
146–147	เปิดยาไฟ: จำนวนวันที่จัดต่อเนื่อง และวันนี้จัดแล้วหรือยัง กดแล้วไปหน้าความสำเร็จ
148–158	สัริม สปริง ปุ่มไปหน้า achievements พื้นที่กดขยาย 6 ป้ายอ่านจ่อ "จัดต่อเนื่อง 23 วัน ดูความสำเร็จ"
159–170	แคปซูลสูง 36 พื้นทองอ่อน ขอบทองเฉพาะวันที่จัดแล้ว
171	ไอคอนไฟ สีส้มถ้าวันนี้จัดแล้ว ไม่ขึ้นสีจาง
172–174	ตัวเลขจำนวนวัน

ส่วนที่ 5: ข้อมูลของหน้าหลัก (บรรทัด 180–212) – ส่วนสำคัญของ FR-2

`src/app/(tabs)/index.tsx`

บรรทัด 180–212

```

180 export default function Home() {
181   const theme = useTheme();
182   const { profile, txs, goals, repo, refresh, refreshing, loadError, today, newDay } = useApp();
183   const { balance, runway, average, drafts } = useMoney();
184   const [range, setRange] = useState<RangeKey>('1m');
185
186   // `today` changes at 00:00 Bangkok time, so every "วันนี้" number starts over.
187   // eslint-disable-next-line react-hooks/exhaustive-deps
188   const summary = useMemo(() => summarizeRange(txs, range), [txs, range, today]);
189   // eslint-disable-next-line react-hooks/exhaustive-deps
190   const week = useMemo(() => dailyTotals(txs, 7), [txs, today]);
191   const recent = useMemo(
192     () => [...txs].filter((t) => t.status === 'confirmed').sort((a, b) =>
193       b.occurredAt.localeCompare(a.occurredAt)).slice(0, 5),
194     [txs],

```

```

194 );
195 const safeToday = safeDailySpend({ balanceSatang: balance, floorSatang: runway.floorSatang,
    targetDay: endOfMonthDay() });
196 const spentMonth = monthExpense(txs);
197 const readyDrafts = drafts.filter((d) => d.reviewFlags.length === 0).length;
198 const badge = RUNWAY_BADGE[runway.status];
199 const reduce = useReduceMotion();
200 const recapMonth = useMemo(() => recapToOffer(txs, today), [txs, today]);
201 const ach = useAchievements();
202 const kla = useKla();
203 const halloween = useHalloween();
204 const whatsNew = useWhatsNewPending();
205 const celebrate = useCelebrate();
206 const freshIds = [...ach.fresh.map((b) => b.id), ...kla.fresh.map((id) =>
    `skin:${id}`)].join(',');
207 useEffect(() => {
208     const ids = freshIds ? freshIds.split(',') : [];
209     if (!ids.some((id) => !celebrated.has(id))) return;
210     ids.forEach((id) => celebrated.add(id));
211     celebrate();
212 }, [freshIds, celebrate]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/index.tsx:180`

บรรทัด	ความหมาย
180	component หลักของแท็บหน้าหลัก
181	สัณนิษฐาน
182	จาก useApp: โปรไฟล์, รายการทั้งหมด, กระเป๋า, repo, รีเฟรช, กำลังรีเฟรช, ข้อผิดพลาดโหลด, วันนี้ , วันใหม่ใหม่
183	จาก useMoney: ยอดคงเหลือ, Runway, ค่าเฉลี่ย, แบบร่าง (คำนวณครั้งเดียวที่ AppProvider)
184	state ช่วงที่เลือก เริ่มที่ '1m'
186–188	summary = summarizeRange(txs, range) คำนวณใหม่เมื่อรายการ ช่วง หรือ วันนี้ เปลี่ยน คอมเมนต์อธิบายว่า today เปลี่ยนตอนเที่ยงคืนไทย ทุกตัวเลข "วันนี้" จะเริ่มใหม่ (บรรทัด eslint-disable บอกเครื่องมือตรวจโค้ดว่าตั้งใจใส่ today ไว้ แม้ไม่ได้ใช้ในสูตรตรง ๆ)
189–190	week = dailyTotals 7 วัน (กราฟ)
191–194	recent = คัดลอกรายการ กรองที่ยืนยัน เรียงเวลาใหม่ไปเก่า เอา 5 ตัวแรก (slice(0, 5))
195	ใช้ได้วันละเท่าไรถึงสิ้นเดือน (FR-6)
196	spentMonth = รายจ่ายเดือนปฏิทินนี้ (แถบงบ)
197	แบบร่างที่พร้อมยืนยัน (ไม่มีธง)
198	ป้ายของสถานะ Runway ปัจจุบัน
199	ผู้ใช้เปิดลดการเคลื่อนไหวใหม่
200	recapMonth = เดือนที่จะเสนอสรุป หรือ null
201–205	เหรียญ, สกิน, ธีมฮาโลวีน, มีอะไรใหม่ค้างอยู่ใหม่, ฟังก์ชันจุดพลุ

บรรทัด	ความหมาย
206	รหัสของเหรียญและสกินที่เพิ่งได้ รวมเป็นข้อความเดียว (ใช้เป็นตัวบอกว่าเปลี่ยนไหม)
207–212	effect: ถ้ามีรหัสใหม่ที่ยังไม่เคยทดลอง จดว่าทดลองแล้ว แล้ว จุดพลุ ครั้งเดียว

★ useMemo กับ dependency

รายการ [txs, range, today] ภาย useMemo คือ "ถ้าค่าเหล่านี้ไม่เปลี่ยน ใช้ผลเดิม" ทำให้การเลื่อนจอหรือแตะน้องกล่าไม่ต้องคำนวณสรุปใหม่ แต่เมื่อแตะปุ่มช่วง (range เปลี่ยน) หรือบันทึกรายการ (txs เปลี่ยน) หรือข้ามเที่ยงคืน (today เปลี่ยน) จะคำนวณใหม่ทันที

ส่วนที่ 6: เอฟเฟกต์เลื่อน และน้องกล่า (บรรทัด 213–250)

src/app/(tabs)/index.tsx

บรรทัด 213–250

```

213 // Parallax: while scrolling, the hero card eases back and the tree drifts up a little.
214 const [scrollY] = useState(() => new Animated.Value(0));
215 const heroScale = reduce ? 1 : scrollY.interpolate({ inputRange: [-120, 0, 260], outputRange:
  [1.05, 1, 0.95], extrapolate: 'clamp' });
216 const treeLift = reduce ? 0 : scrollY.interpolate({ inputRange: [-120, 0, 260], outputRange:
  [10, 0, -22], extrapolate: 'clamp' });
217 const daysShown = useCountUp(runway.days ?? 0, !reduce && runway.days !== null &&
!runway.capped, 900);
218 // Tapping the companion or the tree: it answers for a few seconds, then goes back to its line.
219 const [said, setSaid] = useState<BuddyLine | null>(null);
220 const [hop, setHop] = useState(0);
221 const [treeTaps, setTreeTaps] = useState(0);
222 const pokes = useRef(0);
223 useEffect(() => {
224   if (!said) return;
225   const t = setTimeout(() => setSaid(null), 6000);
226   return () => clearTimeout(t);
227 }, [said]);
228 function pokeBuddy() {
229   pokes.current += 1;
230   setHop((h) => h + 1);
231   // Halloween theme: every other tap is a spooky-cute line.
232   const n = pokes.current;
233   setSaid(halloween && n % 2 === 1 ? { mood: 'cheer', text: HALLOWEEN_LINES[((n - 1) / 2) %
  HALLOWEEN_LINES.length] } : buddyPoke(n, runway.status));
234 }
235 function tapTree() {
236   Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light).catch(() => {});
237   setTreeTaps((n) => n + 1);
238   setHop((h) => h + 1);
239   setSaid(treeLine({ status: runway.status, days: runway.days, capped: runway.capped }));
240 }
241 const buddy = buddyLine({
242   status: runway.status,
243   days: runway.days,
244   capped: runway.capped,
245   safeTodaySatang: safeToday,
246   spentTodaySatang: spentOnDay(tx, today),
247   draftsToReview: drafts.length - readyDrafts,

```

```

248     hasAnyTransaction: txs.length > 0,
249     newDay: newDay ? { yesterdaySpentSatang: spentOnDay(txs, addDays(today, -1)) } : null,
250   });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:213

บรรทัด	ความหมาย
213–214	Parallax: ค่าระยะเลื่อนของหน้า (Animated.Value)
215	การ์ดเขียวหดเหลือ 95% เมื่อเลื่อนลง 260 จุด (ขยาย 105% เมื่อถึงลง) ถาลดการเคลื่อนไหว คงที่ 1
216	ต้นไม้ลอยขึ้นเล็กน้อยเมื่อเลื่อน
217	ตัวเลขวันที่นับขึ้น 0.9 วินาที (FR-6)
218–222	state ของการแตะ: ประโยคที่น้องกล้าตอบ, จำนวนครั้งที่กระโดด, จำนวนครั้งที่แตะต้นไม้, ตัวนับการแตะน้องกล้า (ref ไม่ทำให้วาดใหม่)
223–227	ถ้ามีประโยคตอบ ตั้งเวลา 6 วินาทีให้กลับเป็นประโยคปกติ และยกเลิกเวลาเมื่อเปลี่ยน
228–234	แตะน้องกล้า: นับครั้ง กระโดด ช่วงฮาโลวันครั้งที่นี้เป็นประโยคผี นอกนั้นเคล็ดลับตามสถานะเงิน
235–240	แตะต้นไม้: สั่นเบา ต้นไม้ส่าย น้องกล้ากระโดด และอธิบายความหมายของต้นไม้
241–250	ประโยคปกติของน้องกล้า สร้างจากสถานะ Runway จำนวนวัน ใช้ได้วันนี้ ใช้ไปวันนี้ แบบร่างที่ต้องตรวจ มีรายการหรือยัง และถ้าเป็นวันใหม่ ส่งยอดเมื่อวานไปด้วย

อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 2 — ส่วนบนและการ์ดเขียว

บรรทัด 252–469 วาดทุกอย่างก่อนถึงส่วนภาพรวม

ส่วนที่ 7: หัวหน้า นื่องกล้า และการ์ด "มีอะไรใหม่" (บรรทัด 252–289)

src/app/(tabs)/index.tsx

บรรทัด 252–289

```
252   return (  
253     <Screen scrollY={scrollY} refreshControl={<RefreshControl refreshing={refreshing} onRefresh=  
    {refresh} tintColor={theme.primary} />}>  
254       {halloween ? <HalloweenBunting /> : null}  
255       <Row justify="space-between" style={{ paddingTop: halloween ? 0 : space.sm, marginTop:  
    halloween ? -space.md : 0 }}>  
256         <View style={{ flex: 1, gap: 2 }}>  
257           <T v="label">{formatThaiDayLong(today)}</T>  
258           <T v="h1" numberOfLines={1}>  
259             สวัสดี {profile?.displayName || ''}  
260           </T>  
261         </View>  
262         <Row gap={space.xs}>  
263           <StreakPill days={ach.streak.days} today={ach.streak.today} />  
264           <IconButton icon="settings-outline" label="ตั้งค่า" onPress={() =>  
    router.push('/settings')} />  
265         </Row>  
266       </Row>  
267  
268       <Reveal>  
269         <BuddySays mood={{said ?? buddy}.mood} onPress={pokeBuddy} hop={hop}>  
270           {said ?? buddy}.text  
271         </BuddySays>  
272       </Reveal>  
273  
274       {whatsNew && txs.length > 0 ? (  
275         <Reveal zoom>  
276           <Card onPress={() => router.push('/whatsnew')} style={{ paddingVertical: space.md,  
    borderColor: theme.accent, borderWidth: 1.5 }}>  
277             <Row gap={space.md}>  
278               <View style={{ width: 44, height: 44, borderRadius: 22, backgroundColor:  
    theme.accentSoft, alignItems: 'center', justifyContent: 'center' }}>  
279                 <Ionicons name="gift" size={22} color={theme.accentInk} />  
280               </View>  
281               <View style={{ flex: 1 }}>  
282                 <T v="h3">มีอะไรใหม่ในอัปเดตนี้</T>  
283                 <T v="small">รับฮาโลวีน 🎃, โค้ชน้องกล้าพูดได้, คู่สกิน, การจัดการเงิน, ธีมสี และอีกเพียบ</T>  
284               </View>  
285               <IconButton icon="close" label="ไม่ต้องแสดงอีก" onPress={() => markWhatsNewSeen()} />  
286             </Row>  
287           </Card>  
288         </Reveal>  
289       ) : null}
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:252

บรรทัด	ความหมาย
253	Screen หน้าเลื่อนได้ของ Design System ส่งค่าระยะเลื่อน (parallax) และดึงลงเพื่อโหลดใหม่
254	ธงฮาโลวีน (เฉพาะธึมนั้น)
255–261	ซ้าย: วันที่ยาว "วันศุกร์ที่ 9 ตุลาคม 2569" และ "สวัสดี ชื่อผู้ใช้"
262–265	ขวา: โฟลต์ต่อเนื่อง และปุ่มตั้งค่า
268–272	กล่องคำพูดน้องกล้า ใช้ประโยคตอบถ้ามี (said ?? buddy) ไม่ขึ้นประโยคปกติ แต่แล้วเรียก pokeBuddy
274–289	ถ้ามีอัปเดตใหม่และมีรายการแล้ว: การ์ดของขวัญ "มีอะไรใหม่ในอัปเดตนี้" กดเปิดหน้า whatsnew หรือกด × ไม่ต้องแสดงอีก

ส่วนที่ 8: การ์ดเหรียญใหม่ และสกินใหม่ (บรรทัด 291–347)

src/app/(tabs)/index.tsx

บรรทัด 291–347

```

291     {ach.fresh.length > 0 ? (
292       <Reveal zoom>
293         <Card style={{ borderColor: theme.accent, borderWidth: 1.5, paddingVertical: space.md
  }}}>
294           <Row gap={space.md}>
295             <Medal badge={ach.fresh[0]} size={52} shine />
296             <View style={{ flex: 1, gap: 2 }}>
297               <T v="h3">{ach.fresh.length > 1 ? `ได้เหรียญใหม่ ${ach.fresh.length} เหรียญ!` : 'ได้
เหรียญใหม่!'}</T>
298               <T v="small">{ach.fresh.map((b) => b.title).join(' · ')}</T>
299               <Row gap={space.sm} style={{ marginTop: space.xs }}>
300                 <Button label="ดูเหรียญ" small kind="gold" icon="ribbon" onPress={() =>
router.push('/achievements')} />
301                 <Button label="ไว้หลัง" small kind="ghost" onPress={() =>
ach.markSeen(ach.fresh.map((b) => b.id))} />
302               </Row>
303             </View>
304           </Row>
305         </Card>
306       </Reveal>
307     ) : null}
308
309     {kla.fresh.length > 0 ? (
310       <Reveal zoom>
311         <Card style={{ borderColor: theme.accent, borderWidth: 1.5, paddingVertical: space.md
  }}}>
312           <Row gap={space.md}>
313             <View style={{ width: 60, height: 60 }}>
314               <View style={{ position: 'absolute', top: -12 }}>
315                 <KlaPicture skin={kla.fresh[0]} mood="cheer" width={60} onDark={theme.dark}
extras={false} />
316               </View>
317             </View>
318             <View style={{ flex: 1, gap: 2 }}>
319               <T v="h3">{kla.fresh.length > 1 ? `น้องกล้าได้ชุดใหม่ ${kla.fresh.length} ชุด!` : 'น้องกล้า
ได้ชุดใหม่!'}</T>
320               <T v="small">
321                 {kla.fresh
322                   .map((id) => {

```

```

323         const s = skinById(id);
324         return s.kind === 'limited' ? `${s.name} (สกินลิมิต)` : s.name;
325     })
326     .join(' · ');
327 </T>
328 <Row gap={space.sm} style={{ marginTop: space.xs }}>
329     <Button
330         label="ใส่เลย"
331         small
332         kind="gold"
333         icon="shirt"
334         onPress={() => {
335             kla.equip(kla.fresh[0]);
336             kla.markSeen(kla.fresh);
337             setHop((h) => h + 1);
338             celebrate();
339         }}
340     />
341     <Button label="ดูตู้สกิน" small kind="ghost" onPress={() => router.push({ pathname:
342 '/skins', params: { skin: kla.fresh[0] } })} />
343 </Row>
344 </View>
345 </Row>
346 </Card>
347 ) : null}

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/index.tsx:291`

บรรทัด ความหมาย

291–307 ถ้าได้เหรียญใหม่: การ์ดขอบทอง รูปเหรียญแรกมีประกาย หัวข้อ "ได้เหรียญใหม่!" (หรือ "ได้เหรียญใหม่ 5 เหรียญ!") ชื่อเหรียญคันด้วย · ปุ่ม "ดูเหรียญ" และ "ไว้ทีหลัง" (จดยกเว้นแล้ว)

309–347 ถ้าน้องกล้าได้ชุดใหม่: การ์ดรูปน้องกล้าใส่ชุดใหม่ ชื่อชุด (สกินลิมิตมีวงเล็บบอก) ปุ่ม "ใส่เลย" (ใส่ชุด จดยกเว้นแล้ว) กระโดด จุดพลุ และ "ดูตู้สกิน"

ส่วนที่ 9: การ์ดสถานะต่าง ๆ (บรรทัด 349–368)

`src/app/(tabs)/index.tsx`

บรรทัด 349–368

```

349     /* ฮาโลวีน: catch ghosts in the Halloween theme; other themes get an invitation. */
350     {halloweenOn(today) ? <Reveal zoom>{halloween ? <HalloweenCard /> : <HalloweenInvite />}
351 </Reveal> : null}
352
353     {repo?.mode === 'demo' ? (
354         <Card tone="alt" style={{ paddingVertical: space.md }}>
355             <Row gap={space.sm}>
356                 <Ionicons name="flask-outline" size={18} color={theme.inkSoft} />
357                 <T v="small" style={{ flex: 1 }}>โหมดทดลอง ข้อมูลตัวอย่างอยู่ในเครื่องนี้ ลองแก้หรือลบได้ตามสบายเลย</T>
358             </Row>
359         </Card>
360     ) : null}
361
362     {loadError ? (
363         <Card tone="alt">
364             <T v="body" color={theme.critical}>โหลดข้อมูลล่าสุดไม่สำเร็จ ({loadError})</T>
365             <Button label="ลองอีกครั้ง" kind="soft" small onPress={refresh} icon="refresh" />
366         </Card>
367     ) : null}

```

```

365     </Card>
366   ) : null}
367
368   <AutoScanBanner />

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:349

บรรทัด	ความหมาย
349–350	ช่วงฮาโลวีน: ธีมฮาโลวีนเห็นการ์ดจับผี ธีมอื่นเห็นการ์ดชวน
352–359	โหมดทดลอง: การ์ดทดลองทดลอง "โหมดทดลอง ข้อมูลตัวอย่างอยู่ในเครื่องนี้ ลองแก้หรือลบได้ตามสบายเลย"
361–366	โหลดไม่สำเร็จ: ข้อความแดงพร้อมสาเหตุ และปุ่ม "ลองอีกครั้ง" (ตัวเลขที่เห็นยังเป็นของเดิม ไม่หายไป)
368	แบนเนอร์สแกนสลิปอัตโนมัติ (FR-4)

ส่วนที่ 10: การ์ดเขียว — ยอดคงเหลือ (บรรทัด 370–443)

src/app/(tabs)/index.tsx

บรรทัด 370–443

```

370   {/* Hero: balance + runway (the one place gold is used for the number that matters) */}
371   <Reveal index={1} zoom>
372     <Animated.View style={{ transform: [{ scale: heroScale }] }}>
373       <View
374         style={{
375           borderRadius: radius.xl,
376           overflow: 'hidden',
377           shadowColor: theme.heroDeep,
378           shadowOpacity: theme.dark ? 0 : 0.28,
379           shadowRadius: 18,
380           shadowOffset: { width: 0, height: 10 },
381           elevation: 8,
382         }}
383         accessible
384         accessibilityLabel={`ยอดคงเหลือ ${formatBaht(balance)} ${runway.days !== null ? `เงินพอใช้อีก
385           ${runway.days} วัน` : ''}`}
386       >
387         <LinearGradient
388           colors={[theme.heroTop, theme.hero, theme.heroDeep]}
389           locations={[0, 0.55, 1]}
390           start={{ x: 0, y: 0 }}
391           end={{ x: 1, y: 1 }}
392           style={{ padding: space.xl, gap: space.md }}
393         >
394           <Aurora />
395           <ContourLines width={420} height={260} color={theme.heroAccent} />
396           <Sparkles trigger={balance} count={9} area={{ top: 4, bottom: 55 }} />
397           <Shine trigger={balance} />
398           {halloween ? <SpookyHeroDecor /> : null}
399           <Row align="flex-start" justify="space-between">
400             <View style={{ flex: 1, gap: 2 }}>
401               <T v="label" color={theme.heroInkSoft}>ยอดคงเหลือ</T>
402               <Money satang={balance} size="display" color={theme.heroInk} countUp />
403             <View style={{ marginTop: space.sm }}>
404               <Badge label={badge.label} tone={badge.tone} onDark />
405             </View>
406           </Row>
407         </View>
408       </View>
409     </Animated.View>
410   </Reveal>

```

```

406         <Animated.View style={{ transform: [{ translateY: treeLift }] }}>
407             <Pressable onPress={tapTree} accessibilityRole="button" accessibilityLabel="ต้นไม้
เงิน และเพื่อดูความหมาย" hitSlop={6}>
408                 <MoneyTree
409                     health={treeHealth(runway.status, runway.days)}
410                     size={104}
411                     trunk="#E8E1CC"
412                     leaf={theme.heroAccent}
413                     bare={theme.heroBare}
414                     glow={theme.heroAccent}
415                     grow
416                     sway
417                     wiggle={treeTaps}
418                 />
419                 {treeTaps > 0 ? <Sparkles trigger={treeTaps} count={7} cycles={1} seed=
{treeTaps} area={{ top: 0, bottom: 70 }} /> : null}
420             </Pressable>
421         </Animated.View>
422     </Row>
423     <View style={{ height: 1, backgroundColor: alpha(theme.heroInk, 0.15) }} />
424     <Row justify="space-between" align="flex-end">
425         <View style={{ flex: 1 }}>
426             <T v="small" color={theme.heroInkSoft}>เงินพอใช้อีก</T>
427             {runway.days !== null && runway.status !== 'below_floor' ? (
428                 <T v="h1" color={theme.heroAccent}>
429                     {runway.capped ? '365+ ' : `${daysShown} `}
430                     <T v="body" color={theme.heroInk}>วัน · ถึง
{formatThaiDay(runway.depletionDay!, { year: false })}</T>
431                 </T>
432             ) : runway.status === 'below_floor' ? (
433                 <T v="h3" color={theme.heroInk}>และเส้นเงินสำรองแล้ว</T>
434             ) : (
435                 <T v="h3" color={theme.heroInk}>ยังคำนวณไม่ได้</T>
436             )}
437         </View>
438         <Button label="ดูรายละเอียด" small kind="onDark" icon="chevron-forward" onPress={() =>
router.navigate('/runway')} />
439     </Row>
440 </LinearGradient>
441 </View>
442 </Animated.View>
443 </Reveal>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:370

บรรทัด	ความหมาย
370	คอมเมนต์: Hero = ยอดคงเหลือ + Runway ที่เดียวที่ใช้สีของกับตัวเลขที่สำคัญ
371–372	ไฟล์เป็นลำดับ 1 แบบขยาย และย่อตาม parallax
373–382	กรอบมุมโค้งใหญ่ เงาเขียวเข้ม (โหมดยอดไม่มีเงา) elevation สำหรับ Android
383–384	ทั้งการ์ดอ่านเป็นประโยคเดียว สำหรับโปรแกรมอ่านจอ: "ยอดคงเหลือ ฿11,113.00 เงินพอใช้อีก 15 วัน" (เกสต์ E2E อ่านยอดจากป้ายนี้)
386–392	พื้นไล่สี 3 ระดับจากริม (บน → กลาง → ลึก) แนวทแยง
393–397	ของตกแต่ง: ออโรรา, เส้นคอนทัวร์, ประกาย 9 ดวงและแสงวับ เมื่อยอดเปลี่ยน (trigger = balance), ของฮาโลวีน

บรรทัด	ความหมาย
398–400	แถบบน ซ้าย: ป้าย "ยอดคงเหลือ"
401	Money ขนาด display (ใหญ่สุด) สืบค้นพื้นเขียว นับขึ้น (countUp) ← ตัวเลขหลักของ FR-2
402–404	ป้ายสถานะ (เช่น "สบาย ๆ") แบบบนพื้นมืด
406–421	ขวา: ต้นไม้เงินกดได้ ขนาด 104 ความสมบูรณ์ตาม Runway ใบสีทองของธัม งอกและโยก สายเมื่อแตะ และมีประกายเมื่อแตะ
423	เส้นคั่นบาง ๆ
424–437	แถวล่าง: "เงินพอใช้อีก" แล้ว 3 กรณี: มีจำนวนวัน → "15 วัน · ถึง 24 ต.ค." / ต่ำกว่าเส้น → "แตะเส้นเงินสำรองแล้ว" / คำนวณไม่ได้ → "ยังคำนวณไม่ได้" (FR-6)
438	ปุ่ม "ดูรายละเอียด" ไปเก็บเงินพอถึง (navigate = สลับแท็บ ไม่ซ่อนหน้า)

i Money กับ countUp

Money เป็นชิ้นส่วนแสดงเงินของ Design System แยกมากับสตางค์ (สตางค์ตัวเล็กกว่า) ใส่จุดภาคหลักพัน และตัวเลขกว้างเท่ากัน เมื่อ countUp เปิด ตัวเลขจะวิ่งจากค่าเดิมไปค่าใหม่ ถ้าผู้ใช้เปิด "ลดการเคลื่อนไหว" ตัวเลขจะขึ้นทันที (มีเทสต์ E2E ตรวจสอบข้อนี้)

ส่วนที่ 11: วันนี้ใช้ได้ และแบบร่าง (บรรทัด 445–469)

src/app/(tabs)/index.tsx

บรรทัด 445–469

```

445     { /* Today */ }
446     <Reveal index={2}>
447     <Card tone="accent" style={{ paddingVertical: space.md }}>
448       <Row gap={space.md}>
449         <Icons name="sunny-outline" size={22} color={theme.accentInk} />
450         <View style={{ flex: 1 }}>
451           <T v="small" color={theme.ink}>ถ้าอยากให้เงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ</T>
452           <T v="h3">{formatBaht(safeToday, { decimals: false })} / วัน</T>
453         </View>
454       </Row>
455     </Card>
456   </Reveal>
457
458   {drafts.length > 0 ? (
459     <Card onPress={() => router.push('/drafts')} style={{ borderColor: theme.accent,
borderWidth: 1.5 }}>
460       <Row gap={space.md}>
461         <Icons name="receipt-outline" size={22} color={theme.accentInk} />
462         <View style={{ flex: 1 }}>
463           <T v="h3">สลิปรอยืนยัน {drafts.length} รายการ</T>
464           <T v="small">พร้อมยืนยัน {readyDrafts} · ต้องตรวจ {drafts.length - readyDrafts} · ยังไม่รวม
ในยอดเงิน</T>
465         </View>
466         <Icons name="chevron-forward" size={20} color={theme.inkFaint} />
467       </Row>
468     </Card>

```

469) : null}

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:445

บรรทัด ความหมาย

445–456 การ์ดทองอ่อน ไอคอนดวงอาทิตย์ "ถ้าอยากให้เงินพอถึงสิ้นเดือน วันนี้ใช้ได้ประมาณ" และ "\$461 / วัน" (FR-6)

458–469 ถ้ามีแบบร่าง: การ์ด "สลิปรียืนยัน N รายการ" บอก "พร้อมยืนยัน X · ต้องตรวจ Y · ยังไม่รวมในยอดเงิน" กดไปหน้า drafts (FR-4) — ข้อความ "ยังไม่รวมในยอดเงิน" ย้ำกฎของ FR-2

ส่วนที่ 12: ปุ่มลัด สรุปเดือน และกระปุก (บรรทัด 471–527)

src/app/(tabs)/index.tsx

บรรทัด 471–527

```
471     <Reveal index={3}>
472       <View style={{ gap: space.sm }}>
473         <Row gap={space.sm} align="stretch">
474           <ActionTile icon="scan-outline" label="สแกนสลิป" sub="อ่านจากรูปในเครื่อง" tone="primary"
onPress={() => router.push('/scan')} />
475           <ActionTile icon="mic" label="จดด้วยเสียง" sub="พูดว่า "ข้าว 50 บาท"" tone="gold" pulse
onPress={() => router.push('/voice')} />
476         </Row>
477         <Row gap={space.sm} align="stretch">
478           <ActionTile icon="remove" label="จดยานการ" sub="รายจ่ายเงินสด ฯลฯ" tone="plain" onPress=
{() => router.push('/transaction')} />
479           <ActionTile icon="add" label="เพิ่มเงินเข้า" sub="เงินโอน เงินสด ค่าขนม" tone="income" onPress=
{() => router.push({ pathname: '/transaction', params: { kind: 'income' } })} />
480         </Row>
481       </View>
482     </Reveal>
483
484     {recapMonth ? (
485       <Reveal index={4}>
486         <Pressable
487           onPress={() => router.push({ pathname: '/recap', params: { month: recapMonth } })}
488           accessibilityRole="button"
489           accessibilityLabel={`ดูสรุปเดือน${formatThaiMonth(recapMonth)}`}
490           style={{ borderRadius: radius.lg, overflow: 'hidden' }}
491         >
492           <LinearGradient colors={[theme.heroTop, theme.hero, theme.heroDeep]} start={{ x: 0,
y: 0 }} end={{ x: 1, y: 1 }} style={{ padding: space.lg }}>
493             <Shine times={1} delay={1400} color="rgba(255,255,255,0.18)" />
494             <Row gap={space.md}>
495               <View style={{ width: 44, height: 44, borderRadius: 22, backgroundColor:
alpha(theme.heroAccent, 0.16), alignItems: 'center', justifyContent: 'center' }}>
496                 <Ionicons name="sparkles" size={22} color={theme.heroAccent} />
497               </View>
498               <View style={{ flex: 1 }}>
499                 <T v="h3" color={theme.heroInk}>{`สรุปเดือน${formatThaiMonth(recapMonth)}`}</T>
500                 <T v="small" color={theme.heroInkSoft}>เงินไปไหน วันไหนใช้เยอะ และเก่งขึ้นแค่ไหน</T>
501               </View>
502               <Ionicons name="play-circle" size={30} color={theme.heroAccent} />
503             </Row>
504           </LinearGradient>
505         </Pressable>
506       </Reveal>
507     ) : null}
```

```

508
509     {/* Savings goals: the one closest to its goal, or an invitation to start one */}
510     <Reveal index={5}>
511         {(() => {
512             const open = goals.filter((g) => g.savedSatang < g.targetSatang).sort((a, b) =>
goalProgress(b) - goalProgress(a));
513             const g = open[0] ?? goals[0];
514             return (
515                 <Card onPress={() => router.push(goals.length ? '/goals' : '/goal')} style={{
paddingVertical: space.md }}>
516                     <Row gap={space.md}>
517                         {g ? <Jar progress={goalProgress(g)} size={44} /> : <T v="h1">🐱</T>}
518                         <View style={{ flex: 1 }}>
519                             <T v="h3">{g ? `${g.emoji} ${g.title}` : 'ตั้งกระปุกออม'}</T>
520                             <T v="small">{g ? goalLine(g, today) : 'อยากได้อะไร ตั้งเป้าแล้วหยอดทีละเม็ด'}</T>
521                         </View>
522                         <Ionicons name="chevron-forward" size={20} color={theme.inkFaint} />
523                     </Row>
524                 </Card>
525             );
526         })()}
527     </Reveal>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/index.tsx:471`

บรรทัด	ความหมาย
471–482	ปุ่มลัด 2 แถว: สแกนสลิป / จดด้วยเสียง / จดรายการ / เพิ่มเงินเข้า (อธิบายในเล่ม FR-1)
484	แสดงการ์ดสรุปเดือนเฉพาะเมื่อ <code>recapMonth</code> ไม่เป็น <code>null</code>
486–490	ถ้าการ์ดกดได้ เปิดหน้า /recap พร้อมพารามิเตอร์ <code>month</code> ป้ายอ่านจ่อ "ดูสรุปเดือนตุลาคม 2569"
492–493	พื้นโลโก้เขียวแบบ hero และแสงว๊ิง 1 ครั้ง
495–497	วงกลมไอคอนประกาย
498–501	หัวข้อ "สรุปเดือนตุลาคม 2569" และ "เงินไปไหน วันไหนใช้เยอะ และเก่งขึ้นแค่ไหน"
502	ไอคอนปุ่มเล่น ▶
509–527	การ์ดกระปุกออม (FR-6): หากกระปุกที่ยังไม่ครบ เรียงตามความคืบหน้า แสดงใบที่ใกล้เป่าที่สุด หรือชวนตั้งกระปุก ((() => { . . . }) () คือฟังก์ชันที่เรียกตัวเองทันที ใช้เขียนตัวแปรชั่วคราวใน JSX)

อ่านโค้ด: หน้าหลัก index.tsx ตอนที่ 3 — ส่วนภาพรวมของ FR-2

บรรทัด 529–629 คือส่วนที่ FR-2 เป็นเจ้าของโดยตรง อ่านคู่กับภาพหน้าจอได้ที่ละแถว

ส่วนที่ 13: ปุ่มช่วง และเงินเข้า/ออก (บรรทัด 529–556)

src/app/(tabs)/index.tsx

บรรทัด 529–556

```

529     {/* FR-2 overview by range */}
530     <SectionTitle title="ภาพรวม" />
531     <Segmented<RangeKey> options={RANGE_ORDER.map((k) => ({ key: k, label: RANGE_LABEL[k] })))}
value={range} onChange={setRange} />
532     <Row justify="space-between" style={{ marginTop: -space.sm }}>
533       <Row gap={4}>
534         <Ionicons name="time-outline" size={14} color={theme.inkSoft} />
535         <T v="micro">{formatRangeSpan(range)}</T>
536       </Row>
537       <T v="micro" color={summary.netSatang >= 0 ? theme.good : theme.critical}>
สุทธิ {formatBaht(summary.netSatang, { sign: true, decimals: false })}
538     </T>
539     </Row>
540     <Row gap={space.sm} align="stretch">
541       <Card style={{ flex: 1 }}>
542         <Row gap={6}>
543           <Ionicons name="arrow-down-circle" size={18} color={theme.income} />
544           <T v="small">เงินเข้า</T>
545         </Row>
546         <Money satang={summary.incomeSatang} size="h3" color={theme.income} decimals={false}
countUp />
547       </Card>
548       <Card style={{ flex: 1 }}>
549         <Row gap={6}>
550           <Ionicons name="arrow-up-circle" size={18} color={theme.inkSoft} />
551           <T v="small">เงินออก</T>
552         </Row>
553         <Money satang={summary.expenseSatang} size="h3" decimals={false} countUp />
554       </Card>
555     </Row>
556

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:529

บรรทัด	ความหมาย
529	คอมเมนต์: ภาพรวมตามช่วงของ FR-2
530	หัวข้อส่วน "ภาพรวม"
531	ปุ่มสลับช่วง: สร้างตัวเลือกจาก RANGE_ORDER แต่ละตัวมี key และ label จาก RANGE_LABEL ค่าปัจจุบัน = range และแล้ว setRange → summary คำนวณใหม่ (บรรทัด 188)
532–536	ซ้าย: ไอคอนนาฬิกา และข้อความช่วงจริง "10 ก.ย. – 9 ต.ค. 2569 · 30 วันเต็ม"

บรรทัด	ความหมาย
537–539	ขวา: "สุทธิ +฿1,858" สีเขียวถ้า ≥ 0 สีแดงถ้าติดลบ แสดงเครื่องหมายเสมอ ไม่มีทศนิยม
541	แถวสองการ์ดสูงเท่ากัน (stretch)
542–548	การ์ด "เงินเข้า": ไอคอนลูกศรลงสีรายรับ และ Money ของ incomeSatang สีรายรับ ไม่มีทศนิยม นับขึ้น
549–555	การ์ด "เงินออก": ไอคอนลูกศรขึ้นสีจาง และ Money ของ expenseSatang

i ทำไมเงินเข้าใช้ลูกศรลง เงินออกใช้ลูกศรขึ้น

มองเป็นกระเป๋าเงิน: เงินเข้า = โหล **ลง** มาในกระเป๋า เงินออก = ลอย **ขึ้น** ออกไป (แบบเดียวกับแอปธนาคารหลายแอป) และสังเกตว่าเงินออกไม่ได้ใช้สีแดง เพราะการใช้เงินไม่ใช่เรื่องผิด สีแดงเก็บไว้สำหรับ "ติดลบ" และ "เกินงบ" เท่านั้น

ส่วนที่ 14: การ์ดรายจ่าย/รายรับตามหมวด (บรรทัด 557–581)

src/app/(tabs)/index.tsx

บรรทัด 557–581

```

557     <Card>
558       <Row justify="space-between">
559         <T v="h3">รายจ่ายตามหมวด</T>
560         <T v="small">
561           {formatThaiDay(summary.from, { year: false })} – {formatThaiDay(summary.to, { year:
false })}
562         </T>
563       </Row>
564       {summary.byCategory.expense.length > 0 ? (
565         <CategoryBars items={summary.byCategory.expense} />
566       ) : (
567         <EmptyState
568           icon="pie-chart-outline"
569           art={<Buddy mood="calm" size={84} />}
570           title="ยังไม่มีรายจ่ายในช่วงนี้"
571           body="ลองเลือกช่วงที่ยาวขึ้น หรือสแกนสลิปเพื่อเพิ่มรายการ"
572         />
573       )}
574       {summary.byCategory.income.length > 0 ? (
575         <>
576           <Divider />
577           <T v="h3">รายรับตามหมวด</T>
578           <CategoryBars items={summary.byCategory.income} limit={3} />
579         </>
580       ) : null}
581     </Card>

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:557

บรรทัด	ความหมาย
557–563	หัวการ์ด "รายจ่ายตามหมวด" และมุมมอง "10 ก.ย. – 9 ต.ค." (วันแรก–วันสุดท้ายของ summary ไม่มีปี)
564–565	ถ้ามีรายจ่ายในช่วง: แถบหมวดของรายจ่าย (5 หมวดแรก)

บรรทัด	ความหมาย
566–573	ไม่มี: หน้าว่าง นื่องคลาสสบ "ยังไม่มีรายจ่ายในช่วงนี้" และ "ลองเลือกช่วงที่ยาวขึ้น หรือสแกนสลิปเพื่อเพิ่มรายการ"
574–580	ถ้ามีรายรับ: เส้นคั่น หัวข้อ "รายรับตามหมวด" และแถบหมวด จำกัด 3 หมวด

ส่วนที่ 15: กราฟ 7 วัน และงบเดือน (บรรทัด 583–605)

src/app/(tabs)/index.tsx	บรรทัด 583–605
<pre> 583 <Card> 584 <Row justify="space-between"> 585 <T v="h3">รายจ่าย 7 วันล่าสุด</T> 586 <T v="small">เฉลี่ย {formatBaht(average.averageSatang, { decimals: false })}/วัน</T> 587 </Row> 588 <DayBars days={week} averageSatang={average.averageSatang} /> 589 <T v="micro">เส้นประ = ค่าเฉลี่ยต่อวันที่ใช้คำนวณ Money Runway</T> 590 </Card> 591 592 {profile?.monthlyBudgetSatang ? (593 <Card> 594 <Row justify="space-between"> 595 <T v="h3">งบเดือนนี้</T> 596 <T v="small"> 597 {formatBaht(spentMonth, { decimals: false })} / 598 {formatBaht(profile.monthlyBudgetSatang, { decimals: false })} 599 </T> 600 </Row> 601 <ProgressBar 602 value={spentMonth / profile.monthlyBudgetSatang} 603 color={spentMonth > profile.monthlyBudgetSatang ? theme.critical : spentMonth > 604 profile.monthlyBudgetSatang * 0.85 ? theme.watch : theme.primary} 605 /> 606 </Card> 607) : null} </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:583

บรรทัด	ความหมาย
583–587	หัวการ์ด "รายจ่าย 7 วันล่าสุด" และ "เฉลี่ย ฿695/วัน" (ค่าเฉลี่ยของ FR-6)
588	DayBars ได้ข้อมูล 7 ช่องจาก week และค่าเฉลี่ยไว้วาดเส้นประ
589	คำอธิบาย "เส้นประ = ค่าเฉลี่ยต่อวันที่ใช้คำนวณ Money Runway"
592	การ์ดงบแสดง เฉพาะเมื่อผู้ใช้ตั้งงบ (monthlyBudgetSatang มีค่า)
594–599	"งบเดือนนี้" และ "฿5,091 / ฿10,000"
600–603	แถบความคืบหน้า ค่า = ใช้ไป ÷ งบ สี: เกินงบ → แดง, เกิน 85% → เหลือง, นอกนั้น → สีส้ม

ส่วนที่ 16: รายการล่าสุด (บรรทัด 607–629)

src/app/(tabs)/index.tsx	บรรทัด 607–629
--------------------------	----------------

```

607     <SectionTitle title="รายการล่าสุด" action="ดูทั้งหมด" onAction={() =>
router.navigate('/transactions')} />
608     <Card style={{ paddingVertical: space.xs, gap: 0 }}>
609       {recent.length === 0 ? (
610         <EmptyState
611           icon="wallet-outline"
612           art={<Buddy mood="sleepy" size={84} />}
613           title="ยังไม่มีรายการ"
614           body="เริ่มจากสแกนสลิปใบแคชเลอว์ หรือจดยรายการแรกด้วยตัวเอง"
615           action="สแกนสลิป"
616           onAction={() => router.push('/scan')}
617         />
618       ) : (
619         recent.map((t, i) => (
620           <Reveal key={t.id} index={i} from={10}>
621             {i > 0 ? <Divider /> : null}
622             <TxRow tx={t} />
623           </Reveal>
624         ))
625       )}
626     </Card>
627   </Screen>
628 );
629 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:607

บรรทัด	ความหมาย
607	หัวข้อ "รายการล่าสุด" พร้อมลิงก์ "ดูทั้งหมด" ไปเก็บรายการ
608	การ์ดระยะในแคบ (ให้แถวชิดกัน)
609–617	ไม่มีรายการ: นื่องกล้างวง "ยังไม่มีรายการ" และปุ่ม "สแกนสลิป"
619–624	มี: วนทั้ง 5 รายการ แต่ละแถวค่อย ๆ โผล่ตามลำดับ มีเส้นคั่นยกเว้นแถวแรก วาดด้วย TxRow (ของ FR-1)
627–629	ปิด Screen, return และฟังก์ชัน

อ่านโค้ด: กราฟ charts.tsx ทั้งไฟล์

166 บรรทัด กราฟ 2 แบบ: แท่งรายวัน (DayBars) และแถบหมวด (CategoryBars) วาดด้วย react-native-svg และ View ธรรมดา

≈ SVG คืออะไร

SVG (Scalable Vector Graphics) คือการวาดภาพด้วย "คำสั่ง" แทนจุดสี เช่น "ลากเส้นจาก (0,120) ถึง (300,120)" หรือ "เขียนคำว่า วันนี้ ที่ (280,134)" ขยายแค่ไหนก็คมชัด แอปเราใช้ไลบรารี react-native-svg ให้คำสั่งเดียวกันวาดได้ทั้งบนมือถือและเว็บ ส่วนแท่งกราฟใช้ View (กล่องสี่เหลี่ยม) เพราะทำแอนิเมชันโต้ขึ้นได้เร็วกว่า

ส่วนที่ 1: import และแท่งที่โต้ขึ้น RisingBar (บรรทัด 1-42)

src/ui/charts.tsx

บรรทัด 1-42

```

1  /**
2   * Small charts drawn with react-native-svg. One scale per chart; every label
3   * names a value the chart actually shows.
4   */
5  import { useEffect, useState } from 'react';
6  import { Animated, Easing, View, type LayoutChangeEvent } from 'react-native';
7  import Svg, { Line, Text as SvgText } from 'react-native-svg';
8  import { formatBaht } from '../domain/money';
9  import type { CategoryTotal, DayTotal } from '../domain/summary';
10 import { Row, T } from '../components';
11 import { GrowBar } from '../effects';
12 import { useNative, useReduceMotion } from '../motion';
13 import { fonts, space, useTheme } from '../theme';
14
15 /** One bar of DayBars: it rises out of the baseline (its rounded top stays round). */
16 function RisingBar({ x, top, width, height, color, opacity, delay }: { x: number; top: number;
width: number; height: number; color: string; opacity: number; delay: number }) {
17   const reduce = useReduceMotion();
18   const [v] = useState(() => new Animated.Value(reduce ? 1 : 0));
19   useEffect(() => {
20     if (reduce) {
21       v.setValue(1);
22       return;
23     }
24     const anim = Animated.timing(v, { toValue: 1, duration: 650, delay, easing:
Easing.out(Easing.back(1.4)), useNativeDriver: useNative });
25     anim.start();
26     return () => anim.stop();
27   }, [reduce, v, delay, height]);
28   if (height <= 0) return null;
29   return (
30     <View style={{ position: 'absolute', left: x, top, width, height, overflow: 'hidden',
borderRadius: Math.min(6, width / 2) }}>
31       <Animated.View
32         style={{
33           flex: 1,
34           borderRadius: Math.min(6, width / 2),

```

```

35         backgroundColor: color,
36         opacity,
37         transform: [{ translateY: v.interpolate({ inputRange: [0, 1], outputRange: [height, 0]
38       }) }],
39     }},
40   </View>
41 );
42 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/charts.tsx:1`

บรรทัด	ความหมาย
1–4	คอมเมนต์ (หลักออกแบบกราฟ): หนึ่งกราฟหนึ่งสเกล และ ทุกป้ายต้องบอกค่าที่กราฟแสดงจริง (ไม่มีตัวเลขลอย ๆ ที่ไม่เกี่ยวกับแท่ง)
5–6	hook ของ React และชิ้นส่วน: ค่าภาพเคลื่อนไหว, เส้นโค้งความเร็ว (Easing), กล้อง, ชนิดของเหตุการณ์วัดขนาด
7	จาก react-native-svg: พื้นที่วาด (Svg), เส้น (Line), ตัวหนังสือ (Text เปลี่ยนชื่อเป็น SvgText ไม่ให้ชนกับ Text ปกติ)
8–9	แสดงเงิน และชนิดข้อมูลจาก summary.ts (ยอดหมวด ยอดรายวัน)
10–13	แถว ข้อความ แถบที่โตได้ (GrowBar) การตั้งค่าแอนิเมชัน และค่าการออกแบบ
15–16	RisingBar แท่ง 1 แท่งที่ "ลอยขึ้นจากพื้น" รับตำแหน่ง x, ขอบบน, กว้าง, สูง, สี, ความทึบ, หน่วยงาน
17–18	ถ้าลดการเคลื่อนไหว ค่าเริ่มที่ 1 (แสดงเต็มทันที) ไม่ขึ้นเริ่มที่ 0
19–27	effect: ลดการเคลื่อนไหว → ตั้งเป็น 1 จบ · ไม่ขึ้นเล่นแอนิเมชัน 0 → 1 ใน 0.65 วินาที หน่วยงานตาม delay เส้นโค้งแบบ "เร่งเกินนิดแล้วกลับ" (back 1.4) และหยุดถ้าหน้าถูกปิด
28	สูง 0 หรือติดลบ → ไม่วาด
29–30	กรอบนอกวางตำแหน่งแบบ absolute ตาม x และ top ซ่อนส่วนที่ล้น มุมโค้งไม่เกิน 6 หรือครึ่งความกว้าง
31–39	แท่งจริงข้างในเลื่อนจากล่าง (translateY = height) ขึ้นมาที่ 0 ตามค่าแอนิเมชัน — เหมือนแท่งโผล่ขึ้นจากใต้พื้น มุมบนยังโค้งสวย

ส่วนที่ 2: DayBars — เตรียมสเกล (บรรทัด 44–58)

`src/ui/charts.tsx`

บรรทัด 44–58

```

44 const WEEKDAY_SHORT = ['อา', 'จ', 'อ', 'พ', 'พฤ', 'ศ', 'ส'];
45
46 /** Daily expense bars with a dashed line at the average. */
47 export function DayBars({ days, averageSatang, height = 140 }: { days: DayTotal[];
48   averageSatang?: number; height?: number }) {
49   const theme = useTheme();
50   const [width, setWidth] = useState(0);
51   const onLayout = (e: LayoutChangeEvent) => setWidth(e.nativeEvent.layout.width);
52   const max = Math.max(1, ...days.map((d) => d.expenseSatang), averageSatang ?? 0);
53   const top = 18;
54   const bottom = 22;
55   const plotH = height - top - bottom;
56   const gap = 8;
57   const barW = width > 0 ? (width - gap * (days.length - 1)) / days.length : 0;

```

```

57   const y = (v: number) => top + plotH - (v / max) * plotH;
58   const peak = days.reduce((best, d) => (d.expenseSatang > best.expenseSatang ? d : best),
    days[0]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/charts.tsx:44

บรรทัด	ความหมาย
44	ชื่อวันย่อ อา จ อ พ พ ศ ส (ตำแหน่ง 0 = อาทิตย์)
46–47	DayBars รับ 7 วัน ค่าเฉลี่ย (ไม่บังคับ) และความสูง (ค่าเริ่มต้น 140)
48–50	ความกว้างจริงของการ์ดรู้ได้หลังวาดครั้งแรก (onLayout) จึงเก็บใน state เริ่มที่ 0
51	ค่าสูงสุดของสเกล = มากที่สุดของ (1, ทุกแท่ง, ค่าเฉลี่ย) → แท่งสูงสุดเต็มพื้นที่ และเส้นค่าเฉลี่ยไม่หลุดกราฟ
52–54	เว้นบน 18 (ที่ว่างป้ายยอด) เว้นล่าง 22 (ที่ว่างชื่อวัน) พื้นที่วาดจริง = $140 - 18 - 22 = 100$
55–56	ช่องว่างระหว่างแท่ง 8 ความกว้างแท่ง = $(\text{กว้างทั้งหมด} - \text{ช่องว่าง 6 ช่อง}) \div 7$
57	ฟังก์ชันแปลงยอด → ตำแหน่ง y: ยอดมาก = ใกล้ด้านบน (ในจอ y นับจากบนลงล่าง)
58	หาวันที่ใช้มากที่สุด (peak) เพื่อวางป้ายยอดตัวเดียว

ตัวอย่างจากภาพ: max = ฿3,579 (5 ต.ค.) แท่ง 5 ต.ค. สูงเต็ม 100 จุด · แท่ง 3 ต.ค. ฿335 สูง $335 \div 3,579 \times 100 = 9.4$ จุด · เส้นประจำเฉลี่ย ฿695 อยู่ที่ 19.4 จุดจากพื้น

ส่วนที่ 3: DayBars — วาด (บรรทัด 60–130)

src/ui/charts.tsx

บรรทัด 60–130

```

60   return (
61     <View onLayout={onLayout} style={{ height }} accessibilityLabel={`รายจ่าย ${days.length} วัน
    ล่าสุด`} >
62       /* Bars first, then the lines and labels drawn over them. */
63       {width > 0
64         ? days.map((d, i) => {
65           const h = Math.max(d.expenseSatang > 0 ? 3 : 0, top + plotH - y(d.expenseSatang));
66           const isToday = i === days.length - 1;
67           return (
68             <RisingBar
69               key={d.day}
70               x={i * (barW + gap)}
71               top={top + plotH - h}
72               width={barW}
73               height={h}
74               color={isToday ? theme.accent : theme.primary}
75               // Today stands out by brightness as well as colour (in some dark themes the two
    are close).
76               opacity={isToday ? 1 : 0.5}
77               delay={120 + i * 55}
78             </>
79           );
80         })
81         : null}
82       {width > 0 ? (
83         <Svg width={width} height={height} style={{ position: 'absolute', left: 0, top: 0 }} >

```

```

84      <Line x1={0} x2={width} y1={top + plotH} y2={top + plotH} stroke={theme.line}
strokeWidth={1} />
85      {days.map((d, i) => {
86          const x = i * (barW + gap);
87          const isToday = i === days.length - 1;
88          const dow = new Date(`${d.day}T00:00:00Z`).getUTCDay();
89          return [
90              <SvgText
91                  key={`l${d.day}`}
92                  x={x + barW / 2}
93                  y={height - 6}
94                  fontSize={11}
95                  fontFamily={fonts.sansMedium}
96                  fill={isToday ? theme.ink : theme.inkSoft}
97                  textAnchor="middle"
98              >
99                  {isToday ? 'วันนี้' : WEEKDAY_SHORT[dow]}
100          </SvgText>,
101          ];
102      })}
103      {peak && peak.expenseSatang > 0 ? (
104          <SvgText
105              x={Math.min(width - 4, Math.max(4, days.indexOf(peak) * (barW + gap) + barW / 2))}
106              y={Math.max(11, y(peak.expenseSatang) - 5)}
107              fontSize={10}
108              fontFamily={fonts.sansSemi}
109              fill={theme.inkSoft}
110              textAnchor="middle"
111          >
112              {formatBaht(peak.expenseSatang, { decimals: false })}
113          </SvgText>
114      ) : null}
115      {averageSatang && averageSatang > 0 ? (
116          <Line
117              x1={0}
118              x2={width}
119              y1={y(averageSatang)}
120              y2={y(averageSatang)}
121              stroke={theme.inkSoft}
122              strokeDasharray="4 4"
123              strokeWidth={1}
124          />
125      ) : null}
126      </Svg>
127      ) : null}
128      </View>
129      );
130  }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/charts.tsx:60

บรรทัด	ความหมาย
61	กล่องนอก วัดความกว้าง สูง 140 ป้ายอ่านจอ "รายจ่าย 7 วันล่าสุด"
62-64	วาด แท่งก่อน แล้วค่อยวาดเส้นและตัวหนังสือกับ (รอให้รู้ความกว้างก่อน)
65	ความสูงแท่ง = จากพื้นถึง y ของยอด แต่ถ้ามีรายจ่าย อย่างน้อย 3 จุด (ไม่จันวันใช้ ฿5 จะมองไม่เห็น)
66	เป็นวันนี้ไหม (ตัวสุดท้าย)

บรรทัด	ความหมาย
68–78	RisingBar: $x = \text{ลำดับ} \times (\text{กว้าง} + \text{ช่อง})$, สีทองถ้าวันนี้ ไม่ขึ้นสี่หลัก, ความทึบ 1 สำหรับวันนี้ 0.5 สำหรับวันอื่น (คอมเมนต์: บางริมมิดสองสีใกล้เคียงกัน จึงต่างด้วยความสว่างด้วย), หม่นวงกละ 55 มิลลิวินาที แท่งจึงโตไล่จากซ้ายไปขวา
82–84	พื้นที่ SVG กับเติมกล่อง เส้นพื้นสีจางที่กันกราฟ
85–88	วนทุกวัน หาตำแหน่ง x , วันนี้ใหม่, วันในสัปดาห์ (ใส่ "T00:00:00Z" เพื่อให้ได้วันของวันนั้นตรง ๆ ไม่เพี้ยนตามเขตเวลา)
89–101	ชื่อวันใต้แท่ง ขนาด 11 จัดกลางแท่ง วันนี้เขียน "วันนี้" สีเข้ม วันอื่นใช้ชื่อย่อสีจาง
103–114	ป้ายยอดสูงสุด: เฉพาะเมื่อมีรายจ่าย วางเหนือแท่ง 5 จุด (แต่ไม่สูงเกินขอบ: อย่างน้อย $y = 11$) และไม่ล้นซ้ายขวา ตัวเลขไม่มีทศนิยม เช่น "฿3,579"
115–125	เส้นประจำเฉลี่ย: ลากจากซ้ายสุดถึงขวาสุดที่ y ของค่าเฉลี่ย ลาย "4 4" = ชิด 4 เว้น 4

ส่วนที่ 4: CategoryBars (บรรทัด 132–166)

src/ui/charts.tsx

บรรทัด 132–166

```

132 /** Horizontal bars for categories, biggest first. */
133 export function CategoryBars({ items, limit = 5 }: { items: CategoryTotal[]; limit?: number }) {
134   const theme = useTheme();
135   const shown = items.slice(0, limit);
136   const max = Math.max(1, ...shown.map((c) => c.totalSatang));
137   return (
138     <View style={{ gap: space.md }}>
139       {shown.map((c, i) => (
140         <View key={c.key} style={{ gap: 6 }}>
141           <Row justify="space-between">
142             <Row gap={6} style={{ flex: 1 }}>
143               <T v="body">{c.glyph}</T>
144               <T v="small" color={theme.ink} numberOfLines={1} style={{ flex: 1 }}>
145                 {c.label}
146               </T>
147             </Row>
148             <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi, fontVariant:
149               ['tabular-nums'] }}>
150               {formatBaht(c.totalSatang, { decimals: false })}
151             </T>
152             <T v="micro" style={{ width: 38, textAlign: 'right', fontVariant: ['tabular-nums']
153               }}>
154               {Math.round(c.share * 100)}%
155             </T>
156           </Row>
157           <GrowBar
158             value={c.totalSatang / max}
159             color={i === 0 ? theme.accent : theme.primary}
160             opacity={i === 0 ? 1 : Math.max(0.3, 0.55 - i * 0.06)}
161             track={theme.surfaceAlt}
162             delay={100 + i * 90}
163           />
164         </View>
165       )})
166     </View>
167   );

```

บรรทัด	ความหมาย
132–133	แถบแนวนอนของหมวด มากสุดก่อน รับรายการหมวดและจำนวนที่จะแสดง (ค่าเริ่มต้น 5)
135	ตัดเหลือ limit ตัวแรก (รายการเรียงมาแล้วจาก summary.ts)
136	ค่าสูงสุดของที่จะแสดง (อย่างน้อย 1) ใช้เป็นความยาวเต็มแถบ
139–140	วนทีละหมวด กล่องหมวดละ 1 กล่อง
141–147	ซ้าย: ไอคอน และชื่อหมวด (ยาวเกินตัด ...)
148–150	ยอด เช่น "฿3,500" ตัวหนา ตัวเลขกว้างเท่ากัน
151–153	เปอร์เซ็นต์ในช่องกว้าง 38 ชิดขวา (ปิดเป็นจำนวนเต็ม) ทุกแถวจึงตรงคอลัมน์
155–161	แถบ: ยาว = ยอด ÷ ค่าสูงสุด, อันดับ 1 สีทองทึบเต็ม, อันดับต่อ ๆ ไปสีหลักความทึบลดลงทีละ 0.06 (ต่ำสุด 0.3), รางสีพื้นรอง, หน่วงทีละ 90 มิลลิวินาที

อันดับ (i)	สี	ความทึบ = $\max(0.3, 0.55 - i \times 0.06)$
0 (บิล)	ทอง	1
1 (อาหาร)	สีหลัก	0.49
2 (ช้อปปิ้ง)	สีหลัก	0.43
3 (บันเทิง)	สีหลัก	0.37
4 (สะดวกซื้อ)	สีหลัก	0.31

★ หลัก UX ของกราฟในแอปนี้

(1) เน้นสิ่งเดียวด้วยสีทอง (วันนี้ / หมวดที่มากที่สุด) ที่เหลือลดความเด่นลง (2) ใส่ตัวเลขเฉพาะที่จำเป็น (ยอดสูงสุดตัวเดียวในกราฟแท่ง) ไม่ใส่ทุกแท่งให้รก (3) ทุกกราฟมีป้ายอ่านจอ คนตาบอดก็รู้ว่าเป็นกราฟอะไร (4) แอนิเมชันหยุดเองและเคารพ "ลดการเคลื่อนไหว"

อ่านโค้ด: ปฏิทินการใช้จ่าย SpendCalendar.tsx ทั้งไฟล์

125 บรรทัด วาดเดือนเป็นตาราง 7 คอลัมน์ ระบายสีตามระดับที่ spendCalendar จำนวน

ส่วนที่ 1: สีสันและ props (บรรทัด 1–43)

src/ui/SpendCalendar.tsx

บรรทัด 1–43

```

1  /**
2   * The month as a calendar: each day shaded by how much was spent compared with
3   * the month's usual spending day (one hue, light to dark; ramps checked with the
4   * data-viz validator for both themes). Tap a day to see only its transactions.
5   * Rules: domain/recap.ts spendCalendar.
6   */
7  import { Pressable, View } from 'react-native';
8  import { formatThaiDay } from '../domain/dates';
9  import { formatBaht } from '../domain/money';
10 import type { CalendarDay, SpendCalendar as Calendar } from '../domain/recap';
11 import { Card, IconButton, Row, T } from '../components';
12 import { Reveal } from '../effects';
13 import { fonts, radius, space, useTheme } from '../theme';
14
15 const WEEKDAYS = ['อา', 'จ', 'อ', 'พ', 'พฤ', 'ศ', 'ส'];
16 /** Levels 1..4 (level 0 = no spending uses the neutral surface). */
17 const RAMP_LIGHT = ['#D9AD4E', '#BF8A1C', '#96640C', '#664306'];
18 const RAMP_DARK = ['#5C4A16', '#8C6D1C', '#C4972E', '#F0C75A'];
19
20 export function SpendCalendar({
21   cal,
22   selected,
23   onSelect,
24   onPrev,
25   onNext,
26 }: {
27   cal: Calendar;
28   selected: string | null;
29   onSelect: (day: string | null) => void;
30   onPrev?: () => void;
31   onNext?: () => void;
32 }) {
33   const theme = useTheme();
34   const ramp = theme.dark ? RAMP_DARK : RAMP_LIGHT;
35   const fill = (c: CalendarDay) => (c.level ? ramp[c.level - 1] : theme.surfaceAlt);
36   // Text inside a fill picks the ink that stays readable on it.
37   const ink = (c: CalendarDay) => {
38     if (!c.level) return c.future ? theme.inkFaint : theme.inkSoft;
39     if (theme.dark) return c.level >= 3 ? '#0A1410' : '#F4F1E6';
40     return c.level >= 2 ? '#FFFFFF' : '#11231B';
41   };
42   const picked = selected ? cal.weeks.flat().find((c) => c?.day === selected) ?? null : null;
43

```

▶ เปิดใน VS Code (ใหม่) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/SpendCalendar.tsx:1

บรรทัด	ความหมาย
1–6	คอมเมนต์: ระบายแต่ละวันตามยอดใช้เทียบ "วันปกติ" ของเดือน สัปดาห์อ่อนไปเข้ม (ตรวจสอบด้วยเครื่องมือตรวจสอบสีของการแสดงข้อมูลทั้งสองริม) และวันเพื่อดูเฉพาะรายการวันนั้น คติกาอยู่ใน recap.ts
7–13	import: ปุ่มกด กล้อง, วันไทย, แสดงเงิน, ชนิดของปฏิทิน (เปลี่ยนชื่อเป็น Calendar ไม่ให้ชนกับชื่อ component), การ์ด ปุ่มไอคอน แถว ข้อความ, เอฟเฟกต์ และคำการออกแบบ
15	หัวคอลัมน์ อา–ส
16–18	ไล่สี่ 4 ระดับ ของโหมดสว่าง (ทองอ่อน → น้ำตาลเข้ม) และโหมดมืด (กลับกัน: มืด → ทองสว่าง เพราะบนพื้นมืด สีสว่าง = เด่น) ระดับ 0 ใช้สีพื้นกลาง ๆ
20–32	props: ข้อมูลปฏิทิน, วันที่เลือก (หรือ null), ฟังก์ชันเลือกวัน, ฟังก์ชันเดือนก่อน/ถัดไป (ไม่ส่ง = ปุ่มนั้นกดไม่ได้)
33–34	สีริม และเลือกชุดไล่สีตามโหมด
35	สีพื้นของวัน: ระดับ 1–4 หยิบจากชุดไล่สี (ตำแหน่ง ระดับ – 1) ระดับ 0 สีพื้นรอง
36–41	สีตัวเลขบนพื้น ต้องอ่านออก: ระดับ 0 → สีจาง (วันอนาคตจางกว่า) · โหมดมืด ระดับ 3–4 (พื้นสว่าง) ใช้ตัวเข้ม ระดับ 1–2 ใช้ตัวสว่าง · โหมดสว่าง ระดับ 2–4 (พื้นเข้ม) ใช้ตัวขาว ระดับ 1 ใช้ตัวเข้ม
42	หาข้อมูลของวันที่เลือกจากทุกสัปดาห์ (flat () แฉตาราง 2 ชั้นเป็นรายการเดียว)

ส่วนที่ 2: หัวปฏิทินและตารางวัน (บรรทัด 44–95)

src/ui/SpendCalendar.tsx

บรรทัด 44–95

```

44   return (
45     <Card style={{ gap: space.sm, paddingVertical: space.md }}>
46       <Row justify="space-between">
47         <IconButton icon="chevron-back" label="เดือนก่อน" onPress={() => onPrev?.()} color={onPrev ?
theme.ink : theme.line} />
48         <T v="h3">{cal.label}</T>
49         <IconButton icon="chevron-forward" label="เดือนถัดไป" onPress={() => onNext?.()} color=
{onNext ? theme.ink : theme.line} />
50       </Row>
51       <View style={{ flexDirection: 'row' }}>
52         {WEEKDAYS.map((w) => (
53           <T key={w} v="micro" center style={{ flex: 1 }}>
54             {w}
55           </T>
56         ))}
57       </View>
58       {cal.weeks.map((week, wi) => (
59         <Reveal key={`_${cal.month}-${wi}`} index={wi} from={6}>
60           <View style={{ flexDirection: 'row' }}>
61             {week.map((c, di) =>
62               c ? (
63                 <View key={c.day} style={{ flex: 1, aspectRatio: 1, padding: 2 }}>
64                   <Pressable
65                     disabled={c.future}
66                     onPress={() => onSelect(selected === c.day ? null : c.day)}
67                     accessibilityRole="button"
68                     aria-selected={selected === c.day}

```

```

69         accessibilityLabel={` ${formatThaiDay(c.day)} ${c.expenseSatang ? `ใช้ไป`  

    ${formatBaht(c.expenseSatang)}` : `ไม่มีรายจ่าย`} ${c.incomeSatang ? ` เงินเข้า`  

    ${formatBaht(c.incomeSatang)}` : ``}  

70         style={({ pressed }) => ({  

71             flex: 1,  

72             borderRadius: radius.sm,  

73             backgroundColor: c.future ? 'transparent' : fill(c),  

74             alignItems: 'center',  

75             justifyContent: 'center',  

76             borderWidth: selected === c.day ? 2 : c.isToday ? 1.5 : 0,  

77             borderColor: selected === c.day ? theme.primary : theme.ink,  

78             opacity: pressed ? 0.7 : 1,  

79         })}  

80     >  

81     <T v="small" color={ink(c)} style={{ fontFamily: c.isToday || selected ===  

    c.day ? fonts.sansBold : fonts.sans, fontVariant: ['tabular-nums'] }}>  

82         {Number(c.day.slice(8, 10))}  

83     </T>  

84     {c.incomeSatang > 0 ? (  

85         <View style={{ position: 'absolute', top: 4, right: 4, width: 6, height: 6,  

    borderRadius: 3, backgroundColor: theme.income }} />  

86     ) : null}  

87     </Pressable>  

88 </View>  

89 ) : (  

90     <View key={`e${wi}-${di}`} style={{ flex: 1, aspectRatio: 1 }} />  

91 ),  

92 )}  

93 </View>  

94 </Reveal>  

95 )})

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/SpendCalendar.tsx:44

บรรทัด	ความหมาย
45	การ์ดครอบปฏิทิน
46–50	แถวหัว: ปุ่ม < "เดือนก่อน" (สืจางถ้าไม่มี onPrev), ชื่อเดือน "กันยายน 2569", ปุ่ม > "เดือนถัดไป" (onPrev?.() = เรียกถ้ามี)
51–57	แถวหัวคอลัมน์ 7 ช่องกว้างเท่ากัน
58–60	วนทีละสัปดาห์ แต่ละสัปดาห์ค่อย ๆ โผล่ตามลำดับ
61–63	วนทีละช่อง ถ้าเป็นวันจริง: คล่องสี่เหลี่ยมจัตุรัส (aspectRatio: 1)
64–66	ปุ่มกด ปิดสำหรับวันอนาคต และแล้ว: ถ้าเป็นวันที่เลือกอยู่ → ยกเลิก (null) ไม่จั่นเลือกวันนั้น
67–69	บอกโปรแกรมอ่านจั่วว่าเป็นปุ่ม ถูกเลือกไหม และอ่านเป็นประโยค: "5 ก.ย. 2569 ใช้ไป ฿3,760.00 เงินเข้า ..." (เทสต์ E2E หาปุ่มที่มีคำว่า "ใช้ไป ฿" แล้วแตะ)
70–79	หน้าต่าง: มุมโค้งเล็ก ติดตามระดับ (วันอนาคตโปร่งใส) ขอบ 2 สีหลักถ้าถูกเลือก ขอบ 1.5 สีเข้มถ้าเป็นวันนี้ กดแล้วจางลง
81–83	เลขวัน (Number("05") = 5) ตัวหนาถ้าวันนี้หรือถูกเลือก
84–86	จุดเขี้ยว 6×6 มุมขวาบนถ้ามีเงินเข้า
89–91	ช่องว่าง (วันของเดือนอื่น) เป็นคล่องเปล่าขนาดเท่ากัน ตารางจึงไม่เบี้ยว

ส่วนที่ 3: คำอธิบายสี และแถบวันที่เลือก (บรรทัด 96–125)

src/ui/SpendCalendar.tsx

บรรทัด 96–125

```
96      {/* Legend: the ramp from little to a lot, and the income dot. */}
97      <Row justify="space-between" style={{ marginTop: space.xs }}>
98        <Row gap={4}>
99          <T v="micro">น้อย</T>
100          {ramp.map((c) => (
101            <View key={c} style={{ width: 14, height: 14, borderRadius: 4, backgroundColor: c }}
102          ))}
103          <T v="micro">มาก</T>
104        </Row>
105        <Row gap={4}>
106          <View style={{ width: 6, height: 6, borderRadius: 3, backgroundColor: theme.income }}
107        </Row>
108      </Row>
109      <T v="micro">มีเงินเข้า</T>
110      {cal.usualSatang ? `เทียบกับวันที่ใช้ตามปกติของเดือนนี้ (${formatBaht(cal.usualSatang, { decimals:
111        false })})` : 'เดือนนี้ยังไม่มีรายจ่าย'}
112      </T>
113      {picked ? (
114        <Row justify="space-between" style={{ backgroundColor: theme.surfaceAlt, borderRadius:
115          radius.lg, paddingHorizontal: space.md, paddingVertical: space.sm }}>
116          <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi }}>
117            {formatThaiDay(picked.day)}
118          </T>
119          <T v="small" color={theme.ink}>
120            {picked.expenseSatang ? `ใช้ ${formatBaht(picked.expenseSatang, { decimals: false })}`
121            : 'ไม่มีรายจ่าย'} · {picked.count} รายการ
122          </T>
123        </Row>
124      ) : null}
125    </Card>
  );
}
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/SpendCalendar.tsx:96

บรรทัด	ความหมาย
96–104	คำอธิบาย: "น้อย" □□□□ "มาก" (สี่เหลี่ยม 4 สีของชุดไล่สี)
105–108	จุดเขียว "มีเงินเข้า"
110–112	ถ้ามีวันปกติ: "เทียบกับวันที่ใช้ตามปกติของเดือนนี้ (฿349)" ไม่มี: "เดือนนี้ยังไม่มีรายจ่าย"
113–122	ถ้าเลือกวันอยู่: แถบมุมโค้ง ช้าย "5 ก.ย. 2569" ขวา "ใช้ ฿3,760 · 6 รายการ" (หรือ "ไม่มีรายจ่าย")

i ปฏิทินอยู่ในแท็บรายการ แต่เป็นงานของ FR-2

ไฟล์ `src/app/(tabs)/transactions.tsx` (FR-1) เป็นที่วางปฏิทิน: เก็บเดือนที่ดูและวันที่เลือก แล้วกรอกรายการด้านล่างตามวันที่เลือก ส่วน "คำนวณสิ" (recap.ts) และ "วาดปฏิทิน" (SpendCalendar.tsx) เป็นของ FR-2 นี่คือตัวอย่างที่สอง FR ทำงานร่วมกันบนหน้าเดียว

อ่านโค้ด: src/domain/recap.ts ทั้งไฟล์

179 บรรทัด สมองของ 'สรุปเดือน' และ 'ปฏิทินการใช้จ่าย' เป็นฟังก์ชันล้วนทั้งหมด (TC-69, TC-70)

ส่วนที่ 1: รูปร่างของสรุปเดือน (บรรทัด 1–38)

src/domain/recap.ts

บรรทัด 1–38

```

1  /**
2   * Month recap ("สรุปเดือน"), shown as a story of a few full screens: how much
3   * went out and came in, where it went, the biggest day, habits (slips, voice,
4   * streak) and how it compares with the month before. Pure; unit tests in
5   * __tests__/recap.test.ts (TC-69).
6   */
7   import { VOICE_NOTE } from '../achievements';
8   import { getCategory } from '../categories';
9   import { bkkDayKey, daysInMonth, formatThaiMonth, previousMonth } from '../dates';
10  import type { Transaction } from '../types';
11
12  export interface MonthRecap {
13    /** "YYYY-MM" */
14    month: string;
15    /** "กันยายน 2569" */
16    label: string;
17    /** The month is still going (the recap counts up to today). */
18    partial: boolean;
19    /** Days counted: the whole month, or up to today. */
20    days: number;
21    expenseSatang: number;
22    incomeSatang: number;
23    /** Confirmed transactions in the month. */
24    count: number;
25    /** Up to three categories where the money went, biggest first. */
26    top: { key: string; label: string; glyph: string; amountSatang: number; share: number }[];
27    biggestDay: { day: string; amountSatang: number } | null;
28    /** Days (of those counted) with no spending at all. */
29    noSpendDays: number;
30    avgPerDaySatang: number;
31    /** The payee paid most often. */
32    favorite: { title: string; times: number } | null;
33    /** How the month was recorded. */
34    from: { slips: number; voice: number; typed: number };
35    /** Spending vs the same days of the month before, in percent (null when there is nothing to
36    compare). */
37    changePct: number | null;
38    prevLabel: string;
39  }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recap.ts:1

บรรทัด ความหมาย

1–6 คอมเมนต์: สรุปเดือนแสดงเป็นเรื่องเล่าหลายหน้าจอ: ใช้ไป/เงินเข้า, เงินไปไหน, วันที่ใช้มากที่สุด, บิลล์ (สลิป เสียง จดต่อ
เนื่อง) และเทียบเดือนก่อน เป็นฟังก์ชันล้วน มีเทสต์ TC-69

บรรทัด	ความหมาย
7	VOICE_NOTE: ข้อความที่ FR-3 ใส่ในบันทึกของรายการที่พุดจต (ใช้แยกจากรายการโหนดมาจากเสียง)
8–10	หมวด, ฟังก์ชันวันที่, ชนิด Transaction
12–16	MonthRecap: เดือน "YYYY-MM" และชื่อ "กันยายน 2569"
17–20	partial = เดือนยังไม่จบ (นับถึงวันนี้) · days = จำนวนวันที่นับ
21–24	รายจ่าย รายรับ และจำนวนรายการที่ยืนยันในเดือน
25–26	top = ไม่เกิน 3 หมวดที่ใช้มากที่สุด (รหัส ชื่อ ไอคอน ยอด ส่วนแบ่ง)
27	วันที่ใช้มากที่สุด หรือ null ถ้าไม่มีรายจ่าย
28–30	วันที่ไม่ใช่เงินเลย และเฉลี่ยต่อวัน
31–32	ผู้รับเงินที่จ่ายบอยสุด
33–34	จดด้วยวิธีไหน: สลิป / เสียง / พิมพ์
35–37	เปอร์เซ็นต์เทียบวันเดียวกันของเดือนก่อน (null = ไม่มีให้เทียบ) และชื่อเดือนก่อน

ส่วนที่ 2: recapMonths และ monthRecap ครั้งแรก (บรรทัด 40–72)

src/domain/recap.ts

บรรทัด 40–72

```

40 /** Months that have confirmed transactions, newest first ("YYYY-MM"). */
41 export function recapMonths(txs: Transaction[]): string[] {
42   const set = new Set(txs.filter((t) => t.status === 'confirmed').map((t) =>
43     bkkDayKey(t.occurredAt).slice(0, 7)));
44   return [...set].sort().reverse();
45 }
46 export function monthRecap(txs: Transaction[], month: string, today: string): MonthRecap {
47   const partial = today.slice(0, 7) === month;
48   const days = partial ? Number(today.slice(8, 10)) : daysInMonth(month);
49   const inMonth = txs.filter((t) => t.status === 'confirmed' && bkkDayKey(t.occurredAt).slice(0,
50     7) === month && bkkDayKey(t.occurredAt) <= today);
51   const expenses = inMonth.filter((t) => t.kind === 'expense');
52   const expenseSatang = expenses.reduce((s, t) => s + t.amountSatang, 0);
53   const incomeSatang = inMonth.filter((t) => t.kind === 'income').reduce((s, t) => s +
54     t.amountSatang, 0);
55   const byCat = new Map<string, number>();
56   const byDay = new Map<string, number>();
57   const byPayee = new Map<string, { title: string; times: number }>();
58   for (const t of expenses) {
59     byCat.set(t.categoryKey, (byCat.get(t.categoryKey) ?? 0) + t.amountSatang);
60     const day = bkkDayKey(t.occurredAt);
61     byDay.set(day, (byDay.get(day) ?? 0) + t.amountSatang);
62     const k = t.title.trim().toLowerCase();
63     if (k) byPayee.set(k, { title: t.title.trim(), times: (byPayee.get(k)?.times ?? 0) + 1 });
64   }
65   const top = [...byCat.entries()]
66     .sort((a, b) => b[1] - a[1])
67     .slice(0, 3)

```

```

67     .map(([key, amountSatang]) => {
68         const c = getCategory(key);
69         return { key, label: c.label, glyph: c.glyph, amountSatang, share: expenseSatang ?
amountSatang / expenseSatang : 0 };
70     });
71     const biggest = [...byDay.entries()].sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]))
[0];
72     const favorite = [...byPayee.values()].sort((a, b) => b.times - a.times)[0];

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recap.ts:40

บรรทัด	ความหมาย
40–44	recapMonths: เดือนทั้งหมดที่มีรายการยืนยัน ใหม่สุดก่อน (Set ตัดเดือนซ้ำ, sort เรียงเก่า→ใหม่, reverse กลับด้าน) ใช้ในเก็บรายการหาเดือนเก่าสุดที่ย้อนได้
46	monthRecap รับรายการ เดือน และวันนี้
47	เดือนนี้ยังไม่จบไหม = เดือนของวันนี้ตรงกับเดือนที่สรุป
48	วันที่นับ: ยังไม่จบ → เลขวันของวันนี้ (9) · จบแล้ว → จำนวนวันของเดือน (30 หรือ 31)
49	รายการในเดือน: ยืนยัน + เดือนตรง + ไม่เก็บวันนี้ (รายการล่วงหน้าไม่นับ — เทสต์ใส่รายการ 30 ก.ย. ตอนวันนี้ 15 ก.ย. เพื่อตรวจสอบ)
50–52	แยกรายจ่าย รวมยอดรายจ่าย และรวมยอดรายรับ
54–56	Map 3 ตัว: ยอดตามหมวด, ยอดตามวัน, ผู้รับเงิน (ชื่อ → {ชื่อ, จำนวนครั้ง})
57–60	วนรายจ่าย บวกยอดเข้าหมวดและเข้าวัน
61–62	ชื่อผู้รับแบบตัดช่องว่างและตัวเล็ก ("ข้าวมันไก่" = "ข้าวมันไก่") ถ้าไม่ว่าง นับครั้งเพิ่ม และจำชื่อที่สวย (ตัดช่องว่าง)
64–70	top: เรียงหมวดมากไปน้อย เอา 3 อันดับ เติมชื่อ ไอคอน และส่วนแบ่ง (ยอดหมวด ÷ รายจ่ายรวม ถ้ารายจ่ายเป็น 0 ใช้ 0)
71	วันที่ใช้มากที่สุด: เรียงยอดมากไปน้อย ถ้าเท่ากันเอาวันที่มาก่อน (localeCompare) แล้วเอาตัวแรก
72	ผู้รับที่จ่ายบ่อยสุด (ยังไม่ตรวจว่าถึง 2 ครั้ง ตรวจที่บรรทัด 97)

ส่วนที่ 3: เทียบเดือนก่อน และผลลัพธ์ (บรรทัด 74–106)

src/domain/recap.ts

บรรทัด 74–106

```

74 // Fair comparison: the same number of days at the start of the month before.
75 const prev = previousMonth(month);
76 const prevDays = Math.min(days, daysInMonth(prev));
77 const prevSpent = txs
78     .filter((t) => t.status === 'confirmed' && t.kind === 'expense')
79     .filter((t) => {
80         const d = bkkDayKey(t.occurredAt);
81         return d.slice(0, 7) === prev && Number(d.slice(8, 10)) <= prevDays;
82     })
83     .reduce((s, t) => s + t.amountSatang, 0);
84
85 return {
86     month,
87     label: formatThaiMonth(month),

```

```

88     partial,
89     days,
90     expenseSatang,
91     incomeSatang,
92     count: inMonth.length,
93     top,
94     biggestDay: biggest ? { day: biggest[0], amountSatang: biggest[1] } : null,
95     noSpendDays: Math.max(0, days - byDay.size),
96     avgPerDaySatang: days ? Math.round(expenseSatang / days) : 0,
97     favorite: favorite && favorite.times >= 2 ? favorite : null,
98     from: {
99       slips: inMonth.filter((t) => t.source === 'slip').length,
100      voice: inMonth.filter((t) => t.source !== 'slip' && t.note === VOICE_NOTE).length,
101      typed: inMonth.filter((t) => t.source !== 'slip' && t.note !== VOICE_NOTE).length,
102    },
103     changePct: prevSpent > 0 ? Math.round(((expenseSatang - prevSpent) / prevSpent) * 100) :
      null,
104     prevLabel: formatThaiMonth(prev),
105   };
106 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/recap.ts:74`

บรรทัด	ความหมาย
74	คอมเมนต์: เก็บอย่างยุติธรรม = จำนวนวันเท่ากัน นับจากต้นเดือนก่อน
75	เดือนก่อน
76	จำนวนวันที่เก็บ = น้อยกว่าระหว่าง (วันที่นับเดือนนี้, จำนวนวันของเดือนก่อน) เช่น สรุ 31 มี.ค. เก็บ ก.พ. ได้แค่ 28 วัน
77–83	รวมรายจ่ายยืนยันของเดือนก่อนที่วันที่ $\leq$ จำนวนวันนั้น
85–92	ผลลัพธ์: เดือน ชื่อ ยังไม่จบไหม วันที่นับ รายจ่าย รายรับ จำนวนรายการ
93	top 3 หมวด
94	วันที่ใช้มากที่สุด (ถ้ามี)
95	วันไม่ใช้เงิน = วันที่นับ – จำนวนวันที่มีรายจ่าย (ไม่ต่ำกว่า 0)
96	เฉลี่ยต่อวัน = รายจ่าย ÷ วันที่นับ ปิดเป็นสตางค์จำนวนเต็ม
97	จ่ายบ่อยสุด ต้องอย่างน้อย 2 ครั้ง (จ่ายครั้งเดียวไม่ถือว่า "บ่อย")
98–102	นับวิธีจด: สลิป = source 'slip' · เสียง = ไม่ใช้สลิปและบันทึก = VOICE_NOTE · พิมพ์ = ที่เหลือ
103	เปอร์เซ็นต์เปลี่ยน = (เดือนนี้ – เดือนก่อน) ÷ เดือนก่อน × 100 ปิดเป็นจำนวนเต็ม ถ้าเดือนก่อนเป็น 0 → null (หารศูนย์ไม่ได้)
104	ชื่อเดือนก่อน "สิงหาคม 2569"

ส่วนที่ 4: recapToOffer (บรรทัด 108–120)

`src/domain/recap.ts`

บรรทัด 108–120

```

108 /**
109  * Which month to offer on the home screen: last month during the first week
110  * of a new month (if it had anything), otherwise this month once it has a few
111  * records. Null when there is not enough to tell a story yet.
112  */
113 export function recapToOffer(txs: Transaction[], today: string): string | null {
114   const months = recapMonths(txs);
115   const current = today.slice(0, 7);
116   const prev = previousMonth(current);
117   if (Number(today.slice(8, 10)) <= 7 && months.includes(prev)) return prev;
118   const thisMonth = txs.filter((t) => t.status === 'confirmed' &&
    bkkDayKey(t.occurredAt).slice(0, 7) === current).length;
119   return thisMonth >= 3 ? current : null;
120 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recap.ts:108

บรรทัด	ความหมาย
108–112	คอมเมนต์: สัปดาห์แรกของเดือนใหม่ เสนอเดือนที่แล้ว (ถ้ามีอะไร) นอกนั้นเสนอเดือนนี้เมื่อมีรายการพอ ไม่พอเล่าเรื่อง → null
114–116	เดือนที่มีข้อมูล เดือนนี้ และเดือนก่อน
117	วันที่ 1–7 และเดือนก่อนมีข้อมูล → เดือนก่อน
118	นับรายการยืนยันของเดือนนี้
119	อย่างน้อย 3 รายการ → เดือนนี้ ไม่จั้น null

ส่วนที่ 5: ข้อมูลปฏิทิน (บรรทัด 122–145)

src/domain/recap.ts

บรรทัด 122–145

```

122 // -----
123 // Spending calendar (a month as a heatmap)
124 // -----
125
126 export interface CalendarDay {
127   day: string;
128   expenseSatang: number;
129   incomeSatang: number;
130   count: number;
131   /** 0 = no spending; 1..4 = compared with the month's usual spending day (under half, under 1x,
    under 2x, 2x or more). */
132   level: 0 | 1 | 2 | 3 | 4;
133   isToday: boolean;
134   /** After today: nothing to show yet. */
135   future: boolean;
136 }
137
138 export interface SpendCalendar {
139   month: string;
140   label: string;
141   /** Weeks from Sunday to Saturday; null = a day of another month. */
142   weeks: (CalendarDay | null)[][];
143   /** The usual spending day this month (average of days with spending). */
144   usualSatang: number;

```

145 }

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recap.ts:122

บรรทัด ความหมาย

126–136 **CalendarDay** 1 ช่อง: วันที่, รายจ่าย, รายรับ, จำนวนรายการ, ระดับ 0–4 (0 = ไม่มีรายจ่าย 1–4 = เกือบวันปกติ: ต่ำกว่าครึ่ง, ต่ำกว่า 1 เท่า, ต่ำกว่า 2 เท่า, 2 เท่าขึ้นไป), เป็นวันนี้ใหม่, เป็นอนาคตใหม่

138–145 **SpendCalendar**: เดือน, ชื่อเดือน, สลิปดาห์ (อาทิตยสังเขป ช่องที่เป็นเดือนอื่น = null), วันปกติ (ค่าเฉลี่ยของวันที่มีรายจ่าย)

ส่วนที่ 6: spendCalendar – สร้างตารางเดือน (บรรทัด 147–179)

src/domain/recap.ts

บรรทัด 147–179

```

147 export function spendCalendar(txs: Transaction[], month: string, today: string): SpendCalendar {
148   const total = daysInMonth(month);
149   const per = new Map<string, { e: number; i: number; n: number }>();
150   for (const t of txs) {
151     if (t.status !== 'confirmed') continue;
152     const d = bkkDayKey(t.occurredAt);
153     if (d.slice(0, 7) !== month) continue;
154     const cur = per.get(d) ?? { e: 0, i: 0, n: 0 };
155     if (t.kind === 'expense') cur.e += t.amountSatang;
156     else cur.i += t.amountSatang;
157     cur.n += 1;
158     per.set(d, cur);
159   }
160   const spending = [...per.values()].filter((v) => v.e > 0);
161   const usual = spending.length ? spending.reduce((s, v) => s + v.e, 0) / spending.length : 0;
162   const level = (e: number): CalendarDay['level'] => {
163     if (e <= 0 || usual <= 0) return 0;
164     const r = e / usual;
165     return r < 0.5 ? 1 : r < 1 ? 2 : r < 2 ? 3 : 4;
166   };
167   const [y, m] = month.split('-').map(Number);
168   const firstWeekday = new Date(Date.UTC(y, m - 1, 1)).getUTCDay();
169   const cells: (CalendarDay | null)[] = Array.from({ length: firstWeekday }, () => null);
170   for (let d = 1; d <= total; d++) {
171     const day = `${month}-${String(d).padStart(2, '0')}`;
172     const v = per.get(day) ?? { e: 0, i: 0, n: 0 };
173     cells.push({ day, expenseSatang: v.e, incomeSatang: v.i, count: v.n, level: level(v.e),
isToday: day === today, future: day > today });
174   }
175   while (cells.length % 7) cells.push(null);
176   const weeks: (CalendarDay | null)[][] = [];
177   for (let i = 0; i < cells.length; i += 7) weeks.push(cells.slice(i, i + 7));
178   return { month, label: formatThaiMonth(month), weeks, usualSatang: Math.round(usual) };
179 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recap.ts:147

บรรทัด ความหมาย

147–148 รับรายการ เดือน วันนี้ · จำนวนวันของเดือน

บรรทัด	ความหมาย
149	Map วัน $\rightarrow$ { e: รายจ่าย, i: รายรับ, n: จำนวน }
150–159	วนรายการ: ข้ามที่ไม่ขึ้นยัน, ข้ามที่ไม่ใช่เดือนนี้, บวกยอดตามประเภท นับจำนวน เก็บกลับ
160	วันที่มีรายจ่าย
161	วันปกติ = รวมรายจ่ายของวันเหล่านั้น $\div$ จำนวนวัน (ไม่มีเลย = 0)
162–166	ฟังก์ชันระดับ: ไม่มีรายจ่าย หรือวันปกติเป็น 0 $\rightarrow$ 0 $\cdot$ อัตราส่วน $r = \text{รายจ่าย} \div \text{วันปกติ}$ $\cdot r < 0.5 \rightarrow 1, r < 1 \rightarrow 2, r < 2 \rightarrow 3$, นอกนั้น 4
167–168	วันแรกของเดือนตรงกับวันอะไร (0 = อาทิตย์) 1 ต.ค. 2569 = 4 (พฤหัสบดี)
169	เติมช่องว่าง (null) ข้างหน้าเท่ากับจำนวนนั้น (<code>Array.from({ length: 4 }, () => null)</code>)
170–174	วนวันที่ 1 ถึงวันสุดท้าย: สร้างข้อความวัน "2026-10-01" ถึงยอด (ไม่มี = 0) แล้วใส่ช่องพร้อมระดับ วันที่ใหม่ อนาคต ไหม
175	เติม nullท้ายจนครบสัปดาห์ (จำนวนช่องหาร 7 ลงตัว)
176–177	หั่นทีละ 7 ช่องเป็นสัปดาห์
178	คำนวณ วันปกติปิดเป็นสตางค์จำนวนเต็ม (34,924 สตางค์ = ฿349.24)

อ่านโค้ด: หน้าสรุปเดือน app/recap.tsx

365 บรรทัด ตัวหน้าจอลองเรื่องเล่า 7 หน้า: ข้อมูลมาจาก monthRecap ส่วนไฟล์นี้จัดการภาพ แอนิเมชัน และการ

และ

ส่วนที่ 1: import ค่าคงที่ และตัวช่วย (บรรทัด 1-58)

src/app/recap.tsx

บรรทัด 1-58

```

1  /**
2   * "สรุปเดือน": the month as a story of full screens (like a year-in-review),
3   * each on its own colour, moving on by itself every few seconds. Tap the right
4   * side for the next screen, the left side to go back. Rules: domain/recap.ts.
5   */
6  import { LinearGradient } from 'expo-linear-gradient';
7  import { useLocalSearchParams } from 'expo-router';
8  import { useCallback, useEffect, useMemo, useRef, useState, type ReactNode } from 'react';
9  import { Animated, Easing, Pressable, View } from 'react-native';
10 import { SafeAreaView } from 'react-native-safe-area-context';
11 import { useApp, useMoney } from '../data/AppProvider';
12 import { BUDDY_NAME } from '../domain/buddy';
13 import { formatThaiDay } from '../domain/dates';
14 import { formatBaht } from '../domain/money';
15 import { monthRecap } from '../domain/recap';
16 import { useAchievements } from '../services/useAchievements';
17 import { treeHealth, MoneyTree } from '../ui/art';
18 import { Buddy } from '../ui/Buddy';
19 import { Button, Ionicons, Row, T } from '../ui/components';
20 import { Aurora, GrowBar, Reveal, Sparkles, useCelebrate } from '../ui/effects';
21 import { useCountUp, useNative, useReduceMotion } from '../ui/motion';
22 import { goBack } from '../ui/nav';
23 import { fonts, space, useTheme } from '../ui/theme';
24
25 const SLIDE_MS = 6500;
26 const INK = '#F4F1E6';
27 const SOFT = '#C9D8CF';
28
29 /** Colours of the story screens in between (the first and last use the theme's own). */
30 const GRADIENTS: [string, string, string][] = [
31   ['#6B4A0E', '#3B2A0B', '#1D1405'],
32   ['#0F3D4A', '#0A2A33', '#061A20'],
33   ['#4A1F3D', '#2E1327', '#1A0A16'],
34   ['#123A6B', '#0B2547', '#061528'],
35   ['#1F5A3A', '#123A27', '#08201A'],
36 ];
37
38 /** A baht amount that counts up when the screen appears. */
39 function BigBaht({ satang, color }: { satang: number; color?: string }) {
40   const theme = useTheme();
41   const reduce = useReduceMotion();
42   const baht = useCountUp(Math.round(satang / 100), !reduce, 1200);
43   return (
44     <T v="display" color={color ?? theme.heroAccent} style={{ fontFamily: fonts.serifBold }}>
45       {formatBaht(baht * 100, { decimals: false })}
46     </T>

```

```

47   );
48 }
49
50 function Line({ children, index = 0, big }: { children: ReactNode; index?: number; big?: boolean }) {
51   return (
52     <Reveal index={index} from={16}>
53       <T v={big ? 'h2' : 'body'} color={big ? INK : SOFT} center>
54         {children}
55       </T>
56     </Reveal>
57   );
58 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/recap.tsx:1

บรรทัด	ความหมาย
1–5	คอมเมนต์: เดือนเป็นเรื่องเล่าเต็มจอแบบ "year-in-review" แต่ละหน้ามีสีของตัวเอง ไปต่อเองทุกไม่กี่วินาที และขวา = ถัดไป และซ้าย = ย้อน คติกาอยู่ใน recap.ts
6–23	import: พื้นหลัง, อ่านพารามิเตอร์ URL, hook, ภาพเคลื่อนไหว, กันรอยบาก, ข้อมูลกลาง, ชื่อน้องกล้า, วันไทย, เงิน, monthRecap, เหรียญ, ต้นไม้, น้องกล้า, ชิ้นส่วน, เอฟเฟกต์, ตัวนับ, ย้อนกลับ, ธีม
25	แต่ละหน้าอยู่ 6.5 วินาที
26–27	สีตัวหนังสือหลักและรอง (สีอ่อนบนพื้นเข้มทุกหน้า)
29–36	สีไล่ 5 ชุดของหน้ากลาง ๆ (ทอง ฟ้าน้ำทะเล ม่วงแดง น้ำเงิน เขียว) ส่วนหน้าแรกและหน้าสุดท้ายใช้สีของธีม
38–48	BigBaht: ตัวเลขบาทใหญ่ที่นับขึ้นใน 1.2 วินาที (ลดการเคลื่อนไหว = ขึ้นทันที) ฟอนต์ serif ตัวหนา
50–58	Line: ข้อความ 1 บรรทัดที่ค่อย ๆ โผล่ตามลำดับ (index) แบบ big = หัวข้อใหญ่สีเข้ม ไม่ขึ้นตัวปกติสีรอง จัดกลาง

ส่วนที่ 2: เตรียมข้อมูล (บรรทัด 60–80)

src/app/recap.tsx

บรรทัด 60–80

```

60 export default function Recap() {
61   const params = useLocalSearchParams<{ month?: string }>();
62   const theme = useTheme();
63   const { txs, today, profile } = useApp();
64   const { runway } = useMoney();
65   const { streak, badges } = useAchievements();
66   const celebrate = useCelebrate();
67   const reduce = useReduceMotion();
68   const month = params.month && /^d{4}-d{2}$/.test(params.month) ? params.month :
today.slice(0, 7);
69   const r = useMemo(() => monthRecap(txs, month, today), [txs, month, today]);
70   const [index, setIndex] = useState(0);
71   const [round, setRound] = useState(0);
72   const [progress] = useState(() => new Animated.Value(0));
73   const celebrated = useRef(false);
74
75   const restart = () => {
76     setIndex(0);
77     setRound((n) => n + 1);
78   };

```

```
79 const top = r.top[0];
80 const change = r.changePct;
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/recap.tsx:60

บรรทัด	ความหมาย
61	อ่านพารามิเตอร์ month จาก URL (/recap?month=2026-10)
63–67	รายการ วันที่ โปรไฟล์ · Runway (FR-6) · สถิติจุดต่อเนื่องและเหรียญ · พลุ่ · ลดการเคลื่อนไหว
68	ตรวจพารามิเตอร์: ต้องเป็นรูปแบบ "4 หลัก-2 หลัก" (regular expression) ไม่จำเป็นต้องใช้เดือนนี้ — กัน URL แปลก ๆ
69	r = monthRecap(...) คำนวณครั้งเดียว (useMemo)
70–73	หน้าที่แสดง (0–6), รอบการเล่น (เพิ่มเมื่อกดดูอีกครั้ง), ค่าแถบความคืบหน้า, จุดพลุแล้วหรือยัง
75–78	restart: กลับหน้าแรก และเพิ่มรอบ (ทำให้ทุกกองอย่างเล่นใหม่)
79–80	หมวดอันดับ 1 และเปอร์เซ็นต์เปลี่ยน (ตั้งชื่อสั้นไว้ใช้ข้างล่าง)

ส่วนที่ 3: หน้าที่ 1–4 (บรรทัด 81–179)

src/app/recap.tsx

บรรทัด 81–179

```
81 const slides: { key: string; body: ReactNode }[] = [
82   {
83     key: 'intro',
84     body: (
85       <View style={{ alignItems: 'center', gap: space.md }}>
86         <Reveal zoom>
87           <Buddy mood="cheer" size={150} onDark />
88         </Reveal>
89         <Line index={1}>สรุปเดือน</Line>
90         <Reveal index={2} zoom>
91           <T v="display" color={theme.heroAccent} center style={{ fontFamily: fonts.serifBold
92         }}>
93             {r.label}
94           </T>
95         </Reveal>
96         <Line index={3}>{'ของ${profile?.displayName ? `คุณ${profile.displayName}` :
97         'คุณ'}${r.partial ? ` · ${r.days} วันแรก` : ``}</Line>
98         <Line index={5}>และด้านขวาเพื่อไปต่อ</Line>
99       </View>
100     ),
101   },
102   {
103     key: 'spent',
104     body: (
105       <View style={{ alignItems: 'center', gap: space.md }}>
106         <Line>{r.partial ? 'เดือนนี้ใช้ไปแล้ว' : 'ทั้งเดือนใช้ไป'}</Line>
107         <BigBaht satang={r.expenseSatang} />
108         <Line index={2}>{'จาก ${r.count} รายการ · เฉลี่ยวันละ ${formatBaht(r.avgPerDaySatang, {
109         decimals: false })}'</Line>
110         <Reveal index={4}>
111           <Row gap={space.sm} style={{ marginTop: space.lg, backgroundColor:
112           'rgba(244,241,230,0.10)', borderRadius: 999, paddingHorizontal: space.lg, paddingVertical:
113           space.sm }}>
```

```

109      <Icons name="arrow-down-circle" size={20} color="#7FD1A8" />
110      <T v="body" color={INK}>
111        เงินเข้า {formatBaht(r.incomeSatang, { decimals: false })}
112      </T>
113    </Row>
114  </Reveal>
115 </View>
116 ),
117 },
118 {
119   key: 'where',
120   body: top ? (
121     <View style={{ alignItems: 'center', gap: space.md, alignSelf: 'stretch' }}>
122       <Line>เงินไปให้คนมากที่สุด</Line>
123       <Reveal index={1} zoom>
124         <T v="display" center style={{ fontSize: 72, lineHeight: 96 }}>
125           {top.glyph}
126         </T>
127       </Reveal>
128       <Line index={2} big>
129         {top.label}
130       </Line>
131       <Line index={3}>`$${formatBaht(top.amountSatang, { decimals: false })} ·
132   ${Math.round(top.share * 100)}% ของรายจ่าย`</Line>
133       <View style={{ alignSelf: 'stretch', gap: space.md, marginTop: space.lg }}>
134         {r.top.map((c, i) => (
135           <Reveal key={c.key} index={4 + i}>
136             <View style={{ gap: 6 }}>
137               <Row justify="space-between">
138                 <T v="small" color={INK}>`$${c.glyph} ${c.label}`</T>
139                 <T v="small" color={SOFT}>{formatBaht(c.amountSatang, { decimals: false })}
140               </Row>
141               <GrowBar value={c.share} color={i === 0 ? theme.heroAccent : '#7FD1A8'}
142   track="rgba(244,241,230,0.12)" height={10} delay={500 + i * 150} />
143             </View>
144           </Reveal>
145         ))}
146       </View>
147     </View>
148   ) : (
149     <View style={{ alignItems: 'center', gap: space.md }}>
150       <Line big>{r.partial ? 'เดือนนี้ยังไม่มีรายจ่าย' : 'เดือนนี้ไม่มีรายจ่ายเลย'}</Line>
151       <Line index={1}>ลองจดยอดรายการหรือสแกนสลิป แล้วสรุปจะมีเรื่องเล่าเยอะขึ้น</Line>
152     </View>
153   ),
154 },
155 {
156   key: 'days',
157   body: (
158     <View style={{ alignItems: 'center', gap: space.md }}>
159       {r.biggestDay ? (
160         <
161           <Line>วันที่ใช้เยอะที่สุด</Line>
162           <Line index={1} big>
163             {formatThaiDay(r.biggestDay.day)}
164           </Line>
165           <BigBaht satang={r.biggestDay.amountSatang} color="#F29E9E" />
166         </>
167       ) : null}
168       <Reveal index={3} zoom>

```

```

167         <View style={{ alignItems: 'center', marginTop: space.xl, gap: space.xs }}>
168             <T v="display" color="#7FD1A8" style={{ fontFamily: fonts.serifBold }}>
169                 {r.noSpendDays}
170             </T>
171             <T v="body" color={INK} center>
172                 วันที่ไม่ใช้เงินเลย
173             </T>
174         </View>
175     </Reveal>
176     {r.favorite ? <Line index={5}>{`จ่ายบ่อยที่สุด: ${r.favorite.title} (${r.favorite.times}
    ครั้ง)`}</Line> : null}
177 </View>
178 ),
179 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/recap.tsx:81

บรรทัด	ความหมาย
81	slides = รายการหน้า แต่ละหน้ามี key และ body (JSX ของหน้านั้น)
82–99	หน้า intro: บ๊องกลัดใจตัวใหญ่ 150, "สรุปเดือน", ชื่อเดือนตัวใหญ่สีทอง, "ของคุณ[ชื่อ] · 9 วันแรก" (ต่อท้ายเฉพาะเดือนที่ยังไม่จบ), "แตะด้านขวาเพื่อไปต่อ"
100–117	หน้า spent: "เดือนนี้ใช้ไปแล้ว" (หรือ "ทั้งเดือนใช้ไป"), BigBaht ของรายจ่าย, "จาก 30 รายการ · เฉลี่ยวันละ ฿566", แคลปซูล "เงินเข้า ฿10,000" ไอคอนเขียว
118–145	หน้า where (ถ้ามีหมวด): "เงินไปที่ไหนมากที่สุด", ไอคอนหมวดใหญ่ 72, ชื่อหมวด, "฿3,500 · 69% ของรายจ่าย" แล้วแถบ 3 หมวด แถบแรกสีทอง ที่เหลือเขียว ความยาว = ส่วนแบ่ง
146–151	ถ้าไม่มีรายจ่าย: "เดือนนี้ยังไม่มีรายจ่าย" และชวนจดหรือสแกน
153–165	หน้า days: ถ้ามีวันใช้มากที่สุด "วันที่ใช้เยอะที่สุด" วันที่ และยอดสีชมพูแดง
166–175	ตัวเลขใหญ่สีเขียว = วันที่ไม่ใช้เงินเลย
176	ถ้ามีผู้รับที่จ่ายบ่อย: "จ่ายบ่อยที่สุด: คาแฟยามเช้า (7 ครั้ง)"

ส่วนที่ 4: หน้า 5–7 (บรรทัด 180–279)

src/app/recap.tsx

บรรทัด 180–279

```

180     {
181       key: 'habits',
182       body: (
183         <View style={{ alignItems: 'center', gap: space.lg, alignSelf: 'stretch' }}>
184           <Line>เดือนนี้จดด้วยวิธีไหน</Line>
185           <Row justify="space-around" style={{ alignSelf: 'stretch' }}>
186             {[
187               { icon: 'receipt' as const, n: r.from.slips, label: 'สลิป' },
188               { icon: 'mic' as const, n: r.from.voice, label: 'พูดจด' },
189               { icon: 'create' as const, n: r.from.typed, label: 'พิมพ์เอง' },
190             ]}.map((h, i) => (
191               <Reveal key={h.label} index={1 + i} zoom>
192                 <View style={{ alignItems: 'center', gap: space.xs, width: 92 }}>
193                   <View style={{ width: 64, height: 64, borderRadius: 32, backgroundColor:
'rgba(244,241,230,0.10)', alignItems: 'center', justifyContent: 'center' }}>

```

```

194         <Icons name={h.icon} size={30} color={theme.heroAccent} />
195     </View>
196     <Text v="h1" color={INK}>
197         {h.n}
198     </Text>
199     <Text v="small" color={SOFT}>
200         {h.label}
201     </Text>
202 </View>
203 </Reveal>
204 )})
205 </Row>
206 <Reveal index={5}>
207     <Row gap={space.sm} style={{ backgroundColor: 'rgba(244,241,230,0.10)', borderRadius:
999, paddingHorizontal: space.lg, paddingVertical: space.sm }}>
208         <Icons name="flame" size={20} color="#F29E4C" />
209         <Text v="body" color={INK}>
210             จดต่อเนื่องสูงสุด {streak.best} วัน
211         </Text>
212     </Row>
213 </Reveal>
214 </View>
215 ),
216 },
217 {
218     key: 'compare',
219     body: (
220         <View style={{ alignItems: 'center', gap: space.md }}>
221             <Text>`เทียบกับ${r.prevLabel}${r.partial ? ` (${r.days} วันแรกเท่านั้น)` : ''}`</Text>
222             {change === null ? (
223                 <>
224                     <Text index={1} big>
225                         เดือนแรกของการจด
226                     </Text>
227                     <Text index={2}>เริ่มต้นได้ดีมาก เดือนหน้าจะได้เห็นว่าเปลี่ยนไปแค่ไหน</Text>
228                 </>
229             ) : (
230                 <>
231                     <Reveal index={1} zoom>
232                         <Row gap={space.sm}>
233                             <Icons name={change <= 0 ? 'trending-down' : 'trending-up'} size={44} color=
{change <= 0 ? '#7FD1A8' : '#F2B36B'} />
234                             <Text v="display" color={change <= 0 ? '#7FD1A8' : '#F2B36B'} style={{ fontFamily:
fonts.serifBold }}>
235                                 `${change > 0 ? '+' : change < 0 ? '-' : ''}${Math.abs(change)}%`
236                             </Text>
237                         </Row>
238                     </Reveal>
239                     <Text index={2} big>
240                         {change < 0 ? 'ใช้น้อยลง เก่งมาก!' : change === 0 ? 'เท่าเดิมพอดี' : 'ใช้มากขึ้นนิดหน่อย'}
241                     </Text>
242                     <Text index={3}>
243                         {change <= 0
244                             ? `${BUDDY_NAME}ภูมิใจมาก รักษาจังหวะนี้ไว้นะ`
245                             : top
246                             ? `ไม่เป็นไร เดือนหน้าลองดูหมวด${top.label}ก่อน ค่อย ๆ ปรับไปด้วยกัน`
247                             : 'ไม่เป็นไร ค่อย ๆ ปรับไปด้วยกัน'}
248                     </Text>
249                 </>
250             )}
251         </View>

```

```

252     ),
253   },
254   {
255     key: 'outro',
256     body: (
257       <View style={{ alignItems: 'center', gap: space.md }}>
258         <Reveal zoom>
259           <MoneyTree health={treeHealth(runway.status, runway.days)} size={150} trunk="#E8E1CC"
leaf={theme.heroAccent} bare={theme.heroBare} glow={theme.heroAccent} grow sway />
260         </Reveal>
261         <Line index={1} big>
262           {runway.days !== null && runway.status !== 'below_floor'
263             ? runway.capped
264             ? 'เงินพอใช้เกิน 1 ปี'
265             : `ตอนนี้เงินพอใช้อีก ${runway.days} วัน`
266             : 'เดือนหน้าเริ่มใหม่ไปด้วยกัน'}
267         </Line>
268         <Line index={2}>`เหรียญที่ได้แล้ว ${badges.filter((b) => b.earned).length} จาก
${badges.length}`</Line>
269         <Reveal index={4}>
270           <Row gap={space.sm} style={{ marginTop: space.lg }}>
271             <Button label="ดูอีกครั้ง" kind="onDark" icon="refresh" onPress={() => restart()} />
272             <Button label="เสร็จ" kind="gold" icon="checkmark" onPress={() => goBack()} />
273           </Row>
274         </Reveal>
275       </View>
276     ),
277   },
278 ];
279 const last = slides.length - 1;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/recap.tsx:180

บรรทัด	ความหมาย
180–205	หน้า habits: "เดือนนี้จัดด้วยวิธีไหน" วงกลม 3 วง (สลิป / พุดจต / พิมพ์เอง) แต่ละวงมีไอคอน ตัวเลขใหญ่ และชื่อ สร้างจากรายการ 3 ตัวด้วย map
206–213	แคปซูลไฟ "จดต่อเนื่องสูงสุด 51 วัน"
217–221	หน้า compare: "เทียบกับกันยายน 2569 (9 วันแรกเท่านั้น)"
222–228	ไม่มีให้เทียบ (null): "เดือนแรกของการจด" และกำลังใจ
229–238	มี: ลูกศรลง/ขึ้น และตัวเลข "+7%" / "-12%" / "0%" สีเขียวถ้าไม่เพิ่ม สีส้มถ้าเพิ่ม
239–241	หัวข้อ: ลดลง "ใช้น้อยลง เก่งมาก!" / เท่ากัน "เท่าเดิมพอดี" / เพิ่ม "ใช้มากขึ้นนิดหน่อย"
242–248	ประโยคปลอบ/ชม: ไม่เพิ่ม → "น้องกล้าภูมิใจมาก..." · เพิ่มและมีหมวดอันดับ 1 → "ไม่เป็นไร เดือนหน้าลองดูหมวด...ก่อน" (ไม่ตำหนิ ชี้ทางแก้)
254–260	หน้า outro: ต้นไม้เงินตามสถานะ Runway ใหญ่ 150
261–267	"ตอนนี้เงินพอใช้อีก 15 วัน" (เกินปี → "เงินพอใช้เกิน 1 ปี", คำนวนไม่ได้/ต่ำกว่าเส้น → "เดือนหน้าเริ่มใหม่ไปด้วยกัน")
268	"เหรียญที่ได้แล้ว 13 จาก 13"
269–274	ปุ่ม "ดูอีกครั้ง" (restart) และ "เสร็จ" (ย้อนกลับ)

ส่วนที่ 5: การเปลี่ยนหน้า และการวาด (บรรทัด 281–365)

src/app/recap.tsx

บรรทัด 281–365

```

281  const go = useCallback(
282    (to: number) => {
283      if (to < 0) return;
284      if (to > last) {
285        goBack();
286        return;
287      }
288      setIndex(to);
289    },
290    [last],
291  );
292
293  // The bar at the top fills up; when it is full the next screen comes.
294  useEffect(() => {
295    progress.setValue(0);
296    if (index === last) {
297      progress.setValue(1);
298      return;
299    }
300    const anim = Animated.timing(progress, { toValue: 1, duration: SLIDE_MS, easing:
Easing.linear, useNativeDriver: useNative });
301    anim.start(({ finished }) => {
302      if (finished) setIndex((i) => Math.min(i + 1, last));
303    });
304    return () => anim.stop();
305  }, [index, last, progress, round]);
306
307  useEffect(() => {
308    if (index === last && !celebrated.current) {
309      celebrated.current = true;
310      celebrate();
311    }
312  }, [index, last, celebrate]);
313
314  const heroColors: [string, string, string] = [theme.heroTop, theme.hero, theme.heroDeep];
315  const colors = index === 0 || index === last ? heroColors : GRADIENTS[(index - 1) %
GRADIENTS.length];
316  const slide = slides[index];
317  return (
318    <View style={{ flex: 1, backgroundColor: colors[2] }}>
319      <LinearGradient colors={colors} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={{ flex:
1 }}>
320        <Aurora key={`aurora-${index}-${round}`} cycles={1} strength={0.4} seed={index * 7 + 3}
/>
321        {index === last ? <Sparkles key={`stars-${round}`} count={12} cycles={2} /> : null}
322      <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1 }}>
323        {/* progress, one segment per screen */}
324        <Row gap={4} style={{ paddingHorizontal: space.md, paddingTop: space.sm }}>
325          {slides.map((s, i) => (

```

```

326         <View key={s.key} style={{ flex: 1, height: 3, borderRadius: 2, backgroundColor:
'rgba(244,241,230,0.25)', overflow: 'hidden' }}>
327             <Animated.View
328                 style={{
329                     height: 3,
330                     backgroundColor: INK,
331                     transformOrigin: 'left',
332                     transform: [{ scaleX: i < index ? 1 : i > index ? 0 : reduce ? 1 : progress
}],
333                 }}
334             />
335         </View>
336     )})
337 </Row>
338 <Row justify="space-between" style={{ paddingHorizontal: space.md, paddingTop: space.sm
}}>
339     <T v="small" color={S0FT}>
340         {'สรุปเดือน · ${r.label}'}
341     </T>
342     <Pressable onPress={() => goBack()} accessibilityRole="button" accessibilityLabel="ปิด
สรุปเดือน" hitSlop={10}>
343         <Ionicons name="close" size={26} color={INK} />
344     </Pressable>
345 </Row>
346 <View style={{ flex: 1 }}>
347     /* Tap zones: left goes back, right goes on (behind the content, so its buttons
still work). */
348     <Row style={{ position: 'absolute', left: 0, right: 0, top: 0, bottom: 0 }}
align="stretch">
349         <Pressable style={{ flex: 3 }} onPress={() => go(index - 1)}
accessibilityRole="button" accessibilityLabel="ย้อนกลับ" />
350         <Pressable style={{ flex: 7 }} onPress={() => go(index + 1)}
accessibilityRole="button" accessibilityLabel="ถัดไป" />
351     </Row>
352     <View
353         key={`${slide.key}-${round}`}
354         // Taps go through to the tap zones, except on the last screen's buttons.
355         pointerEvents={index === last ? 'box-none' : 'none'}
356         style={{ flex: 1, justifyContent: 'center', paddingHorizontal: space.xl }}
357     >
358         {slide.body}
359     </View>
360 </View>
361 </SafeAreaView>
362 </LinearGradient>
363 </View>
364 );
365 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/recap.tsx:281

บรรทัด	ความหมาย
281–291	go(to): น้อยกว่า 0 → ไม่ทำอะไร · เกินหน้าสุดท้าย → ปิดหน้า · นอกนั้นไปหน้านั้น
293–305	ตัวจับเวลา: ทุกครั้งที่เปลี่ยนหน้า ตั้งแถบเป็น 0 ถ้าเป็นหน้าสุดท้ายให้เต็มเลยและหยุด (ไม่ปิดเอง) ไม่ขึ้นเล่นแถบ 0→1 ใน 6.5 วินาทีแบบความเร็วคงที่ เต็มแล้วไปหน้าถัดไป และหยุดแถบเมื่อเปลี่ยนหน้าก่อนเต็ม
307–312	ถึงหน้าสุดท้ายครั้งแรก → จุดพลุครั้งเดียว

บรรทัด	ความหมาย
314–316	สีของหน้า: แรกและสุดท้ายใช้สีส้ม หน้ากลางวนใช้ชุดสี 5 ชุด · หยิบหน้าปัจจุบัน
318–321	พื้นไล่สีเต็มจอ ออโรรา (เปลี่ยนลายทุกหน้า) และดาวในหน้าสุดท้าย
323–337	แถบความคืบหน้า 7 ช่อง: ช่องที่ผ่านแล้วเต็ม ช่องที่ยังไม่ถึงว่าง ช่องปัจจุบันขยายตามค่าแถบ (scaleX จากซ้าย) ลดการเคลื่อนไหว = เพิ่มทันที
338–345	แถบบน: "สรุปเดือน · ตุลาคม 2569" และปุ่ม × "ปิดสรุปเดือน"
347–351	พื้นที่แตะ 2 ส่วน วางไว้ข้างหลังเนื้อหา: ซ้าย 3 ส่วน "ย้อนกลับ" ขวา 7 ส่วน "ถัดไป"
352–359	เนื้อหาของหน้า key เปลี่ยนตามหน้าและรอบ (จึงเล่นแอนิเมชันใหม่ทุกครั้ง) pointerEvents="none" = การแตะทะลุไปถึงพื้นที่แตะ ยกเว้นหน้าสุดท้ายที่ต้องกดปุ่มได้ (box-none)

i ทำไมแตะขวาได้พื้นที่มากกว่า

คนส่วนใหญ่ถือมือถือด้วยมือขวาและอยากไป "ต่อ" มากกว่า "ย้อน" แบบเดียวกับสตอรี่ใน Instagram จึงให้พื้นที่ขวา 70% ซ้าย 30% และใส่ป้ายอ่านจอ "ถัดไป"/"ย้อนกลับ" ให้คนที่ใช้โปรแกรมอ่านหน้าจอได้เหมือนปุ่ม (เกสต์ E2E ก็กดปุ่ม "ถัดไป" นี้)

อ่านโค้ด: ข้อมูลกลางที่ FR-2 ใช้ (AppProvider)

ตัวเลขบนหน้าหลักต้อง (1) ใช้ข้อมูลชุดล่าสุด (2) ไม่ปนบัญชี (3) รีเฟรชแล้วไม่หาย (4) เริ่มใหม่ตอนเที่ยงคืน — ทั้ง 4 ข้ออยู่ในไฟล์ `src/data/AppProvider.tsx`

ส่วนที่ 1: `useMoney` — ตัวเลขเงินที่ทุกหน้าใช้ร่วมกัน (บรรทัด 442–459)

`src/data/AppProvider.tsx`

บรรทัด 442–459

```
442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */
443 export function useMoney() {
444   const { profile, txs, carrySatang, goals, today } = useApp();
445   return useMemo(() => {
446     const opening = profile?.openingBalanceSatang ?? 0;
447     const floor = profile?.runwayFloorSatang ?? 50_000;
448     // Money from before the loaded window (over ~13 months ago) still counts.
449     const balance = computeBalance(opening + carrySatang, txs);
450     const average = averageDailyExpense(txs);
451     // Money in savings goals is set aside: it does not count as money to spend.
452     const reserved = reservedSatang(goals);
453     const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved,
      averageSatang: average.averageSatang });
454     const drafts = txs.filter((t) => t.status === 'draft');
455     return { balance, average, runway, drafts, reserved, floor };
456     // `today` makes the runway and averages start over at midnight.
457     // eslint-disable-next-line react-hooks/exhaustive-deps
458   }, [profile, txs, carrySatang, goals, today]);
459 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
442	คอมเมนต์: ตัวเลขเงินที่คำนวณแล้ว ใช้หลายหน้า (ยอดคงเหลือ FR-2, Runway FR-6)
443–444	hook <code>useMoney</code> ดึงโปรไฟล์ รายการ ยอดยกมา กระปุก วันนี้ จาก <code>useApp</code>
445	คำนวณใน <code>useMemo</code> (คำนวณใหม่เมื่อค่าในบรรทัด 458 เปลี่ยนเท่านั้น)
446	ยอดตั้งต้นจากโปรไฟล์ (ยังโหลดไม่เสร็จ = 0)
447	เส้นสำรอง ค่าเริ่มต้น ฿500
448–449	ยอดคงเหลือ = <code>computeBalance(ยอดตั้งต้น + ยอดยกมา, รายการ)</code> คอมเมนต์: เงินจากก่อนช่วงที่โหลด (เกิน ~13 เดือน) ยังนับ
450	ค่าเฉลี่ยรายจ่าย 7 วัน (เส้นประในกราฟ)
451–453	เงินในกระปุกถูกกั้นไว้ ไม่นับเป็นเงินที่ใช้ได้ → Runway (FR-6)
454	แบบร่างทั้งหมด (สำหรับการ์ดสลิปรอยืนยัน)
455	คืนทุกตัวเป็น object เดียว

บรรทัด	ความหมาย
--------	----------

456–458 dependency: โพรไฟล์ รายการ ยอดยกมา กระปุก และ **today** (ค่าเฉลี่ยและ Runway เริ่มใหม่ตอนเที่ยงคืน)

★ ทำไมต้องรวมไว้ที่เดียว

ถ้าหน้าหลักคำนวณยอดเอง เก็บเงินพอถึงคำนวณเอง หน้าโคชคำนวณเอง วันหนึ่งมีคนแก้สูตรที่เดียวแล้วลืมอีกสองที่ หน้าจอจะโชว์ยอดไม่ตรงกัน การรวมไว้ใน useMoney คือ "แหล่งความจริงแหล่งเดียว" (single source of truth)

ส่วนที่ 2: loadAll – โหลดข้อมูลและกั้นข้อมูลปน (บรรทัด 127–149)

src/data/AppProvider.tsx

บรรทัด 127–149

```

127 const loadAll = useCallback(async (r: Repo) => {
128   const gen = ++loadGen.current;
129   localChanges.current = new Map();
130   setLoadError(null);
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
r.listGoals().catch(() => [] as SavingsGoal[])]);
134     if (gen !== loadGen.current || repoRef.current !== r) return;
135     // A slip saved by the automatic scan during the load is not lost when the list arrives.
136     const changes = localChanges.current;
137     const fresh = list.txs.filter((t) => !changes.has(t.id));
138     const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
139     setProfile(p);
140     setTxs([...kept, ...fresh]);
141     setCarry(list.carrySatang);
142     setGoals(g);
143     setStatus('ready');
144   } catch (e) {
145     if (gen !== loadGen.current || repoRef.current !== r) return;
146     setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
147     setStatus('ready');
148   }
149 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
--------	----------

127 ฟังก์ชันโหลดทุกอย่างจาก Repo (useCallback = ใช้ฟังก์ชันเดิมทุกครั้งที่วาด)

128 เลขรอบการโหลด เพิ่มขึ้นทุกครั้ง (gen)

129 ล้างสมุดการเปลี่ยนแปลงในเครื่อง (เริ่มจดใหม่สำหรับรอบนี้)

130 ล้างข้อผิดพลาด

132–133 โหลด 3 อย่าง **พร้อมกัน** (Promise.all): โพรไฟล์ รายการ กระปุก — กระปุกโหลดไม่ได้ไม่เป็นไร (ได้ [] แทน) แอปส่วนอื่นยังทำงาน

134 ถ้ามีรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทิ้งผลนี้ (ข้อมูลอาจเป็นของอีกบัญชี)

บรรทัด	ความหมาย
135–138	รายการที่เปลี่ยนในเครื่องระหว่างโหนด (เช่น สลิปที่สแกนอัตโนมัติบันทึกตอนกำลังโหนด) ไม่หาย: ตัดตัวที่เปลี่ยนออกจากผลโหนด แล้วเอาตัวในเครื่องที่ยังไม่ถูกลบมาใส่
139–143	ตั้งค่า: โพรไฟล์, รายการ (ของในเครื่องก่อน แล้วของที่โหนด), ยอดยกมา, กระปุกลูก, สถานะพร้อม
144–147	ถ้าพัง (และยังเป็นรอบล่าสุด) เก็บข้อความผิดไว้แสดงการ์ดแดงบนหน้าหลัก แต่สถานะยังเป็นพร้อม (ข้อมูลเดิมยังใช้ได้)

ส่วนที่ 3: ยอดยกมาจากฐานข้อมูล (repo.ts บรรทัด 180–216)

src/data/repo.ts

บรรทัด 180–216

```

180  async listTransactions() {
181    // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182    const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183    const all: Transaction[] = [];
184    for (let from = 0; ; from += 1000) {
185      const { data, error } = await db
186        .from('transactions')
187        .select('*')
188        .gte('occurred_at', since)
189        .order('occurred_at', { ascending: false })
190        // A unique tiebreaker, so rows with the same time are never skipped or read twice
191        // across pages.
192        .order('id', { ascending: true })
193        .range(from, from + 999);
194      if (error) throw error;
195      all.push(...(data as TxRow[]).map(fromRow));
196      if (!data || data.length < 1000) break;
197    }
198    // Older confirmed money still counts in the balance: add it up (two columns only).
199    let carrySatang = 0;
200    for (let from = 0; ; from += 1000) {
201      const { data, error } = await db
202        .from('transactions')
203        .select('kind, amount_satang')
204        .eq('status', 'confirmed')
205        .lt('occurred_at', since)
206        .order('occurred_at', { ascending: false })
207        // A unique tiebreaker, so rows with the same time are never skipped or read twice
208        // across pages.
209        .order('id', { ascending: true })
210        .range(from, from + 999);
211      if (error) throw error;
212      for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
213        carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
214        Number(r.amount_satang);
215      }
216      if (!data || data.length < 1000) break;
217    }
218    return { txs: all, carrySatang };
219  },

```

► เปิดใน VS Code (แล้ว) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:180

บรรทัด	ความหมาย
180–182	โหลดรายการย้อนหลังประมาณ 13 เดือน (400 วัน) พอสำหรับช่วง "1 ปี" คอมเมนต์: แบ่งหน้าเพื่อไม่เก็บเพดานจำนวนแถวของ API
183	รายการที่โหลดได้ทั้งหมด
184	วนทีละหน้า หน้าละ 1,000 แถว (from = 0, 1000, 2000, ...)
185–192	ขอจากตาราง transactions ทุกคอลัมน์ เวลาตั้งแต่ since เรียงเวลาใหม่ไปเก่า แล้วเรียงตาม id เป็นตัวตัดสินเมื่อเวลาเท่ากัน (คอมเมนต์: กันแถวหายหรือซ้ำระหว่างหน้า) และขอช่วงแถว from ถึง from+999
193–195	พึง → โยน · ได้ → แปลงเป็นรายการของแอป · ได้น้อยกว่า 1,000 แถว = หน้าสุดท้าย หยุด
197–198	คอมเมนต์: เงินที่ยืนยันซึ่งต่ำกว่านั้นยังนับในยอด รวมเฉพาะ 2 คอลัมน์ (ประหยัดเน็ต)
199–208	ขอเฉพาะ kind และ amount_satang ของรายการที่ยืนยันและ ต่ำกว่า since แบ่งหน้าแบบเดียวกัน
210–212	ทบยอด: รายรับบวก รายจ่ายลบ → carrySatang
215	คืนทั้งรายการที่โหลด และยอดยกมา

≈ ยอดยกมา = หน้าแรกของสมุดเล่มใหม่

สมุดบัญชีเล่มเก่าเต็มแล้ว เราไม่ได้ถือไปทุกที แต่เขียน "ยอดยกมา" ไว้หน้าแรกของเล่มใหม่ แอปก็เหมือนกัน: รายการต่ำกว่า 400 วันไม่โหลดมาทีละรายการ (หนักและช้า) แต่รวมเป็นตัวเลขเดียว ยอดคงเหลือจึงถูกต้องแม้ใช้แอปมาหลายปี (บันทึกถูกพบและแก้ไขระหว่างตรวจ repository 1–3 ต.ค.)

ส่วนที่ 4: เกียงคืนเวลาไทย (บรรทัด 182–212)

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })

```

```

204     .catch(() => {});
205     const sub = AppState.addListener('change', (s) => {
206       if (s === 'active') check();
207     });
208     return () => {
209       clearTimeout(timer);
210       sub.remove();
211     };
212   }, []);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลาไทย ทั้งตอนเปิดแอปค้างไว้ (ตัวจับเวลา) และตอนกลับมาที่แอปเข้าวันถัดไป (AppState)
184–185	effect ทำงานครั้งเดียวตอนเริ่ม (dependency ว่าง) ตัวแปรเก็บตัวจับเวลา
186–187	check: วันไทยตอนนี้
188–193	ถ้าไม่ตรงกับวันที่จำไว้: จำวันใหม่, setToday (ทุก useMemo ที่มี today จะคำนวณใหม่), บอกว่าเป็นวันใหม่ (น้องกล้า ทักทาย), บันทึกวันล่าสุดลงเครื่อง
194–195	ตั้งเวลาปลุกครั้งถัดไป = จนถึงเที่ยงคืนไทยถัดไป + 1 วินาที (กันปลุกเร็วไปนิดแล้วยังเป็นวันเดิม)
197	เช็ครั้งแรกทันที
198–204	เปิดแอปครั้งแรกของวันที่ต่างจากครั้งก่อน (อ่านจากเครื่อง) → เป็นวันใหม่ แล้วบันทึกวันนี้ไว้
205–207	เมื่อแอปกลับมาอยู่หน้าจอ (active) เช็กรั้งอีกครั้ง — เพราะมือถือมักหยุดตัวจับเวลาตอนแอปอยู่เบื้องหลัง
208–211	เก็บกวาดเมื่อเลิกใช้: ยกเลิกเวลา และเลิกฟัง AppState

i ทำไมใช้ทั้งตัวจับเวลาและ AppState

ตัวจับเวลาอย่างเดียวไม่พอ: ถ้าผู้ใช้สลับไปแอปอื่นตอน 23:00 แล้วกลับมาตอน 08:00 ระบบมือถืออาจหยุดตัวจับเวลาไปแล้ว AppState บอกว่า "แอปกลับมาแล้ว" จึงเช็กรั้งใหม่ได้ทันที ส่วนบนเว็บและตอนเปิดค้าง ตัวจับเวลาทำงานตรงเวลา (E2E ทดสอบด้วยนาฬิกาจำลอง)

ส่วนที่ 5: ดึงลงเพื่อรีเฟรช (บรรทัด 359–366)

src/data/AppProvider.tsx

บรรทัด 359–366

```

359     loadError,
360     refreshing,
361     async refresh() {
362       if (!repoRef.current) return;
363       setRefreshing(true);
364       await loadAll(repoRef.current);
365       setRefreshing(false);
366     },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:359

บรรทัด	ความหมาย
359–360	ส่งข้อผิดพลาดโหลด และสถานะกำลังรีเฟรช ให้นับจอ
361–362	refresh: ถ้ายังไม่เข้าสู่ระบบ ไม่ทำอะไร
363–365	แสดงวงจรมุม โหลดทั้งหมดใหม่ (loadAll ส่วนที่ 2) แล้วซ่อนวงจรมุม — หน้าหลักและแท็บรายการส่งฟังก์ชันนี้ให้ RefreshControl

อ่านโค้ด: PeriodSummary — ภาพรวมย่อในหน้าสแกน

src/ui/PeriodSummary.tsx 49 บรรทัด ชิ้นส่วนของ FR-2 ที่ FR-4 ยืมไปใช้

src/ui/PeriodSummary.tsx

บรรทัด 1-49

```

1  /**
2   * Money in and out per period, with the exact span of each period so there is
3   * no doubt what counts: วันนี้ = 00:00-23:59 today (Bangkok time), 7 วัน = the
4   * last 7 full days ending today, and so on. Only confirmed items count.
5   */
6  import { useMemo } from 'react';
7  import { View } from 'react-native';
8  import { useApp } from '../data/AppProvider';
9  import { formatRangeSpan, RANGE_LABEL } from '../domain/dates';
10 import { formatBaht } from '../domain/money';
11 import { summarizeRange } from '../domain/summary';
12 import type { RangeKey, Transaction } from '../domain/types';
13 import { Card, Divider, Row, T } from './components';
14 import { fonts, space, useTheme } from './theme';
15
16 const DEFAULT_RANGES: RangeKey[] = ['today', '7d', '1m'];
17
18 export function PeriodSummary({ txs, ranges = DEFAULT_RANGES, title }: { txs: Transaction[];
19   ranges?: RangeKey[]; title?: string }) {
20   const theme = useTheme();
21   const { today } = useApp(); // periods move forward at 00:00
22   const rows = useMemo(
23     () => ranges.map((r) => ({ range: r, span: formatRangeSpan(r), sum: summarizeRange(txs, r)
24   })),
25     // eslint-disable-next-line react-hooks/exhaustive-deps
26     [txs, ranges, today],
27   );
28   return (
29     <Card style={{ gap: space.sm }}>
30       {title ? <T v="h3">{title}</T> : null}
31       {rows.map(({ range, span, sum }, i) => (
32         <View key={range} style={{ gap: space.sm }}>
33           {i > 0 ? <Divider /> : null}
34           <Row justify="space-between" align="flex-start" gap={space.md}>
35             <View style={{ flex: 1, gap: 2 }}>
36               <T v="body" style={{ fontFamily: fonts.sansSemi }}>{RANGE_LABEL[range]}</T>
37               <T v="micro">{span}</T>
38             </View>
39             <View style={{ alignItems: 'flex-end', gap: 2 }}>
40               <T v="small" color={theme.income}>เข้า {formatBaht(sum.incomeSatang, { decimals:
41 false }}}</T>
42               <T v="small" color={theme.expense}>ออก {formatBaht(sum.expenseSatang, { decimals:
43 false }}}</T>
44               <T v="micro" color={sum.netSatang >= 0 ? theme.good : theme.critical}>
45                 สุทธิ {formatBaht(sum.netSatang, { sign: true, decimals: false })}
46               </T>
47             </View>
48           </Row>
49         </View>
50       ))}
51     </Card>
52   );
53 }

```

```

45     </View>
46   )})
47 </Card>
48 );
49 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/PeriodSummary.tsx:1`

บรรทัด	ความหมาย
1–5	คอมเมนต์: เงินเข้า-ออกต่อช่วง พร้อมช่วงเวลาที่แน่นอน วันนี้ = 00:00–23:59 เวลาไทย, 7 วัน = 7 วันเต็มจบที่วันนี้ ฯลฯ นับเฉพาะที่ยืนยัน
6–14	import: memo, กล้อง, ข้อมูลกลาง, ข้อความช่วง และชื่อช่วง, เงิน, summarizeRange (ตัวเดียวกับหน้าหลัก), ชนิด, ชิ้นส่วน, ธีม
16	ช่วงที่แสดงเริ่มต้น: วันนี้ 7 วัน 1 เดือน
18	รับรายการ ช่วง และหัวข้อ (ไม่บังคับ)
19–20	สัปดาห์ และวันนี้ (ช่วงเลื่อนตามเที่ยงคืน)
21–25	แต่ละช่วงคำนวณ: ข้อความช่วง และผลสรุป (useMemo กับ today)
27–28	การ์ด และหัวข้อถ้ามี
29–32	วนแต่ละช่วง เส้นคั่นยกเว้นแถวแรก แถวซ้าย-ขวา
33–36	ซ้าย: ชื่อช่วงตัวหนา และข้อความช่วงจริง
37–42	ขวา: "เข้า ฿...", "ออก ฿..." และ "สุทธิ ..." สีเขียว/แดง

“ ประโยคเชื่อม FR

"หน้าสแกนสลิปของ FR-4 ใช้ PeriodSummary ของ FR-2 แสดงยอดวันนี้/7 วัน/1 เดือนหลังสแกนเสร็จ เพราะเรียก summarizeRange ตัวเดียวกัน ตัวเลขในหน้าสแกนจึงตรงกับหน้าหลักเสมอ (E2E ตรวจสอบข้อความ '00:00–23:59' และ 'ออก ฿120' หลังสแกน)"

การทดสอบ FR-2

ฉันพิสูจน์ยังงี้ว่าตัวเลขบนหน้าหลักถูกทุกตัว

157/157

unit test ทั้งโปรเจกต์ผ่าน (รันซ้ำ 9 ต.ค. 2569)

TC-03, 08–11, 69,

70

unit test ของ FR-2 โดยตรง

10+ ข้อ

E2E: เกียงคิ่น, ปฏิทิน, สรุปเดือน, ลดการเคลื่อนไหว, ยอดตามการเพิ่ม/แก้/ลบ

Unit test: วันไทยและช่วงเวลา (TC-03, TC-08)

src/domain/__tests__/domain.test.tsบรรทัด 35–36

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
```

[▶ เปิดใน VS Code \(เว็บ\)](#) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

ทุกเทสต์ใน domain.test.ts ใช้ "ตอนนี้" ตายตัว = วันอาทิตย์ 27 ก.ย. 2569 เวลา 14:00 น. (ไทย) = 07:00 UTC ผลจึงเหมือนเดิมไม่ว่ารันวันไหน

src/domain/__tests__/domain.test.tsบรรทัด 91–95

```
91 it('TC-03 groups by Bangkok day, not UTC day', () => {
92   // 23:30 Bangkok on 26 Sep is 16:30 UTC on 26 Sep; 00:30 Bangkok on 27 Sep is still 26 Sep in UTC.
93   expect(bkkDayKey('2026-09-26T17:30:00.000Z')).toBe('2026-09-27');
94   expect(bkkDayKey('2026-09-26T16:30:00.000Z')).toBe('2026-09-26');
95 });
```

[▶ เปิดใน VS Code \(เว็บ\)](#) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:91

บรรทัด	ความหมาย
91	TC-03 "จัดกลุ่มตามวันไทย ไม่ใช่วัน UTC"
92	คอมเมนต์: 23:30 ไทยของ 26 ก.ย. = 16:30 UTC วันที่ 26 ส่วน 00:30 ไทยของ 27 ก.ย. ยังเป็นวันที่ 26 ใน UTC
93	17:30 UTC วันที่ 26 → ต้องเป็นวันไทย 27 (เพราะ +7 ชม. = 00:30 วันที่ 27)
94	16:30 UTC วันที่ 26 → ยังเป็นวันไทย 26 (23:30)

src/domain/__tests__/domain.test.tsบรรทัด 127–132

```
127 it('TC-08 ranges mean the same days everywhere', () => {
128   expect(rangeDays('today', NOW)).toEqual({ from: '2026-09-27', to: '2026-09-27' });
129   expect(rangeDays('7d', NOW)).toEqual({ from: '2026-09-21', to: '2026-09-27' });
130   expect(rangeDays('1m', NOW).from).toBe('2026-08-29');
131   expect(rangeDays('1y', NOW).from).toBe('2025-09-28');
```

132 });

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:127

บรรทัด	ความหมาย
127	TC-08 "ช่วงหมายถึงวันเดียวกันทุกที"
128	วันนี้ = 27 ก.ย. ถึง 27 ก.ย.
129	7 วัน = 21–27 ก.ย. (ไม่ใช่ 20–27)
130	1 เดือน เริ่ม 29 ส.ค. (30 วันรวมวันนี้)
131	1 ปี เริ่ม 28 ก.ย. 2025 (2568) — 365 วันรวมวันนี้

Unit test: ภาพรวม (TC-09, TC-10, TC-11)

src/domain/__tests__/domain.test.ts

บรรทัด 135–163

```
135 describe('FR-2 overview', () => {
136   const txs = [
137     tx({ day: '2026-09-27', amount: 100 }),
138     tx({ day: '2026-09-25', amount: 50, categoryKey: 'transport' }),
139     tx({ day: '2026-09-01', amount: 9000, kind: 'income', categoryKey: 'allowance' }),
140     tx({ day: '2026-09-27', amount: 999, status: 'draft' }),
141   ];
142
143   it('TC-09 balance ignores drafts', () => {
144     expect(computeBalance(500_000, txs)).toBe(500_000 + 900_000 - 15_000);
145   });
146
147   it('TC-10 summarizes a range with categories', () => {
148     const s = summarizeRange(txs, '7d', NOW);
149     expect(s.expenseSatang).toBe(15_000);
150     expect(s.incomeSatang).toBe(0);
151     expect(s.byCategory.expense[0]).toMatchObject({ key: 'food', totalSatang: 10_000 });
152     expect(s.byCategory.expense[0].share).toBeCloseTo(2 / 3);
153     const month = summarizeRange(txs, '1m', NOW);
154     expect(month.incomeSatang).toBe(900_000);
155   });
156
157   it('TC-11 daily totals include empty days', () => {
158     const d = dailyTotals(txs, 7, NOW);
159     expect(d).toHaveLength(7);
160     expect(d[6]).toMatchObject({ day: '2026-09-27', expenseSatang: 10_000 });
161     expect(d[5].expenseSatang).toBe(0);
162   });
163 });
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:135

บรรทัด	ความหมาย
135–141	ข้อมูลทดสอบ 4 รายการ: อาหาร ฿100 (27 ก.ย.), เดินทาง ฿50 (25 ก.ย.), รายรับเงินจากที่บ้าน ฿9,000 (1 ก.ย.), แบบร่าง ฿999 (27 ก.ย.) — ฟังก์ชัน tx ช่วยสร้างรายการโดยใช้แค่ช่องที่ต่าง

บรรทัด	ความหมาย
143–145	TC-09: ยอดตั้งต้น 500,000 + 900,000 – 15,000 (แบบร่างไม่นับ)
147–152	TC-10: ช่วง 7 วัน: รายจ่าย 15,000, รายรับ 0, หมวดอันดับแรก = อาหาร 10,000 สตางค์, ส่วนแบ่งใกล้ 2/3 (toBeCloseTo = ใกล้เคียง เพราะทศนิยมไม่ลงตัว)
153–154	ช่วง 1 เดือน: รายรับ 900,000 (เงินวันที่ 1 ก.ย. อยู่ในช่วง 29 ส.ค.–27 ก.ย.)
157–161	TC-11: dailyTotals 7 วัน ต้องได้ 7 ช่วง, ช่วงสุดท้าย 27 ก.ย. = 10,000, ช่วงก่อนหน้า (26 ก.ย.) = 0

Unit test: สรุปเดือนและปฏิทิน (TC-69, TC-70)

src/domain/__tests__/recap.test.ts

บรรทัด 33–80

```

33  it('TC-69 sums the month, finds where the money went, the biggest day, habits, and compares
    fairly with last month', () => {
34      expect(formatThaiMonth('2026-09')).toBe('กันยายน 2569');
35      expect(previousMonth('2026-01')).toBe('2025-12');
36      expect(daysInMonth('2028-02')).toBe(29);
37
38      const txs = [
39          tx('2026-09-01', 50),
40          tx('2026-09-01', 60, { title: 'ข้าวมันไก่ ' }),
41          tx('2026-09-03', 400, { categoryKey: 'transport', title: 'BTS', source: 'slip' }),
42          tx('2026-09-10', 1200, { categoryKey: 'shopping', title: 'Shopee', note: VOICE_NOTE }),
43          tx('2026-09-12', 5000, { kind: 'income', categoryKey: 'allowance', title: 'แม่' }),
44          tx('2026-09-12', 999, { status: 'draft' }), // not confirmed: not counted
45          tx('2026-09-30', 80), // after today: not counted yet
46          // August, first 15 days: 1,000; later: not compared
47          tx('2026-08-05', 1000, { title: 'ค่าหอ' }),
48          tx('2026-08-20', 3000, { title: 'รองเท้า' }),
49      ];
50      const r = monthRecap(txs, '2026-09', '2026-09-15');
51      expect(r.label).toBe('กันยายน 2569');
52      expect(r.partial).toBe(true);
53      expect(r.days).toBe(15);
54      expect(r.expenseSatang).toBe(171_000);
55      expect(r.incomeSatang).toBe(500_000);
56      expect(r.count).toBe(5);
57      expect(r.top.map((t) => t.key)).toEqual(['shopping', 'transport', 'food']);
58      expect(r.top[0].share).toBeCloseTo(1200 / 1710, 5);
59      expect(r.biggestDay).toEqual({ day: '2026-09-10', amountSatang: 120_000 });
60      expect(r.noSpendDays).toBe(12); // 15 days, spending on the 1st, 3rd and 10th
61      expect(r.avgPerDaySatang).toBe(11_400);
62      expect(r.favorite).toEqual({ title: 'ข้าวมันไก่', times: 2 });
63      expect(r.from).toEqual({ slips: 1, voice: 1, typed: 3 });
64      // Same 15 days of August: ฿1,000 -> +71%.
65      expect(r.changePct).toBe(71);
66      expect(r.prevLabel).toBe('สิงหาคม 2569');
67
68      const aug = monthRecap(txs, '2026-08', '2026-09-15');
69      expect(aug.partial).toBe(false);
70      expect(aug.days).toBe(31);
71      expect(aug.expenseSatang).toBe(400_000);
72      expect(aug.changePct).toBeNull(); // nothing in July to compare with
73      expect(aug.favorite).toBeNull(); // paid only once each
74

```

```

75     expect(recapMonths(txs)).toEqual(['2026-09', '2026-08']);
76     const empty = monthRecap([], '2026-09', '2026-09-15');
77     expect(empty.top).toEqual([]);
78     expect(empty.biggestDay).toBeNull();
79     expect(empty.noSpendDays).toBe(15);
80 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/recap.test.ts:33

บรรทัด	ความหมาย
34–36	ตัวช่วยวันที่: ชื่อเดือนไทย, เดือนก่อนของมกราคม, กุมภาพันธ์ปีย่อริศสุรทิน 29 วัน
38–49	ข้อมูล: ข้าวมันไก่ 2 ครั้ง (ครั้งหนึ่งมีช่องว่างท้าย), BTS จากสลิป, Shopee จากเสียง, รายรับ ฿5,000, แบบร่าง (ไม่นับ), รายการ 30 ก.ย. (หลังวันนี้ ไม่นับ), สิงหาคม 2 รายการ (5 และ 20 ส.ค.)
50–56	สรุป ก.ย. ณ 15 ก.ย.: ชื่อ, ยังไม่จบ, 15 วัน, รายจ่าย ฿1,710, รายรับ ฿5,000, 5 รายการ
57–58	หมวดอันดับ: ซัปปิง, เดินทาง, อาหาร · ซัปปิงได้ $1,200 \div 1,710$
59–61	วันใช้มากที่สุด 10 ก.ย. $\$1,200 \cdot$ วันไม่ใช้เงิน 12 (15 วัน มีรายจ่าย 3 วัน) · เฉลี่ย $\$114/\text{วัน}$
62–63	จ่ายบ่อยสุด "ข้าวมันไก่" 2 ครั้ง (ช่องว่างท้ายไม่ทำให้นับแยก) · สลิป 1 เสียง 1 พิมพ์ 3
64–66	เทียบธุรกรรม: 15 วันแรกของ ส.ค. = $\$1,000$ (ไม่นับ $\$3,000$ วันที่ 20) → +71%
68–73	สรุป ส.ค.: จบแล้ว 31 วัน $\$4,000$ ไม่มีกรกฎาคมให้เทียบ (null) จ่ายชื่อละครั้งเดียว จึงไม่มี "จ่ายบ่อยสุด"
75–79	เดือนที่มีข้อมูล [ก.ย., ส.ค.] และเดือนว่าง: ไม่มีหมวด ไม่มีวันใช้มากที่สุด วันไม่ใช้เงิน 15

src/domain/__tests__/recap.test.ts

บรรทัด 84–111

```

84   it('TC-70 lays the month out Sunday to Saturday and shades each day against the usual spending
    day', () => {
85       const txs = [
86         tx('2026-09-01', 100), // Tuesday
87         tx('2026-09-02', 40),
88         tx('2026-09-03', 400),
89         tx('2026-09-04', 60),
90         tx('2026-09-04', 0.5, { kind: 'income' }),
91         tx('2026-09-05', 999, { status: 'draft' }),
92         tx('2026-10-01', 500),
93       ];
94       const cal = spendCalendar(txs, '2026-09', '2026-09-15');
95       expect(cal.label).toBe('กันยายน 2569');
96       // September 2026 starts on a Tuesday: Sunday and Monday are empty.
97       expect(cal.weeks[0].slice(0, 2)).toEqual([null, null]);
98       expect(cal.weeks.every((w) => w.length === 7)).toBe(true);
99       expect(cal.weeks.flat().filter(Boolean)).toHaveLength(30);
100      // Usual day = (100 + 40 + 400 + 60) / 4 = $150.
101      expect(cal.usualSatang).toBe(15_000);
102      const byDay = Object.fromEntries(cal.weeks.flat().filter((c) => c).map((c) => [c!.day, c!]));
103      expect(byDay['2026-09-01'].level).toBe(2); // 0.67x
104      expect(byDay['2026-09-02'].level).toBe(1); // 0.27x
105      expect(byDay['2026-09-03'].level).toBe(4); // 2.7x
106      expect(byDay['2026-09-04']).toMatchObject({ level: 1, count: 2, incomeSatang: 50 });
107      expect(byDay['2026-09-05'].level).toBe(0); // a draft does not count
108      expect(byDay['2026-09-15'].isToday).toBe(true);

```

```

109     expect(byDay['2026-09-16'].future).toBe(true);
110     expect(spendCalendar([], '2026-02', '2026-09-
111       15').weeks.flat().filter(Boolean)).toHaveLength(28);
112   });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/recap.test.ts:84

TC-70 ทดสอบปฏิทินตามตัวอย่างที่ 4 ในบทหลักการ: ก.ย. 2569 เริ่มวันอังคาร (ช่องอาทิตย์-จันทร์ว่าง), ทุกสัปดาห์ 7 ช่อง, มีวันจริง 30 ช่อง, วันปกติ ฿150, ระดับ 2/1/4/1, แบบร่างได้ระดับ 0, วันนี้และวันอนาคตถูกต้อง และ ก.พ. 2026 มี 28 ช่อง

E2E: ทดสอบเหมือนคนใช้จริง

e2e/web-auth.mjs

บรรทัด 374–392

```

374 // 10. A new day: at 00:00 Bangkok time "วันนี้" starts over by itself
375 {
376   const { ctx, page } = await freshPage('midnight');
377   await page.clock.install({ time: new Date('2026-09-28T16:59:40Z') }); // 23:59:40 on 28 Sep
   in Bangkok
378   await page.goto(APP);
379   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
380   await introGone(page);
381   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
382   const before = await visible(page, 'วันจันทร์ที่ 28 กันยายน 2569', 8000);
383   await page.clock.fastForward(30_000); // past midnight
384   await page.waitForTimeout(500);
385   const after = await visible(page, 'วันอังคารที่ 29 กันยายน 2569', 5000);
386   check('At midnight the date on the home screen moves to the new day by itself', before &&
   after);
387   check('The companion greets the new day with yesterday\'s spending', await visible(page, 'วัน
   ใหม่แล้ว!'));
388   const picked = await pickPeriod(page, 'วันนี้');
389   check('"วันนี้" now means the new day, 00:00–23:59', picked && (await visible(page, '29 ก.ย.
   2569 · 00:00–23:59', 8000)), picked ? '' : 'tab not selected');
390   await shot(page, '17-new-day');
391   await ctx.close();
392 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:374

บรรทัด	ความหมาย
374–377	เปิดหน้าใหม่ ติดตั้งนาฬิกาจำลอง 16:59:40 UTC = 23:59:40 ไทย วันที่ 28 ก.ย.
378–382	เปิดแอป เข้าโหมดทดลอง ต้องเห็น "วันจันทร์ที่ 28 กันยายน 2569"
383–386	เร่งเวลา 30 วินาที (ผ่านเที่ยงคืน) ต้องเห็น "วันอังคารที่ 29 กันยายน 2569" โดยไม่รีเฟรช
387	น้องกล่าวก "วันใหม่แล้ว!"
388–389	เลือกช่วง "วันนี้" ต้องเห็น "29 ก.ย. 2569 · 00:00–23:59"
390	ถ่ายภาพ 17-new-day

e2e/web-auth.mjs

บรรทัด 908–940

```

908 // Spending calendar on the transactions tab: each day shaded by spending; tap a day to see
    only it.
909 await page.getByRole('tab', { name: 'รายการ', exact: true }).first().click().catch(() =>
    page.goto(`${APP}transactions`));
910 const calendarShown = (await visible(page, 'มีเงินเข้า')) && (await visible(page, 'เทียบกับวันที่ใช้ตาม
    ปกติ'));
911 check('Transactions: the spending calendar of this month is shown', calendarShown);
912 const spentDay = page.getByRole('button', { name: /ใช้ไป ฿/ }).first();
913 const dayLabel = (await spentDay.getAttribute('aria-label')) ?? '';
914 await spentDay.click();
915 await page.waitForTimeout(600);
916 await shot(page, '27-calendar-day');
917 check('Tapping a day shows only that day (with its total)', (await visible(page, 'ดูทุกวัน')) &&
    dayLabel.length > 0, dayLabel);
918 await button(page, 'ดูทุกวัน').click();
919 check('"ดูทุกวัน" shows every day again', await gone(page, 'ดูทุกวัน'));
920 await page.goto(APP);
921 await introGone(page);
922 await visible(page, 'ยอดคงเหลือ', 8000);
923
924 // Month recap: a story of full screens that moves on by itself; tap right for the next one.
925 await button(page, 'ดูสรุปเดือน').click();
926 check('Recap: opens on the month\'s title screen', await visible(page, 'แตะด้านขวาเพื่อไปต่อ'));
927 // This month so far, or (on the 1st, when the recap is the month just ended) the whole
    month.
928 check('Recap: moves on to the next screen by itself', await visible(page, /เดือนนี้ใช้ไปแล้ว|ทั้งเดือนใช้
    ไป/, 9000));
929 await button(page, 'ถัดไป').click();
930 const where = await visible(page, 'เงินไปที่ไหนมากที่สุด');
931 await page.waitForTimeout(1600);
932 await shot(page, '25-recap-where');
933 check('Recap: where the money went, biggest first', where);
934 for (let i = 0; i < 4; i++) await button(page, 'ถัดไป').click();
935 const outro = await visible(page, 'ดูอีกครั้ง');
936 await page.waitForTimeout(1200);
937 await shot(page, '26-recap-end');
938 check('Recap: ends with the money tree and the badges', outro && (await visible(page, 'เหรียญที่
    ได้แล้ว')));
939 await button(page, 'เสร็จ').click();
940 check('Recap: "เสร็จ" returns home', await visible(page, 'ยอดคงเหลือ', 8000));

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:908

บรรทัด	ความหมาย
908–911	ไปแท็บรายการ ต้องเห็นปฏิทิน (คำว่า "มีเงินเข้า" และ "เทียบกับวันที่ใช้ตามปกติ")
912–917	หาปุ่มวันที่มีคำว่า "ใช้ไป ฿" ตัวแรก แตะ ถ่ายภาพ ต้องเห็น "ดูทุกวัน"
918–919	กด "ดูทุกวัน" ปุ่มต้องหายไป
924–926	กลับหน้าหลัก กด "ดูสรุปเดือน" ต้องเห็นหน้าเปิด "แตะด้านขวาเพื่อไปต่อ"
927–928	รอเฉย ๆ ต้องไปหน้าถัดไปเอง ("เดือนนี้ใช้ไปแล้ว" หรือ "ทั้งเดือนใช้ไป") ภายใน 9 วินาที
929–933	กดถัดไป ต้องเห็น "เงินไปที่ไหนมากที่สุด"
934–938	กดถัดไปอีก 4 ครั้ง ต้องถึงหน้าสุดท้ายที่มี "ดูอีกครั้ง" และ "เหรียญที่ได้แล้ว"

บรรทัด	ความหมาย
939-940	กด "เสร็จ" ต้องกลับหน้าหลัก

E2E อื่นที่ตรวจตัวเลข FR-2: "Money: add, edit, delete and undo" (ยอดลด ฿123 → เปลี่ยนเป็น ฿150 → เพิ่มเงินเข้า ฿1,000 แล้วยอดตามทุกขั้น), "Voice: the balance follows (-฿25 -฿65 +฿500)", "Reduce motion: the balance appears at once" และ "Idle home screen": เปิดหน้าหลักทิ้งไว้ 9 วินาที แล้วนับเฟรมแอนิเมชันใน 2 วินาทีถัดมา ต้องน้อยกว่า 5 เฟรม (ไม่มีแอนิเมชันวิ่งค้างกินแบตเตอรี่)

เช็กलिस्टทดสอบเองบนมือถือ

#	ทำ	ต้องเห็น
1	จดรายจ่าย ฿85 หมวดอาหาร (MT-02)	ยอดคงเหลือลด ฿85, อาหารในภาพรวมเพิ่ม ฿85, แท่ง "วันนี้" สูงขึ้น
2	สแกนสลิปให้ได้แบบร่าง แต่ยังไม่ยืนยัน	ยอดคงเหลือ ภาพรวม กราฟ ปฏิทิน ไม่เปลี่ยน · ยืนยันแล้วจึงเปลี่ยน
3	แตะปุ่มช่วงที่ละปุ่ม	ข้อความใต้ปุ่มเปลี่ยนเป็นช่วงจริง และเงินเข้า/ออก/หมวดเปลี่ยนตาม
4	จดรายการเวลา 23:50 แล้วรอข้ามเที่ยงคืน	วันที่บนหน้าหลักเปลี่ยนเอง "วันนี้" เริ่มที่ ฿0
5	ตั้งงบเดือน แล้วจดจนเกิน 85% และเกินงบ	แถบเปลี่ยนเป็นเหลือง แล้วแดง
6	เปิด "ลดการเคลื่อนไหว" ในเครื่อง	ยอดเงินขึ้นทันที ไม่นับขึ้น กราฟไม่โต

FR-2 เชื่อมกับ FR อื่นยัง

FR-2 รับข้อมูลจากทุก FR และส่งยอดคงเหลือให้ FR-6 กับ FR-5

FR	ความสัมพันธ์	ตรงไหนในโค้ด
FR-1 จดรายการ	ทุกการเพิ่ม/แก้/ลบ เปลี่ยน txs → ภาพรวมจำนวนใหม่ · FR-2 ให้ groupByDay และปฏิทินแก่แถบรายการ · หน้าหลักใช้ TxRow ของ FR-1	<code>src/domain/summary.ts:111</code> , <code>src/app/(tabs)/transactions.tsx:36</code>
FR-3 พุดจด	รายการจากเสียงนับในภาพรวมทันที · สรุปเดือนนับ "พุดจด" จาก VOICE_NOTE	<code>src/domain/recap.ts:100</code>
FR-4 สลิป	แบบร่างไม่นับในตัวเลขใดของ FR-2 จนกว่ายืนยัน · หน้าสแกนใช้ PeriodSummary · ช่วงเวลาใช้ RANGE_DAYS เดียวกัน	<code>src/domain/summary.ts:9</code> , <code>src/ui/PeriodSummary.tsx</code> , <code>src/domain/dates.ts:58</code>
FR-5 โค้ช	โค้ชใช้ยอดคงเหลือ ยอดตามหมวด และค่าเฉลี่ยเป็นบริบทของ AI	<code>src/domain/insights.ts</code>
FR-6 เงินพอถึง	ใช้ยอดคงเหลือของ FR-2 จำนวนจำนวนวัน · แสดงบนการ์ดเขียวเดียวกัน · กราฟ 7 วันวาดเส้นค่าเฉลี่ยของ FR-6	<code>src/data/AppProvider.tsx:449</code>

“ **ประโยชน์ที่สรุป FR-2**

"FR-2 เปลี่ยนรายการดิบหลายร้อยรายการให้เป็นคำตอบไม่กี่ข้อที่ผู้ใช้อยากรู้ทันที: มีเท่าไร เข้า-ออกเท่าไร ไปไหนมากที่สุด วันไหนใช้เยอะ และเทียบเดือนก่อนเป็นยังไง โดยนับเฉพาะที่ยืนยันแล้วและใช้วันไทยเสมอ"

ทำไมฉันออกแบบแบบนี้

เหตุผลของการตัดสินใจแต่ละข้อ

ยอดคงเหลืออยู่บนสุด ตัวใหญ่สุด	เป็นคำถามแรกๆ ที่ผู้ใช้อยากรู้ ลำดับข้อมูลทั้งหน้าเรียงตามคำถาม มีเท่าไร → พอที่วัน → วันนี้ใช้ได้เท่าไร → เงินไปไหน
นับเฉพาะที่ยืนยันแล้ว	ผลจาก AI อาจผิด ถ้านับทันที ตัวเลขจะกระโดดขึ้นลงตามความผิดของ AI ผู้ใช้จะไม่เชื่อตัวเลข (ข้อตกลงใน Charter)
ช่วงแบบ "N วันเต็มรวมวันนี้"	เทียบกันได้ทุกวัน (1 เดือน = 30 วันเสมอ) และใช้นิยามเดียวกับ FR-4 · เขียนช่วงจริงได้ปุ๊บเพื่อไม่ให้รำคาญ
งบเดือนใช้เดือนปฏิทิน	คนตั้งงบเป็นรายเดือน (ค่าขณมเข้าวันที่ 1) ต่างจากปั๊มช่วงโดยตั้งใจ และตั้งชื่อต่างกัน
เปอร์เซ็นต์หมวดเทียบประเภทเดียวกัน	"อาหาร 30% ของรายจ่าย" เข้าใจง่ายกว่า "อาหาร 14% ของเงินทั้งหมดที่ไหลผ่าน"
แสดง 5 หมวดรายจ่าย 3 หมวดรายรับ	การ์ดไม่ยาวเกินจอ และหมวดท้าย ๆ มักเล็กลงไม่มีผลต่อการตัดสินใจ
กราฟ 7 วันมีเส้นค่าเฉลี่ย	ตอบคำถาม "วันไหนใช้มากกว่าปกติ" ได้ทันที และเชื่อมให้ผู้เข้าใช้เข้าใจที่มาของ Money Runway
ปฏิทินเทียบ "วันปกติ" ไม่ใช่บาทตายตัว	ใช้ได้ทั้งคนใช้น้อยและใช้มาก ทุกคนเห็น "วันที่ผิดปกติ" ของตัวเอง
สีเขียวไล่อ่อนเข้มในปฏิทิน	คนตามอดสีอ่านได้ (ใช้ความสว่าง ไม่ใช่เขียว-แดง) และตรวจความต่างสีทั้งโหมดสว่าง/มืด
สรุปเดือนเทียบจำนวนวันเท่ากัน	ยุติธรรม ไม่ชมผิดตอนต้นเดือน
สรุปเดือนเป็นเรื่องเล่าเต็มจอ	ดึงให้ผู้กลับมาดูทุกเดือน (แบบ year-in-review) ไม่ตำหนิแม่ใช้มากขึ้น
ตัวเลขนับขึ้น/กราฟโต แต่เคารพผลการเคลื่อนไหว	ทำให้รู้สึกมีชีวิต แต่ไม่รบกวนคนที่เวียนหัวกับแอนิเมชัน และหยุดเองเพื่อประหยัดแบตเตอรี่
สูตรทั้งหมดอยู่ใน domain และ useMoney	ทดสอบได้โดยไม่ต้องเปิดแอป และทุกหน้าใช้ตัวเลขชุดเดียวกัน

ข้อจำกัด และสิ่งที่จะทำต่อ

สิ่งที่ฉันพบจากการอ่านโค้ดทุกบรรทัดและรันตัวเลขจริง

ข้อจำกัด	ตัวอย่าง (ตรวจด้วยการรันโค้ด)	แก้ยังไง
แถบหมวดแสดงแค่ 5 หมวด ไม่มี "อื่น ๆ"	ช่วง 1 เดือน เดินทาง ฿609 และการเรียน ฿154 หายไป เปอร์เซ็นต์บนจอรวม 93%	รวมหมวดที่เกินเป็นแถว "อื่น ๆ" หรือมีปุ่ม "ดูทุกหมวด"
วันจ่ายก่อนใหญ่ถึง "วันปกติ" ของปฏิทินสูง	ต.ค. วันปกติ ฿566 เพราะค่าหอ ฿3,500 วันอื่นเกือบทั้งหมด เป็นสีอ่อนสุด	ใช้มาตรฐานแทนค่าเฉลี่ย หรือแยก บิลประจำออก
สรุปเดือนบอก "เดือนแรกของการจด" ผิดได้	วันนี้ 12 ต.ค. เดือนก่อนมีรายจ่ายแค่ 20 ก.ย. → 12 วันแรกของ ก.ย. = ฿0 → null → ขึ้น "เดือนแรกของการจด" ทั้งที่จดมาแล้ว	แยกกรณี "เดือนก่อนไม่มีข้อมูลเลย" กับ "มีแต่ไม่อยู่ในช่วงเทียบ"
เปอร์เซ็นต์พุ่งเมื่อเดือนก่อนมีข้อมูลไม่ครบ	ส.ค. เทียบ ก.ค. (ข้อมูลเริ่ม 27 ก.ค.) = +929%	ไม่แสดงเปอร์เซ็นต์ถ้าเดือนก่อนเริ่มจดกลางเดือน
ปิดเป็นจำนวนเต็มแล้วบอก "เท่าเดิมพอดี"	ต.ค. +0.28% แสดง 0% ไอคอนลูกศรลงสีเขียว	ใช้คำว่า "ใกล้เคียงเดิม" เมื่อ $ % < 1$
"จ่ายน้อยสุด" นับตามชื่อเป๊ะ ๆ	"ข้าวมันไก่" กับ "ข้าวมันไก่ป่าแดง" นับแยก	รวมชื่อที่คล้ายกัน หรือนับตามหมวด
กราฟ 7 วันกดดูรายละเอียดไม่ได้	อยากรู้ว่าวันจันทร์ใช้อะไรต้องไปปฏิทินเอง	แตะแล้วเปิดเก็บรายการกรอกรวันนั้น
งบเดือน vs ช่วง 1 เดือน อาจทำให้งง	หน้าเดียวกันเห็น ฿5,091 (งบ) และ ฿10,142 (1 เดือน)	เพิ่มคำอธิบายเล็ก ๆ ใต้การ์ดงบ "นับตั้งแต่วันที่ 1"

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ตอบได้ทุกข้อจากสิ่งที่อ่านในเล่มนี้

1. FR-2 แสดงอะไรบ้าง	ยอดคงเหลือ, เงินเข้า/ออก/สุทธิตามช่วง 5 แบบ, รายจ่าย/รายรับตามหมวด, กราฟรายจ่าย 7 วัน, งบเดือน, รายการล่าสุด, ปฏิทินการใช้จ่าย และสรุปเดือน
2. ยอดคงเหลือคำนวณอย่างไร	ยอดตั้งต้น + ยอดยกมา + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน ด้วย computeBalance ใน summary.ts เรียกผ่าน useMoney
3. ทำไมแบบร่างไม่นับ	เพราะเป็นผลจาก AI ที่ยังไม่ได้ตรวจ ถ้านับแล้ว AI อ่านผิด ตัวเลขทั้งแอปจะผิด ทุกสูตรจึงผ่าน confirmedOnly (TC-09)
4. "1 เดือน" คือตั้งแต่เมื่อไร	30 วันเต็มนับรวมวันนี้ เช่นวันนี้ 9 ต.ค. คือ 10 ก.ย.-9 ต.ค. แอปเขียนช่วงจริงใต้ปุ่ม (TC-08)
5. ทำไมต้องใช้เวลาไทย	รายการตอน 00:30 คืนวันที่ 10 ในเวลา UTC ยังเป็นวันที่ 9 ถ้าใช้ UTC จะไปอยู่ผิดวัน bkkDayKey บวก 7 ชั่วโมงก่อนตัดวัน (TC-03)
6. เปอร์เซ็นต์ของหมวดเทียบกับอะไร	ยอดรวมของประเภทเดียวกัน หมวดรายจ่ายเทียบรายจ่ายรวม หมวดรายรับเทียบรายรับรวม
7. กราฟ 7 วันทำไมมีวันที่เป็น 0	dailyTotals สร้างครบ 7 ช่องก่อนแล้วค่อยเติมยอด วันที่ไม่ใช้เงินจึงเป็น 0 ไม่หายไป (TC-11)
8. เส้นประในกราฟคืออะไร	ค่าเฉลี่ยรายจ่ายต่อวัน 7 วันที่ FR-6 ใช้คำนวณ Money Runway
9. สีของปฏิทินคิดอย่างไร	เทียบรายจ่ายของวันกับ "วันปกติ" (ค่าเฉลี่ยของวันที่มีรายจ่ายในเดือน) ต่ำกว่าครึ่ง = ระดับ 1, ต่ำกว่า 1 เท่า = 2, ต่ำกว่า 2 เท่า = 3, 2 เท่าขึ้นไป = 4 (TC-70)
10. สรุปเดือนเทียบเดือนก่อนอย่างไร	เทียบจำนวนวันเท่ากันจากต้นเดือน เช่นวันที่ 9 เทียบ 9 วันแรกของเดือนก่อน ไม่เทียบทั้งเดือน (TC-69)
11. การ์ดสรุปเดือนขึ้นเมื่อไร	วันที่ 1-7 เสนอเดือนที่แล้ว (ถ้ามีข้อมูล) นอกนั้นเสนอเดือนนี้เมื่อมีรายการยืนยันอย่างน้อย 3 รายการ
12. เปิดแอปค้างข้ามคืน ตัวเลข "วันนี้" ถูกไหม	ถูก AppProvider ตั้งเวลาปลุกเที่ยงคืนไทย และเช็คเมื่อแอปกลับมาหน้าจอ เมื่อ today เปลี่ยน ทุก useMemo ที่มี today คำนวณใหม่ (E2E midnight)
13. ใช้แอปมาเกิน 1 ปี ยอดยังถูกไหม	ถูก Repo โหลดรายการ 400 วันล่าสุด และรวมรายการที่เก่ากว่าเป็น "ยอดยกมา" (carrySatang) บวกเข้ายอดตั้งต้น
14. ถ้าเน็ตหลุดตอนโหลด	การ์ดแดง "โหลดข้อมูลล่าสุดไม่สำเร็จ" พร้อมปุ่มลองอีกครั้ง ตัวเลขเดิมยังแสดงอยู่
15. ทำไมหน้าหลักไม่ซ้ำแม้คำนวณเยอะ	ใช้ useMemo คำนวณใหม่เฉพาะเมื่อข้อมูลที่เกี่ยวข้องของเปลี่ยน และสูตรเป็นการวนรายการครั้งเดียว (ไม่กัฟไฟนรายการ = มิลลิวินาที)
16. กราฟวาดด้วยอะไร	react-native-svg สำหรับเส้นและตัวหนังสือ ส่วนแท่งเป็น View ที่เลื่อนขึ้นด้วย Animated เพื่อให้แอนิเมชันสั้น

17. คนตาบอดใช้หน้านี้ได้ไหม

ได้ การ์ดเขียวอ่านเป็นประโยค "ยอดคงเหลือ ฿... เงินพอใช้อีก ... วัน" กราฟและวันในปฏิทินมีป้ายอ่านจอ และ toast อ่านออกเสียง

แบบฝึกหัดเชิงความเข้าใจ

ทุกข้อตรวจคำตอบด้วยการรันโค้ดจริงแล้ว

- 1. ยอดตั้งต้น ฿2,000 มีรายรับ ฿5,000 และรายจ่าย ฿300, ฿1,200 (ยืนยันแล้ว) กับแบบร่าง ฿800 ยอดคงเหลือเท่าไร
- 2. วันนี้คือ 15 พ.ย. 2569 ช่วง 7 วันและ 1 เดือนเริ่มวันไหน และข้อความใต้ปุ่มเขียนว่าอะไร
- 3. ช่วง 7 วันมีรายจ่าย อาหาร ฿600 เดินทาง ฿300 บันเทิง ฿100 ส่วนแบ่งของแต่ละหมวดเท่าไร และแถบของบันเทิงยาวกี่เปอร์เซ็นต์ของแถบเต็ม
- 4. วันนี้ 15 พ.ย. มีรายจ่าย ฿200 วันที่ 9 พ.ย. และ ฿50 วันนี้ dailyTotals 7 วันได้อะไร
- 5. เดือน พ.ย. มีรายจ่าย 4 วัน: ฿100, ฿300, ฿50, ฿550 วันปกติเท่าไร และแต่ละวันได้ระดับอะไร
- 6. จบ ฿6,000 ใช้ไป ฿5,200 แถบสีอะไร ถ้าใช้ ฿5,000 ละ
- 7. สรุปเดือน ต.ค. ณ 12 ต.ค. ใช้ไป ฿3,300 ส่วน ก.ย. มีรายจ่าย ฿3,000 วันที่ 5 และ ฿5,000 วันที่ 20 เปอร์เซ็นต์เปลี่ยนเท่าไร ถ้า ก.ย. มีแค่ ฿5,000 วันที่ 20 ละ
- 8. มีรายการ 20 ต.ค., 2 พ.ย., 3 พ.ย. ถ้าวันนี้ 5 พ.ย. การ์ดสรุปเดือนเสนอเดือนไหน ถ้าวันนี้ 10 พ.ย. ละ
- 9. รายการเวลา 2026-10-09T17:30:00Z อยู่วันไหนในแถบรายการ

เฉลย

ข้อ	คำตอบ
1	$2,000 + 5,000 - 300 - 1,200 = \text{฿5,500}$ (แบบร่างไม่นับ)
2	7 วัน: 9 พ.ย. → "9 - 15 พ.ย. 2569 · 7 วันเต็ม" · 1 เดือน: 17 ต.ค. → "17 ต.ค. - 15 พ.ย. 2569 · 30 วันเต็ม"
3	อาหาร 60% เดินทาง 30% บันเทิง 10% · แถบบันเทิง = $100 \div 600 = 16.7\%$ ของแถบเต็ม (เทียบหมวดที่มากที่สุด)
4	[9 พ.ย. ฿200, 10-14 พ.ย. ฿0 ทั้ง 5 วัน, 15 พ.ย. ฿50] รวม 7 ช่วง
5	วันปกติ = $1,000 \div 4 = \text{฿250}$ · ฿100 → 0.4 → ระดับ 1 · ฿300 → 1.2 → ระดับ 3 · ฿50 → 0.2 → ระดับ 1 · ฿550 → 2.2 → ระดับ 4
6	$5,200 \div 6,000 = 86.7\% \rightarrow \text{เหลือง}$ · $5,000 \div 6,000 = 83.3\% \rightarrow \text{สีหลัก}$
7	12 วันแรกของ ก.ย. = ฿3,000 → $(3,300 - 3,000) \div 3,000 = +10\%$ · ถ้ามีแค่วันที่ 20 → 12 วันแรก = ฿0 → null → ขึ้น "เดือนแรกของการจด" (ข้อจำกัดที่บันทึกไว้)
8	5 พ.ย. (สัปดาห์แรก) → ตุลาคม · 10 พ.ย. → พ.ย. มีแค่ 2 รายการ → ไม่แสดง (ถ้ามีรายการที่ 3 จึงเสนอพฤศจิกายน)
9	$17:30 \text{ UTC} + 7 = 00:30$ ของ 10 ต.ค.

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงได้เลย

“ สคริปต์ 1 นาที

"FR-2 คือหน้าภาพรวม เปิดแอปมาเห็นยอดคงเหลือตัวใหญ่บนสุดทันที คำนวณจากยอดตั้งต้นบวกรายรับลบรายจ่าย โดยนับเฉพาะรายการที่ยืนยันแล้ว แบบร่างจากสลิปไม่นับจนกว่าผู้ใช้จะตรวจ

ส่วนภาพรวมเลือกช่วงได้ 5 แบบ วันนี้ถึง 1 ปี ทุกช่วงเขียนวันที่จริงไว้ใต้ปุ่ม เช่น 30 วันเต็ม แสดงเงินเข้า เงินออก สุทธิ และรายจ่ายตามหมวดเรียงจากมากไปน้อย มีกราฟ 7 วันพร้อมเส้นค่าเฉลี่ยที่ใช้คำนวณ Money Runway และแถบงบเดือนที่เปลี่ยนเป็นเหลืองเมื่อเกิน 85%

ในแท็บรายการมีปฏิทินไล่สีตามยอดใช้เทียบกับวันปกติของเดือน และมีสรุปเดือนแบบเรียงเล่า ที่เทียบกับเดือนก่อนแบบจำนวนวันเท่ากันเพื่อความยุติธรรม

ทุกสูตรอยู่ใน summary.ts และ recap.ts ใช้เวลาไทยเสมอ และเริ่มวันใหม่เองตอนเที่ยงคืน ทดสอบด้วย unit test TC-03, 08 ถึง 11, 69, 70 และ E2E ที่จำลองนาฬิกาข้ามเที่ยงคืน"

คำศัพท์ของ FR-2

คำ	ความหมาย
Dashboard	หน้าสรุปข้อมูลสำคัญให้เห็นในครั้งเดียว เหมือนหน้าปิดรถ
confirmed / draft	ยืนยันแล้ว (นับ) / แบบร่าง (ยังไม่นับ)
opening balance	ยอดตั้งต้น เงินที่มีตอนเริ่มใช้แอป
carry (ยอดยกมา)	ผลสุทธิของรายการเก่ากว่า 400 วันที่ไม่ได้โหลดมา
range	ช่วงเวลา: วันนี้ 7 วัน 1 เดือน 6 เดือน 1 ปี
net (สุทธิ)	รายรับ – รายจ่าย
share (ส่วนแบ่ง)	ยอดหมวด ÷ ยอดรวมของประเภทนั้น
reduce	คำสั่งทบยอดทีละรายการจนได้ค่าเดียว
useMemo	จำผลคำนวณ คำนวณใหม่เมื่อค่าที่ระบุเปลี่ยน
heatmap	ตารางที่ใช้ความเข้มของสีแทนจำนวน (ปฏิทินการใช้จ่าย)
usual day (วันปกติ)	ค่าเฉลี่ยรายจ่ายของวันที่มีรายจ่ายในเดือนนั้น
recap	สรุปเดือนแบบเรียงเล่า
fair comparison	เทียบจำนวนวันเท่ากัน

คำ	ความหมาย
SVG	การวาดภาพด้วยคำสั่งเส้น/ตัวอักษร คมชัดทุกขนาด
parallax	เอฟเฟกต์ที่ชั้นต่าง ๆ เคลื่อนเร็วไม่เท่ากันตอนเลื่อนจอ
count-up	ตัวเลขวิ่งจากค่าเดิมไปค่าใหม่
reduce motion	การตั้งค่าเครื่องให้ลดแอนิเมชัน แอปต้องเคารพ
single source of truth	คำวนยที่เดียว ทุกหน้าใช้ร่วมกัน (useMoney)
AppState	สถานะของแอป (อยู่หน้าจอ / เบื้องหลัง) ใช้เช็ควินใหม่